POLITECNICO DI TORINO

Master's Degree in COURSE



Master's Degree Thesis

TITLE

Supervisors

Prof. Stefano DI CARLO

Dott. Filippo MAROSTICA

Dott. Alessio CAVIGLIA

Candidate

Emanuele BERNACCHI

Novembre 2026

Summary

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Acknowledgements

ACKNOWLEDGMENTS

"HI"
Goofy, Google by Google

Table of Contents

List of Tables viii

List of Figures x

Acronyms xvii

Introduzione 1

1 Fondamenti teorici e stato dell'arte 2
- 1.1 Event camera e rappresentazione degli eventi . . . 2
  - 1.1.1 Dal contrasto luminoso al flusso di eventi . . . 3
  - 1.1.2 Discretizzazione e significato del clock . . . 5
  - 1.1.3 Conteggi e informazione intra-bin . . . 8
  - 1.1.4 Tempo valido, silenzi e padding . . . 12
- 1.2 Dinamica neuronale e memoria nelle Spiking Neural Networks . . . 12
  - 1.2.1 Integrazione, soglia e reset . . . 13
  - 1.2.2 Decadimento discreto e scala temporale fisica . . . 14
  - 1.2.3 Addestramento con surrogate gradient . . . 17
  - 1.2.4 Memoria neuronale e relazioni tra feature . . . 20
- 1.3 Dalle SNN agli Spiking Transformer . . . 21
- 1.4 QKFormer come riferimento architetturale . . . 29
- 1.5 Strategie per la modellazione temporale esplicita . . . 39
- 1.6 Efficienza neuromorfica e vincoli hardware . . . 44

2 Baseline compatta e metodologia sperimentale 54
- 2.1 Dataset . . . 54
  - 2.1.1 DVS-Lip . . . 55
  - 2.1.2 DVS128 Gesture . . . 55
  - 2.1.3 CIFAR10-DVS . . . 56
  - 2.1.4 THU-EACT-50-CHL . . . 56
  - 2.1.5 Partizioni di sviluppo . . . 56

2.2 Rappresentazione dell'input nella baseline . . . 58
2.3 Architettura della baseline compatta . . . 61
2.3.1 Neurone e convenzioni di calcolo . . . 61
2.3.2 Primo patch embedding . . . 62
2.3.3 Stadio con Q-K Token Attention . . . 63
2.3.4 Secondo patch embedding e stadio con SSA . . . 63
2.3.5 Readout . . . 64
2.3.6 Bilancio di parametri e stato . . . 64
2.3.7 Differenze rispetto a QKFormer . . . 66
2.4 Protocollo di addestramento . . . 67
2.4.1 Ricetta di ottimizzazione . . . 67
2.4.2 Microbatch e batch normalization . . . 67
2.4.3 Verifiche preliminari . . . 68
2.5 Metriche e valutazione . . . 69
2.5.1 Macro-F1 e metriche complementari . . . 69
2.5.2 Selezione del checkpoint e incertezza dei confronti . . . 69
2.5.3 Valutazione sui prefissi . . . 70
2.5.4 Uso delle partizioni riservate . . . 70
2.6 Disegno dei controlli sperimentali . . . 71
2.7 Caratterizzazione computazionale . . . 73
2.8 Sintesi del protocollo di riferimento . . . 74

3 Riallocazione del budget nella baseline compatta 76

4 Il clock dell'input come variabile sperimentale 77
4.1 I limiti della finestra fissa . . . 78
4.1.1 Una coda che non è silenzio . . . 78
4.1.2 Registrazioni più lunghe della finestra . . . 80
4.1.3 Quando il movimento è più rapido del clock . . . 80
4.2 Tre clock a confronto . . . 82
4.2.1 Frame a eventi costanti . . . 82
4.2.2 Durata fissa e lunghezza variabile . . . 84
4.2.3 Risultati sui quattro dataset . . . 85
4.2.4 Assestamento e costante di tempo . . . 89
4.3 Conseguenze per i confronti successivi . . . 89

5 Memoria temporale relazionale 92
5.1 Dal richiamo del passato al confronto tra passato e presente . . . 92
5.2 Memoria causale a intervalli e disponibilità della storia . . . 96
5.2.1 Dai ritardi puntuali alle bande . . . 96
5.2.2 Medie sulla storia disponibile . . . 96

5.2.3 Quanta storia è davvero disponibile 98
5.2.4 Implementazione e costo 100

A 102

Bibliografia 103

List of Tables

1.1 Informazione conservata e principali limiti delle rappresentazioni considerate. 11
1.2 Evoluzione dei meccanismi di attenzione spiking rilevanti per il lavoro. 30
1.3 Evoluzione delle dimensioni della rappresentazione lungo QKFormer. La gerarchia riduce progressivamente il numero dei token e aumenta la dimensionalità dei canali, mentre il tipo di attenzione della configurazione standard varia in funzione del regime N/D. Le dimensioni e la strategia di mixed attention seguono la formulazione originale di QKFormer [17]. 39
1.4 Metriche utilizzabili ai differenti livelli di valutazione. 52

2.1 Sintesi dei dataset e delle partizioni di sviluppo. La partizione riservata è il test ufficiale, tranne che per CIFAR10-DVS, per il quale coincide con l'insieme di holdout. Le durate sono calcolate sulle sole partizioni di sviluppo e riportano il valore mediano e, tra parentesi, il massimo. 58
2.2 Parametri della rappresentazione a finestra fissa. Le ultime due colonne si riferiscono alle partizioni di sviluppo. La frazione di finestra occupata è il rapporto tra la durata mediana delle registrazioni e la durata della finestra. 59
2.3 Struttura della baseline compatta per un ingresso di 128×128 pixel. Le uscite sono indicate come canali $\times$ altezza $\times$ larghezza per ciascun passo. Lo stato conta i potenziali di membrana di tutti i neuroni del blocco. I parametri della testa dipendono dal numero C delle classi, e il totale si riferisce a DVS-Lip. 65
2.4 Confronto tra la configurazione neuromorfica di QKFormer, secondo la descrizione della sezione 1.4, e la baseline compatta. 66
2.5 Ricetta di addestramento comune a tutti i modelli e a tutti i dataset. 67
2.6 Grandezze raccolte nella caratterizzazione computazionale di ciascun checkpoint. 73
2.7 Protocollo di riferimento della sperimentazione. 75

4.1 Calibrazione della rappresentazione a lunghezza variabile. La durata Δ è calibrata sul solo addestramento e applicata invariata alla validazione. I ritardi del mixer temporale sono riscalati per conservare gli orizzonti fisici della finestra fissa, indicati nell'ultima colonna. . . 85
4.2 Proprietà dei tre clock dell'input. 86
4.3 Accuratezza e Macro-F1 [%] del modello di riferimento con i tre clock, sulla validazione di sviluppo, seed 42. Le repliche disponibili con altri seed danno, in Macro-F1, 54,88 e 56,56 per la finestra fissa su DVS-Lip, 86,52 per la finestra fissa e 95,19 per la lunghezza variabile su DVS128 Gesture, 67,24 e 68,08 per la finestra fissa e 67,17 e 68,27 per la lunghezza variabile su CIFAR10-DVS, 48,67 per la finestra fissa su THU-EACT-50-CHL. 87

List of Figures

1.1 Generazione degli eventi in un pixel ideale, su un segnale sintetico con soglia $C = 0{,}2$. In alto, il logaritmo dell'intensità (linea continua), il valore di riferimento del pixel (tratteggio) e la fascia di ampiezza $\pm C$ attorno a esso. Ogni volta che la curva esce dalla fascia il pixel emette un evento, positivo (triangolo rosso) o negativo (triangolo blu), e il riferimento si sposta di C. In basso, la sequenza di eventi che il pixel trasmette. L'oscillazione centrale, più piccola della soglia, non produce alcun evento. 4

1.2 Quattro modi di suddividere lo stesso flusso sintetico, che contiene una raffica di attività tra 0,35 e 0,55 s e termina a circa 1,17 s. In alto, la frequenza degli eventi. Sotto, i passi prodotti da ciascuna regola, colorati in base al numero di eventi che contengono. La finestra fissa lascia una coda di passi vuoti (tratteggiati), la registrazione intera termina con l'ultimo evento, il numero di passi fisso allarga i bin per coprire la durata, la partizione a eventi costanti produce passi con lo stesso contenuto ma di durata molto diversa, stretti durante la raffica e larghi altrove. 7

1.3 Tre rappresentazioni degli stessi 100 ms di una registrazione di DVS128 Gesture, durante una rotazione del braccio sinistro (circa 9 600 eventi). (a) Event frame, con i conteggi degli eventi positivi e negativi in due canali separati. (b) Voxel grid con $B = 5$ bin temporali, in cui la distribuzione dell'attività cambia da un bin all'altro, con i valori positivi in rosso e quelli negativi in blu. (c) Time surface dell'ultimo evento di entrambe le polarità, valutata alla fine dell'intervallo, in cui la posizione più recente del movimento appare come una scia. 10

1.4 Integrazione, soglia e soft reset in un neurone IF (tratteggio arancione) e in un neurone LIF con $\lambda = 0{,}5$ (linea blu), entrambi con soglia $V_{\text{th}} = 1$. I due neuroni ricevono tre ingressi da 0,6 (in basso), consecutivi in (a) e distanziati in (b). I marcatori indicano il potenziale H_k appena caricato, i tratti verticali il soft reset che segue uno spike, la fila in alto gli spike emessi. Il neurone IF, che non dimentica, emette uno spike in entrambi i casi. Il LIF lo emette soltanto quando gli ingressi sono abbastanza ravvicinati da sommarsi prima che il potenziale decada. 15

1.5 Principio del surrogate gradient, in funzione della distanza $z = H_k - V_{\text{th}}$ tra potenziale e soglia. (a) Nel passaggio in avanti lo spike è prodotto dal gradino Θ (linea nera). La sigmoide $\sigma(4z)$ (tratteggio) non viene mai calcolata, ma è la funzione di cui il passaggio all'indietro usa la derivata. (b) La derivata del gradino è nulla ovunque tranne nell'origine, dove non è definita (freccia), e non trasmette alcun gradiente. La surrogata g_α la sostituisce con una campana centrata sulla soglia, tanto più alta e stretta quanto maggiore è α. Il valore evidenziato, $\alpha = 4$, è quello adottato in questo lavoro. 18

1.6 Progressiva riformulazione della self-attention nel dominio spiking. Spikformer introduce Query, Key e Value in forma spiking ed elimina la softmax. Spike-driven Transformer sostituisce inoltre la correlazione tra coppie di token con operazioni di mascheramento e accumulo, privilegiando una computazione lineare e spike-driven. . . 25

1.7 Principali passaggi concettuali nell'evoluzione degli Spiking Transformer rilevanti per questa tesi. I diversi lavori affrontano progressivamente compatibilità della self-attention con gli spike, purezza event-driven, residual learning e scalabilità del meccanismo di attenzione. 28

1.8 Principio della Q-K Attention. QKTA comprime la Query lungo i canali e costruisce una maschera binaria sui token, mentre QKCA la comprime lungo i token e costruisce una maschera sui canali. In entrambi i casi l'attenzione evita la costruzione della matrice $N \times N$ delle relazioni tra coppie di token. 33

1.9 Struttura gerarchica di QKFormer. La risoluzione spaziale viene progressivamente ridotta attraverso SPEDS, mentre aumenta la dimensionalità dei canali. Nella configurazione predefinita QKTA viene utilizzata negli stage ad alta numerosità di token e SSA nello stage finale. La dimensione temporale T viene preservata lungo il backbone. 36

1.10 Tre modalità differenti di integrare il passato. La dinamica neuronale comprime ricorsivamente la storia nello stato corrente, l'attenzione spazio-temporale confronta direttamente feature appartenenti a più timestep, il temporal mixing mediante ritardi accede selettivamente a istanti passati diversi senza costruire una correlazione spazio-temporale completa. 41

1.11 La sparsità delle attivazioni produce un beneficio hardware soltanto quando l'architettura evita realmente le operazioni associate agli elementi nulli. Il vantaggio netto dipende dal rapporto tra computazione risparmiata e overhead introdotto dalla gestione della rappresentazione sparsa. 47

1.12 Possibile decomposizione funzionale di uno Spiking Transformer su FPGA. Oltre alle unità di calcolo, il costo hardware comprende memoria dei pesi, stato neuronale persistente, buffer temporali, routing degli spike e logica necessaria a sfruttarne la sparsità. . . . 49

2.1 Esempi di campioni dei quattro dataset. Per ciascuno sono mostrati alcuni frame di conteggio, con gli eventi ON e OFF in colori distinti, e l'andamento nel tempo della frequenza di eventi dell'intera registrazione. 57

2.2 Costruzione della rappresentazione a finestra fissa. Il flusso di eventi viene suddiviso in $T = 40$ bin della stessa durata a partire dall'origine del campione, e in ogni bin si contano separatamente gli eventi ON e OFF di ciascun pixel. I bin successivi all'ultimo evento restano vuoti ma fanno parte della sequenza elaborata dalla rete. In basso, la distribuzione delle durate di sviluppo di ciascun dataset rispetto alla finestra adottata. 60

2.3 Architettura della baseline compatta. Il primo patch embedding riduce la risoluzione di un fattore otto con un ramo principale di quattro convoluzioni e una scorciatoia diretta dalla prima mappa di feature. Seguono un blocco con Q-K Token Attention, un secondo patch embedding e un blocco con spiking self-attention. Il readout media le feature finali su spazio e tempo prima della testa lineare. Le dimensioni indicate si riferiscono a un ingresso di 128×128 pixel. 61

2.4 Uso delle partizioni nel protocollo sperimentale. Addestramento e selezione del checkpoint impiegano esclusivamente le partizioni di sviluppo, ricavate dal training ufficiale. La partizione riservata resta esclusa dallo sviluppo e viene impiegata soltanto per analisi diagnostiche a posteriori su checkpoint già selezionati. 71

2.5 Tipologie di controllo sperimentale. L'ablazione riaddestrata confronta due modelli addestrati da zero che differiscono per un solo componente. L'intervento sul checkpoint modifica in sola inferenza un modello già addestrato. Il controllo di capacità sostituisce il componente con un'alternativa di costo comparabile ma priva della struttura in esame. 72

4.1 Coda vuota della rappresentazione a finestra fissa per il modello di riferimento, sulla validazione di sviluppo. (a) Frazione dei 40 passi successivi al bin che contiene l'ultimo evento, per ciascun campione, con mediana e intervallo interquartile. (b) Quota dei passi, e quindi delle MAC dense, e quota delle operazioni sinaptiche spese dopo l'ultimo evento, sommate sull'intera validazione. Checkpoint con seed 42, valutati in singola precisione. 79

4.2 Dipendenza dalla coda appresa dal modello addestrato con la finestra fissa. Ogni riga confronta l'accuratezza dello stesso checkpoint quando elabora l'intera finestra e quando l'elaborazione si interrompe al bin che contiene l'ultimo evento. A destra la differenza in punti percentuali. Seed 42, validazione di sviluppo, valutazione in singola precisione. 79

4.3 Aliasing temporale su una rotazione antioraria del braccio sinistro di DVS128 Gesture (validazione di sviluppo). (a) Giri ricostruiti dall'angolo del baricentro degli eventi di ciascun passo, per una traccia di riferimento a 20 ms e per i tre clock. (b) Variazione angolare della traccia di riferimento tra i centri di passi consecutivi. In alto, il numero di intervalli che raggiungono o superano il mezzo giro. Il baricentro è un'approssimazione della posizione del braccio e il campione è stato scelto a posteriori, sicché la figura illustra il meccanismo senza stimarne la frequenza. 81

4.4 Matrici di confusione ristrette alle quattro classi di rotazione di DVS128 Gesture, per il modello di riferimento addestrato con i tre clock (validazione di sviluppo, seed 42). Le righe indicano la classe vera, le colonne la classe predetta, con D e S per il braccio destro e sinistro e le frecce per il verso. Le celle tratteggiate corrispondono alle inversioni di verso a parità di braccio, la colonna *altro* alle predizioni di classi diverse dalle rotazioni. 82

4.5 La registrazione di DVS128 Gesture della figura 4.3, circa 7,5 s di rotazione, suddivisa secondo i tre clock. In alto, la frequenza degli eventi su intervalli di 10 ms. Sotto, i passi delle tre rappresentazioni, con i passi privi di eventi in tinta chiara. La finestra fissa usa 40 bin da 500 ms, dei quali soltanto 16 contengono eventi. I gruppi a eventi costanti e i bin a lunghezza variabile coprono esattamente la registrazione. 83

4.6 Macro-F1 del modello di riferimento con i tre clock, sulla validazione di sviluppo. Il simbolo pieno indica l'esecuzione con seed 42, i simboli vuoti le repliche con altri seed disponibili. Ogni pannello ha una propria scala verticale. Su THU-EACT-50-CHL la rappresentazione a eventi costanti non ha superato le verifiche preliminari. 86

4.7 Accuratezza del modello di riferimento con la finestra fissa e con la lunghezza variabile, per quartili di durata della registrazione. Le barre verticali indicano l'intervallo di confidenza al 95% di Wilson, ampio dove i quartili contano pochi campioni, come i 66 di DVS128 Gesture. Seed 42, validazione di sviluppo, valutazione in singola precisione. 88

4.8 Assestamento e costante di tempo su DVS128 Gesture. (a) Frazione del potenziale che sopravvive in assenza di ingresso, in funzione del tempo fisico, per un neurone isolato al di sotto della soglia. Con $\tau_d = 2$ il potenziale si dimezza a ogni passo e, con i bin da 164 ms, svanisce circa tre volte più in fretta che con quelli da 500 ms. Con $\tau_d \approx 4{,}91$ la curva torna a coincidere con quella della finestra fissa ogni 500 ms. (b) Macro-F1 della finestra fissa, della lunghezza variabile e dei due controlli riaddestrati, seed 42. 90

5.1 Dal richiamo al confronto. (a) Il mixer a ritardi del modello di riferimento proietta le correnti di alcuni passi precedenti e le somma alla corrente presente, sicché passato e presente contribuiscono in modo indipendente. (b) Il rilevatore di correlazione di Hassenstein e Reichardt moltiplica il segnale ritardato di un recettore per quello istantaneo dell'altro e sottrae il prodotto speculare, ottenendo una risposta il cui segno dipende dal verso del moto. (c) Il mixer proposto riassume il passato in bande, lo richiama linearmente e lo confronta con il presente attraverso due proiezioni apprese, con un ruolo analogo a quello dei due recettori. 94

5.2 Ritardi del mixer a ritardi e bande della memoria a intervalli, in tempo fisico, con la rappresentazione a lunghezza variabile. I cerchi indicano i quattro ritardi letti dal mixer, i trattini grigi tutti i passi del passato fino al ritardo massimo. Le bande, delimitate dagli stessi ritardi, coprono senza lacune l'intero orizzonte e si allargano allontanandosi dal presente. A destra, quanti dei passi del passato vengono letti dal mixer a ritardi. 97
5.3 Storia disponibile nella memoria a intervalli. (a) Le bande di DVS-Lip al passo $t = 5$, vicino all'inizio della sequenza, e al passo $t = 30$, a regime. I passi tratteggiati precedono il primo evento e non esistono. Sopra e sotto ciascuna banda, il numero di passi osservati rispetto alla sua larghezza. (b) Frazione dei passi della validazione di sviluppo in cui ciascuna banda dispone di una storia completa e, in piccolo, frazione in cui è del tutto vuota. 99

Acronyms

AI

artificial intelligence

Introduzione

L'introduzione sarà redatta al termine della revisione dei capitoli e del consolidamento dei risultati. La traccia seguente ne definisce i contenuti, senza anticipare i contributi definitivi.

- **Contesto e motivazione.** Presentare il problema dell'elaborazione temporale di dati event-based e le ragioni per studiarlo con Spiking Transformer compatti.
- **Problema e domande di ricerca.** Definire quali limiti vengono affrontati e quali relazioni si intendono studiare tra rappresentazione degli eventi, memoria, interazione temporale e capacità del modello.
- **Obiettivi e impostazione metodologica.** Delineare il regime di compattezza, il ruolo dei confronti controllati e la distinzione tra prestazioni predittive e costo computazionale.
- **Contributi ed evidenze.** Enunciare soltanto i contributi sostenuti dai risultati conclusi, precisandone l'ambito di validità e i principali limiti. Distinguere la caratterizzazione software dalle prospettive di implementazione hardware.
- **Organizzazione della tesi.** Presentare la struttura effettiva del documento una volta stabilizzata, evitando di duplicare il background e la metodologia.

Capitolo 1

Fondamenti teorici e stato dell'arte

Questo capitolo raccoglie i fondamenti necessari a comprendere la progettazione di uno Spiking Transformer compatto per dati neuromorfici. L'esposizione segue il cammino dell'informazione, dal sensore alla sua rappresentazione discreta, dalla memoria del singolo neurone alle relazioni tra feature, fino al costo computazionale e di memoria che ogni scelta comporta.

La sezione 1.1 affronta il primo passaggio obbligato, la conversione del flusso asincrono di eventi in tensori regolari, e mostra come questa operazione, apparentemente preliminare, costituisca già una forma di modellazione. La sezione 1.2 descrive la memoria intrinseca dei neuroni spiking e i limiti di uno stato che riassume ricorsivamente il passato. Le sezioni 1.3 e 1.4 ricostruiscono l'incontro tra attenzione e computazione spiking fino a QKFormer, l'architettura da cui deriva la baseline compatta, distinguendo la memoria dei neuroni dalle interazioni esplicite tra istanti diversi. La sezione 1.5 esamina due modi di rendere esplicito l'accesso alla storia, l'attenzione spazio-temporale e il mixing mediante ritardi, mentre la sezione 1.6 ne discute le conseguenze su una piattaforma con risorse limitate.

Non si tratta di una rassegna esaustiva. Ogni contributo discusso è stato selezionato perché orienta una scelta progettuale compiuta nel seguito del lavoro, ed è presentato in funzione di essa.

1.1 Event camera e rappresentazione degli eventi

Una fotocamera convenzionale scatta fotografie a intervalli regolari, per esempio trenta al secondo, e ogni fotografia registra la luminosità di tutti i pixel, anche di quelli in cui non è cambiato nulla. Una event camera funziona in modo opposto. Ogni pixel resta in silenzio finché la luce che riceve non cambia abbastanza, e in

quel momento segnala il cambiamento con un *evento*. Una scena immobile non produce quasi nulla, mentre una mano che si muove produce una scia di eventi lungo i propri contorni, nell'istante esatto in cui si sposta.

Per elaborare questo segnale con una rete neurale, che lavora su tensori di dimensioni fisse, occorre raccogliere gli eventi in una struttura regolare. Questa sezione descrive prima il segnale prodotto dal sensore e poi le scelte necessarie per trasformarlo in un tensore. Sono scelte tutt'altro che neutre, perché decidono quali informazioni arrivano alla rete e quali vanno perse prima ancora che l'elaborazione cominci.

1.1.1 Dal contrasto luminoso al flusso di eventi

Nel modello ideale di un Dynamic Vision Sensor (DVS) ogni pixel lavora indipendentemente dagli altri e osserva il logaritmo dell'intensità luminosa che lo colpisce, $L(x, y, t) = \log I(x, y, t)$ [1]. Il pixel conserva un valore di riferimento, cioè il livello di L al momento del suo ultimo evento, e lo confronta di continuo con il valore attuale. Quando la differenza raggiunge una soglia $C > 0$, verso l'alto o verso il basso, il pixel emette un evento e aggiorna il riferimento. In formule, l'evento i-esimo viene emesso nell'istante t_i in cui

$$L(x_i, y_i, t_i) - L(x_i, y_i, t_i^{\text{ref}}) = p_i\, C, \tag{1.1}$$

dove t_i^{ref} è l'istante a cui risale il riferimento, cioè l'evento precedente dello stesso pixel o la sua inizializzazione, e la polarità p_i vale $+1$ se la luce è aumentata e -1 se è diminuita [2]. Ogni evento è quindi descritto da quattro numeri,

$$e_i = (x_i, y_i, t_i, p_i), \tag{1.2}$$

la colonna e la riga del pixel, l'istante e la polarità. Un evento con $x_i = 64$, $y_i = 30$, $t_i = 1\,250\,400\,\mu$s e $p_i = +1$, per esempio, dice che il pixel della colonna 64 e della riga 30 è diventato più luminoso 1,25 secondi dopo l'inizio della registrazione.

La figura 1.1 mostra il meccanismo su un singolo pixel, attraversato prima da un oggetto chiaro e poi da uno scuro. Finché la luce cresce, ogni incremento pari a C produce un evento positivo, quando cala ogni decremento pari a C ne produce uno negativo. Due aspetti meritano attenzione. La soglia agisce sulla variazione accumulata dall'ultimo evento, non sulla velocità del cambiamento, sicché nel modello ideale una variazione lenta e una rapida della stessa entità producono lo stesso numero di eventi, soltanto distribuiti in tempi diversi. Le oscillazioni più piccole della soglia, invece, non producono nulla.

Perché proprio il logaritmo? Perché la differenza tra due logaritmi è il logaritmo del rapporto,

$$\Delta L = \log \frac{I(x_i, y_i, t_i)}{I(x_i, y_i, t_i^{\text{ref}})}, \tag{1.3}$$

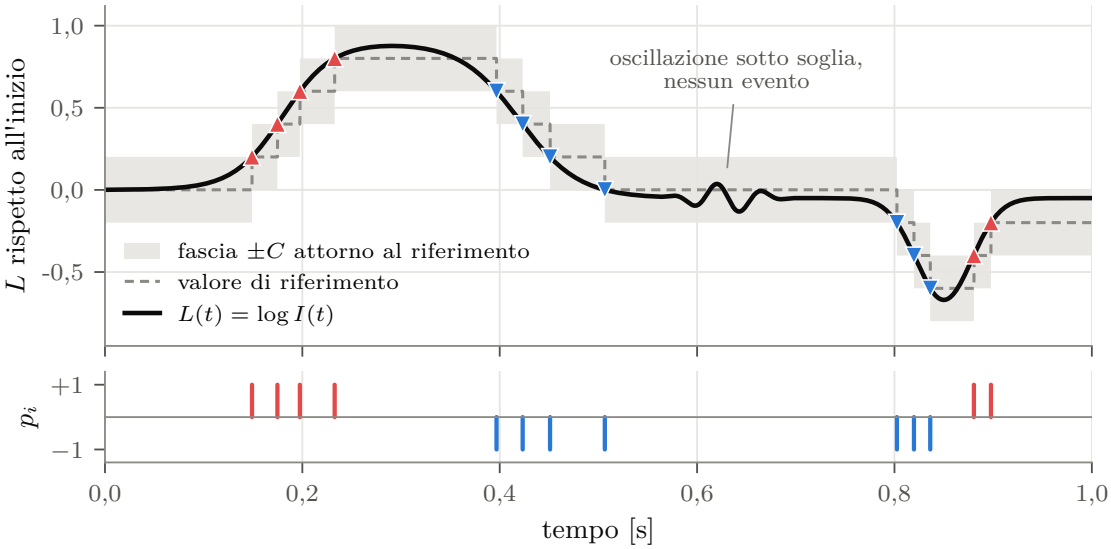


Figura 1.1: Generazione degli eventi in un pixel ideale, su un segnale sintetico con soglia $C = 0{,}2$. In alto, il logaritmo dell'intensità (linea continua), il valore di riferimento del pixel (tratteggio) e la fascia di ampiezza $\pm C$ attorno a esso. Ogni volta che la curva esce dalla fascia il pixel emette un evento, positivo (triangolo rosso) o negativo (triangolo blu), e il riferimento si sposta di C. In basso, la sequenza di eventi che il pixel trasmette. L'oscillazione centrale, più piccola della soglia, non produce alcun evento.

e il pixel reagisce quindi alle variazioni relative della luce, non a quelle assolute. Con una soglia $C = 0{,}2$, scelta qui a titolo illustrativo, un evento positivo richiede un aumento della luce di circa il 22%, poiché $e^{0{,}2} \approx 1{,}22$. Vale allo stesso modo per un pixel in piena luce che passa da 1000 a 1220 unità e per uno in penombra che passa da 10 a 12,2. La stessa scena produce così eventi simili in condizioni di illuminazione molto diverse, ed è una delle ragioni dell'ampia gamma dinamica di questi sensori [2].

La polarità, va precisato, dice soltanto se in quel pixel la luce è aumentata o diminuita, non in quale direzione si muova qualcosa. Si pensi a una barra scura che scorre verso destra su uno sfondo chiaro. I pixel raggiunti dal suo bordo anteriore si scuriscono ed emettono eventi negativi, quelli lasciati dal bordo posteriore tornano chiari ed emettono eventi positivi. Se la barra scorresse verso sinistra, ogni pixel emetterebbe esattamente la stessa sequenza, e la direzione si potrebbe leggere soltanto nell'ordine con cui si attivano pixel vicini. Allo stesso modo, un evento non implica necessariamente un movimento, perché anche una lampada che sfarfalla fa cambiare la luce di molti pixel insieme. Il sensore reale, infine, si allontana dal modello ideale in vari modi. Le soglie positiva e negativa possono differire tra loro e variare da pixel a pixel, e il rumore, la banda limitata del fotorecettore e un breve periodo refrattario dopo ogni evento aggiungono altri scostamenti [2].

Una registrazione è dunque una lunga sequenza di eventi in ordine di tempo,

$$\mathcal{E} = \{e_i\}_{i=1}^{N}, \qquad t_i \leq t_{i+1}. \tag{1.4}$$

Il numero N degli eventi non dipende dalla durata ma da quanto accade nella scena. Nei dataset usati in questo lavoro, per esempio, una parola pronunciata in circa un secondo produce in media poco più di diecimila eventi, mentre un'immagine di CIFAR10-DVS fatta muovere davanti al sensore per un tempo simile ne produce circa duecentomila (sezione 2.1). Anche all'interno della stessa registrazione possono alternarsi lunghi silenzi e raffiche improvvise, e chi elabora il segnale deve saper gestire gli uni e le altre.

1.1.2 Discretizzazione e significato del clock

Una rete neurale spiking elabora l'ingresso a passi discreti. Per presentarle un flusso di eventi bisogna quindi decidere che cosa entri in ciascun passo, cioè come tagliare il flusso in pezzi. La regola che associa ogni passo a un intervallo di tempo, o più in generale a un gruppo di eventi, prende il nome di *clock* dell'input. Le regole possibili sono diverse, e la figura 1.2 le confronta sullo stesso flusso.

La più semplice divide il tempo in intervalli consecutivi della stessa durata Δt, detti bin, a partire da un'origine t_0. Il bin k-esimo raccoglie gli eventi caduti nel suo intervallo,

$$\mathcal{E}_k = \Big\{ e_i \in \mathcal{E} \;\Big|\; t_0 + k\Delta t \leq t_i < t_0 + (k+1)\Delta t \Big\}. \tag{1.5}$$

Con $t_0 = 0$ e $\Delta t = 50\,\text{ms}$, un evento all'istante $t_i = 123\,\text{ms}$ finisce nel bin $k = 2$, quello che va da 100 a 150 ms. Il bordo sinistro è incluso e quello destro escluso, così che un evento che cade esattamente a 150 ms appartenga al bin successivo e mai a due bin insieme. In questo schema ogni passo dura lo stesso tempo, mentre il numero di eventi che contiene cambia liberamente da un passo all'altro.

Fissata la durata dei bin, resta da decidere quanti passi elaborare. La soluzione più comune è una finestra di durata prefissata, $D_{\text{win}} = T\Delta t$, che produce sempre T passi. Con bin da 50 ms e una finestra di 2 s i passi sono 40. Una registrazione di 1,2 s ne riempie i primi 24 e lascia vuoti gli ultimi 16, mentre di una registrazione di 2,5 s vanno persi gli ultimi 0,5 s.

In alternativa si può conservare l'intera registrazione, dal primo evento, che fa da origine t_0, all'ultimo, che cade all'istante t_{last}, mantenendo bin della stessa durata. Il numero di passi diventa allora

$$T = \left\lfloor \frac{t_{\text{last}} - t_0}{\Delta t} \right\rfloor + 1. \tag{1.6}$$

La parte intera conta i bin completamente trascorsi prima dell'ultimo evento, e l'unità aggiunta fa posto al bin che lo contiene. Con bin da 50 ms e un ultimo evento a 1199 ms si ottengono 24 passi. Se l'ultimo evento cadesse esattamente a 1200 ms, sul bordo sinistro di un nuovo bin, i passi sarebbero 25, ed è proprio l'unità aggiunta a garantire che quell'evento trovi posto. In questo schema la durata dei bin è la stessa per tutte le registrazioni, e a cambiare è la lunghezza della sequenza.

Una terza possibilità fa il contrario, fissa il numero di passi T e adatta la durata dei bin a ciascuna registrazione n, di durata D_n,

$$\Delta t_n = \frac{D_n}{T}. \tag{1.7}$$

Con $T = 40$, un gesto di 4 s viene diviso in bin da 100 ms e uno di 8 s in bin da 200 ms. Il clock cambia così da un campione all'altro. Una distanza di cinque passi corrisponde a mezzo secondo nel primo gesto e a un secondo intero nel secondo, e la rete non ha modo di saperlo. Fissare il numero dei bin, in altre parole, significa non fissare la risoluzione temporale.

Si può infine rinunciare del tutto a misurare il tempo e contare gli eventi. Una prima variante chiude un gruppo ogni M eventi, nell'ordine in cui arrivano. Una seconda divide l'intera registrazione in T gruppi che contengono lo stesso numero di eventi, a meno di uno,

$$n_k \in \left\{ \left\lfloor \frac{N}{T} \right\rfloor, \left\lceil \frac{N}{T} \right\rceil \right\}, \qquad \sum_{k=1}^{T} n_k = N, \tag{1.8}$$

dove n_k è il numero di eventi del gruppo k. Con $N = 10\,003$ eventi e $T = 40$ gruppi, per esempio, tre gruppi ricevono 251 eventi e gli altri 37 ne ricevono 250. In questo

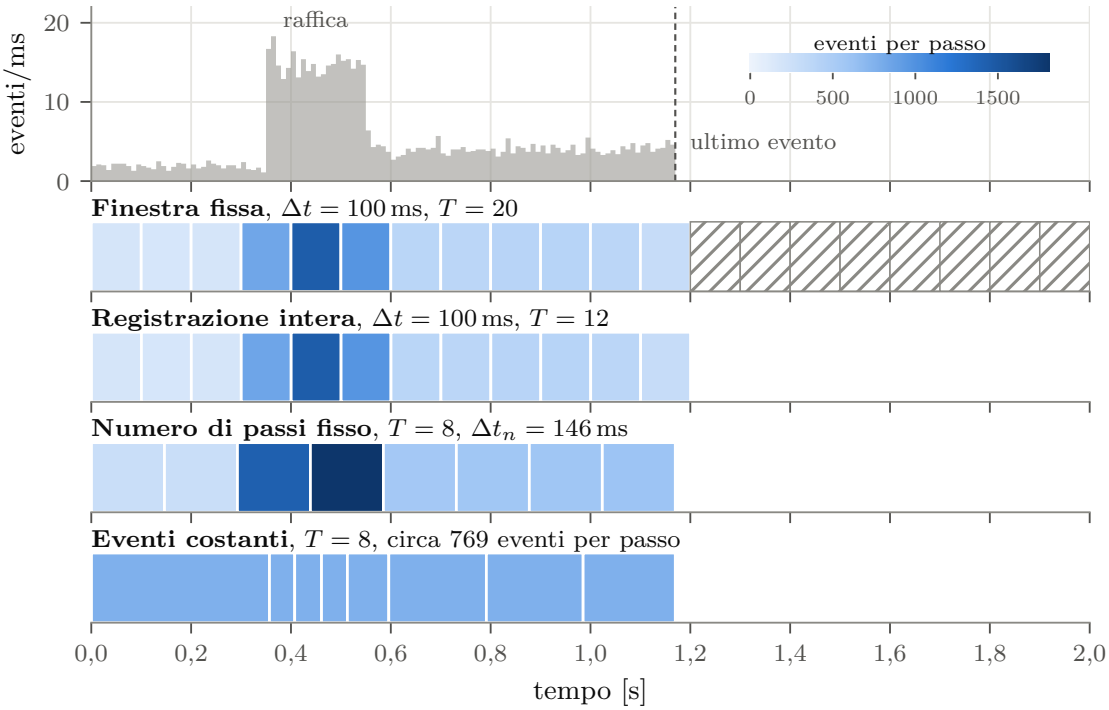


Figura 1.2: Quattro modi di suddividere lo stesso flusso sintetico, che contiene una raffica di attività tra 0,35 e 0,55 s e termina a circa 1,17 s. In alto, la frequenza degli eventi. Sotto, i passi prodotti da ciascuna regola, colorati in base al numero di eventi che contengono. La finestra fissa lascia una coda di passi vuoti (tratteggiati), la registrazione intera termina con l'ultimo evento, il numero di passi fisso allarga i bin per coprire la durata, la partizione a eventi costanti produce passi con lo stesso contenuto ma di durata molto diversa, stretti durante la raffica e larghi altrove.

schema ogni passo contiene la stessa quantità di attività, ma la sua durata dipende da quanto è intensa la scena. Una raffica concentrata in un decimo di secondo può riempire molti gruppi, mentre un secondo di quiete può starne in uno solo. La seconda variante richiede di conoscere in anticipo il numero totale N degli eventi, e si può applicare soltanto a registrazioni già concluse, cioè offline. La prima non ha questo limite, perché ogni gruppo si chiude non appena raggiunge M eventi. In entrambi i casi, però, una distanza misurata in gruppi non corrisponde a un intervallo di tempo fisso.

Quando i passi seguono il numero di eventi, la loro durata non è più fissata in anticipo ma diventa una proprietà di ciascun gruppo, misurabile come intervallo (*span*) tra il suo primo e il suo ultimo evento. Se la rete riceve soltanto i conteggi, però, questa durata le resta invisibile. Un gruppo di 250 eventi raccolti in 5 ms e uno raccolto in mezzo secondo occupano entrambi esattamente un passo.

La scelta del clock non è dunque un dettaglio di preprocessing. Da essa dipende il significato fisico di ogni ritardo espresso in passi e di ogni costante di decadimento della rete. Dimezzare Δt su una stessa registrazione raddoppia il numero dei passi, dimezza in media i conteggi di ciascun bin e fa sì che un neurone che perde una frazione fissa del proprio potenziale a ogni passo dimentichi due volte più in fretta in tempo reale. I protocolli adottati in questo lavoro sono descritti nel capitolo 2 e messi a confronto nel capitolo 4.

1.1.3 Conteggi e informazione intra-bin

Una volta suddiviso il flusso, bisogna decidere che cosa conservare di ciascun bin. La rappresentazione più semplice conta, per ogni pixel e per ogni bin, quanti eventi di ciascuna polarità vi sono caduti,

$$F_k^p(x,y) = \sum_{e_i \in \mathcal{E}_k} \mathbf{1}[x_i = x,\ y_i = y,\ p_i = p], \qquad F_k \in \mathbb{N}_0^{2\times H\times W}. \tag{1.9}$$

L'indicatore $\mathbf{1}[\cdot]$ vale 1 quando la condizione tra parentesi è vera e 0 altrimenti, quindi la somma conta semplicemente gli eventi del bin k caduti nel pixel (x, y) con polarità p. Il risultato è un'immagine a due canali, uno per gli eventi positivi e uno per quelli negativi, spesso chiamata *event frame* (figura 1.3a). Se in un bin un pixel riceve tre eventi positivi e uno negativo, il canale positivo vale 3 e quello negativo 1.

Esistono versioni più povere della stessa idea. Una versione binaria, $\mathbf{1}[F_k^p(x,y) > 0]$, registra soltanto se il pixel è stato attivo, e nell'esempio darebbe 1 in entrambi i canali. Una versione a un solo canale somma gli eventi con il loro segno, e darebbe $3 - 1 = 2$. Quest'ultima ha un difetto evidente, perché due eventi positivi e due negativi nello stesso pixel si annullano e producono zero, come se non fosse accaduto nulla. Tenere separate le polarità evita il problema.

Tutte queste versioni condividono però un limite più sottile. Una somma non dipende dall'ordine degli addendi, e i conteggi di un bin non cambiano se gli eventi al suo interno vengono rimescolati. Si pensi a un oggetto chiaro che attraversa rapidamente un pixel scuro. Il pixel prima si illumina e poi torna scuro, ed emette un evento positivo seguito da uno negativo. Un oggetto scuro che attraversa un pixel chiaro produce la sequenza opposta. Se entrambi gli eventi cadono nello stesso bin, i due casi danno esattamente gli stessi conteggi, uno per canale, e nessuna rete, per quanto sofisticata, potrà distinguerli sulla base di quel solo bin. La rappresentazione fissa un limite all'informazione disponibile, che va tenuto distinto dalla capacità della rete di sfruttare quella che rimane.

La *voxel grid* recupera parte dell'informazione temporale perduta [3]. Si considera un pacchetto di eventi consecutivi, elaborato come un'unica unità, e se ne riscalano

i timestamp in modo che il primo evento valga 0 e l'ultimo $B-1$,

$$t_i^* = (B-1)\frac{t_i - t_{\min}}{t_{\max} - t_{\min}}, \tag{1.10}$$

dove B è il numero di bin temporali desiderati e il pacchetto deve avere durata non nulla, $t_{\max} > t_{\min}$. Ogni evento viene poi distribuito tra i due bin più vicini alla sua posizione t_i^*,

$$\mathcal{V}(b,x,y) = \sum_i p_i \max\bigl(0,\, 1 - |b - t_i^*|\bigr)\mathbf{1}[x_i = x,\ y_i = y], \qquad b \in \{0,\dots,B-1\}. \tag{1.11}$$

Il peso $\max(0, 1-|b-t_i^*|)$ vale 1 quando l'evento cade esattamente sul bin b, decresce in modo lineare con la distanza e si annulla a un bin di distanza. Con $B = 5$, un evento positivo con $t_i^* = 2{,}3$ aggiunge 0,7 al bin 2 e 0,3 al bin 3, e niente agli altri. L'evento non viene attribuito tutto a un bin o tutto all'altro, e la sua posizione temporale sopravvive, sfumata, nella ripartizione dei pesi (figura 1.3b). La formulazione completa interpola anche nello spazio, quando la correzione della distorsione ottica produce coordinate non intere [3].

Due dettagli vanno tenuti presenti. La somma usa la polarità con il suo segno, e può quindi produrre le stesse cancellazioni dell'event frame a un canale, che volumi separati per eventi positivi e negativi eviterebbero. Inoltre la scala temporale è relativa a ciascun pacchetto. Con $B = 5$, un intervallo di 10 ms occupa meno di mezzo bin in un pacchetto lungo 100 ms e ben due bin in uno lungo 20 ms, e non va confuso con un clock fisico comune a tutti i campioni.

La *time surface* adotta un punto di vista diverso. Invece di contare gli eventi, descrive quanto sia recente l'attività di ciascun pixel. Nella forma più semplice,

$$S_p(x,y,t) = \exp\left[-\frac{t - t^p_{\text{last}}(x,y)}{\tau_s}\right], \tag{1.12}$$

dove $t^p_{\text{last}}(x,y)$ è l'istante dell'ultimo evento di polarità p arrivato al pixel prima di t, e il valore è zero dove non è ancora arrivato nulla. Un pixel appena attivato vale circa 1, e il suo valore decade poi in modo esponenziale con costante τ_s. Con $\tau_s = 50$ ms, un pixel attivato 10 ms prima vale circa 0,82, uno attivato 100 ms prima circa 0,14. Il risultato somiglia alla scia di una cometa, intensa dove il movimento è appena passato e sempre più tenue alle sue spalle (figura 1.3c). È la variante elementare di una famiglia più ampia. HATS, per esempio, somma i contributi esponenziali di più eventi in un intorno spazio-temporale e li aggrega in descrittori [4].

Event frame, voxel grid e time surface riassumono dunque aspetti diversi della stessa storia, la quantità dell'attività, la sua collocazione nel tempo e la sua recenza. La tabella 1.1 ne riassume i compromessi. Una rappresentazione più ricca costa di più da costruire e da conservare, e la sua utilità dipende in ultima analisi dalle operazioni che il modello è in grado di applicarvi.

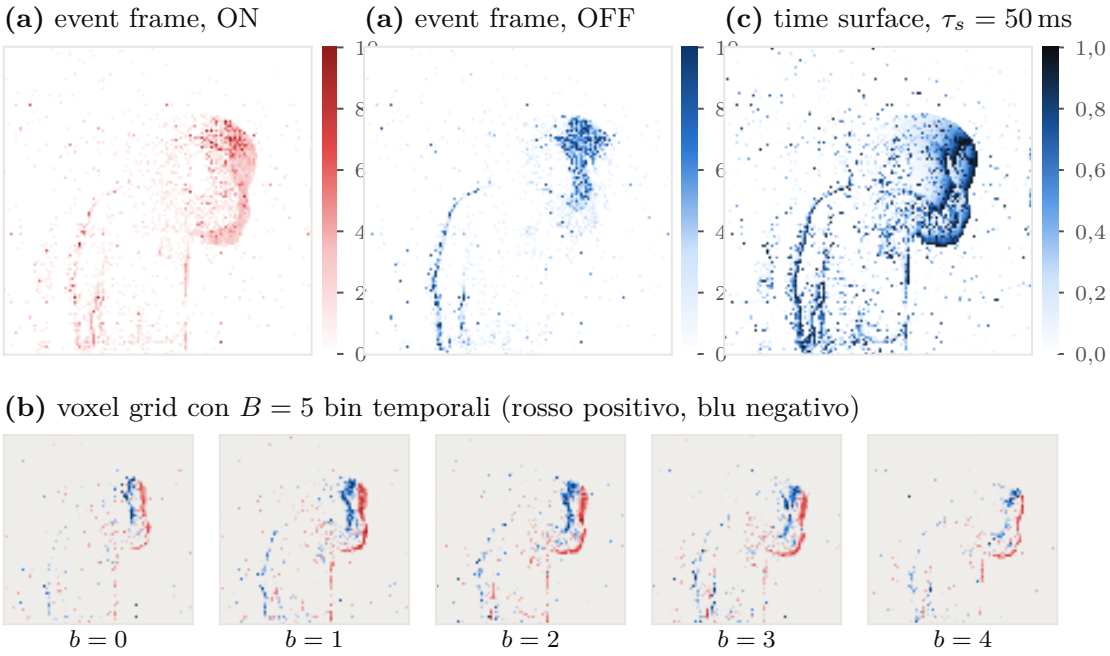


Figura 1.3: Tre rappresentazioni degli stessi 100 ms di una registrazione di DVS128 Gesture, durante una rotazione del braccio sinistro (circa 9 600 eventi). (a) Event frame, con i conteggi degli eventi positivi e negativi in due canali separati. (b) Voxel grid con $B = 5$ bin temporali, in cui la distribuzione dell'attività cambia da un bin all'altro, con i valori positivi in rosso e quelli negativi in blu. (c) Time surface dell'ultimo evento di entrambe le polarità, valutata alla fine dell'intervallo, in cui la posizione più recente del movimento appare come una scia.

Tabella 1.1: Informazione conservata e principali limiti delle rappresentazioni considerate.

Formato	**Informazione conservata**	**Descrizione del tempo**	**Limite principale**
Stream raw	Posizione, polarità e timestamp dei singoli eventi	Istanti originari	Sequenza irregolare e carico variabile
Conteggi ON/OFF	Molteplicità per pixel e polarità	Appartenenza al bin	Ordine intra-bin eliminato
Voxel grid	Accumuli su più bin temporali	Posizione temporale discretizzata o interpolata	Aggregazione e possibile cancellazione nella versione firmata
Time surface dell'ultimo evento	Recenza per pixel e polarità	Tempo trascorso dall'ultima attivazione	Molteplicità e storia anteriore non rappresentate separatamente

1.1.4 Tempo valido, silenzi e padding

Quando le sequenze hanno lunghezze diverse sorge un problema pratico. Le reti vengono addestrate su batch, cioè gruppi di campioni elaborati insieme in un unico tensore, e un tensore richiede che tutte le sequenze abbiano la stessa lunghezza. Le sequenze più brevi vengono quindi allungate con passi fittizi, il cosiddetto *padding*, fino alla lunghezza della più lunga, $T_{\max}$. Per tenere traccia di quali passi siano veri, a ogni campione n di lunghezza T_n si associa una maschera

$$m_{k,n} = \mathbf{1}[k < T_n], \qquad k = 0, \dots, T_{\max} - 1, \tag{1.13}$$

che vale 1 sui passi osservati e 0 su quelli aggiunti. In un batch di tre sequenze lunghe 25, 33 e 40 passi, per esempio, tutte vengono portate a 40, e la maschera della prima vale 1 sui primi 25 passi e 0 sugli ultimi 15.

Un passo di padding e un bin valido senza eventi possono contenere entrambi soltanto zeri, ma significano cose opposte. Il bin vuoto dice che è trascorso del tempo senza attività, ed è a tutti gli effetti un'osservazione. Il padding non osserva nulla. Confonderli altera il campione. Se la rete calcolasse la media delle proprie feature su tutti i 40 passi anziché sui 25 validi, il risultato della prima sequenza verrebbe ridotto di oltre un terzo senza alcuna ragione legata ai dati. Lo stesso accade se il padding entra nelle statistiche della normalizzazione, e un modulo dotato di stato interno deve far avanzare il proprio tempo soltanto sui passi validi.

C'è infine una terza situazione, diversa da entrambe. Quando la registrazione finisce prima della finestra prevista, come nell'esempio della finestra di 2 s, i passi successivi sono validi ma privi di eventi. La rete, però, non resta necessariamente inattiva, perché i potenziali di membrana e le memorie interne continuano a evolvere anche senza ingresso. Questa coda vuota e il padding di un batch vanno quindi dichiarati separatamente.

L'intera catena di elaborazione si può riassumere nello schema

$$\mathcal{E} \xrightarrow{\mathcal{R}} X_{1:T} \xrightarrow{\mathcal{M}} Z_{1:T} \xrightarrow{\mathcal{D}} \hat{y}, \tag{1.14}$$

dove $\mathcal{R}$ costruisce la rappresentazione, $\mathcal{M}$ ne elabora la dinamica e $\mathcal{D}$ prende la decisione. Ognuno dei tre stadi può limitare l'uso dell'informazione temporale. Per questo, quando si confrontano architetture diverse, rappresentazione e clock vanno tenuti fissi, e quando è l'ingresso stesso l'oggetto dell'esperimento ogni sua modifica va dichiarata come tale.

1.2 Dinamica neuronale e memoria nelle Spiking Neural Networks

Un neurone artificiale convenzionale, come quello di un percettrone multistrato con attivazione ReLU, calcola la propria uscita a partire dal solo ingresso corrente. Se

riceve due volte lo stesso ingresso risponde due volte allo stesso modo, e non conserva nulla da un'elaborazione all'altra. Il neurone di una Spiking Neural Network (SNN) si comporta diversamente. Possiede una variabile interna, il potenziale di membrana, che accumula gli ingressi ricevuti e sopravvive da un passo al successivo, e la sua risposta dipende quindi anche da ciò che è accaduto prima. In forma astratta,

$$h_k = F(h_{k-1}, X_k), \tag{1.15}$$

cioè lo stato h_k al passo k si ottiene aggiornando lo stato precedente h_{k-1} con il nuovo ingresso X_k. È la struttura di una rete ricorrente, con una differenza importante. In una rete ricorrente classica il passato rientra attraverso connessioni esplicite che riportano l'uscita di uno strato al suo ingresso, mentre in una SNN la ricorrenza è già dentro ogni neurone, e la rete ha memoria anche quando i suoi strati sono collegati soltanto in avanti [5].

Questa sezione descrive come un neurone accumuli e dimentichi gli ingressi, come la velocità con cui dimentica dipenda dal clock dell'input, come si addestri una rete che comunica con impulsi discontinui e che cosa una memoria di questo tipo permetta di fare.

1.2.1 Integrazione, soglia e reset

Il modello più semplice è l'Integrate-and-Fire (IF), che si può immaginare come un accumulatore. A ogni passo il neurone somma al proprio potenziale il contributo dell'ingresso,

$$H_k = V_{k-1} + X_k, \tag{1.16}$$

e quando il potenziale raggiunge una soglia V_{th} emette uno spike,

$$S_k = \Theta(H_k - V_{\text{th}}), \qquad \Theta(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases} \tag{1.17}$$

La funzione a gradino Θ traduce il confronto con la soglia in una risposta tutto o niente. Lo spike vale 1 se il potenziale ha raggiunto la soglia e 0 altrimenti, senza valori intermedi, e un potenziale appena sopra la soglia produce lo stesso spike di uno che la supera di molto.

Le due equazioni usano due simboli diversi per il potenziale, e conviene chiarire subito perché. All'interno di ogni passo avvengono due cose in sequenza. Prima il neurone riceve l'ingresso e si carica, e H_k indica il potenziale in questo momento, prima che il neurone decida se emettere lo spike. Poi, se lo spike è stato emesso, il potenziale viene riportato verso il basso, ed è il valore che resta dopo questa operazione, indicato con V_k, a passare al passo successivo. Se lo spike non viene emesso, i due valori coincidono.

L'operazione che segue lo spike si chiama reset, e ne esistono due varianti. Con un *hard reset* il potenziale viene riportato a un valore fisso V_{reset},

$$V_k = (1 - S_k)H_k + S_k V_{\text{reset}}, \tag{1.18}$$

mentre un *soft reset* si limita a sottrarre la soglia,

$$V_k = H_k - S_k V_{\text{th}}. \tag{1.19}$$

Se lo spike non c'è, cioè $S_k = 0$, entrambe lasciano $V_k = H_k$. La differenza emerge quando il potenziale supera la soglia di un certo margine. Con soglia 1 e $H_k = 1{,}3$, l'hard reset con $V_{\text{reset}} = 0$ riparte da zero e perde l'eccedenza di 0,3, mentre il soft reset riparte da 0,3 e la conserva per il passo successivo [6]. Qui e nel seguito si adotta la convenzione con potenziale di riposo nullo, in cui anche V_{reset} vale zero.

Il neurone IF ha una memoria perfetta. Al di fuori del reset, un contributo ricevuto all'inizio della sequenza pesa esattamente quanto uno appena arrivato, perché il modello non prevede alcuna forma di oblio. Il Leaky Integrate-and-Fire (LIF) introduce proprio questo elemento, una perdita o *leak*. A ogni passo, prima di sommare il nuovo ingresso, il potenziale perde una frazione fissa del proprio valore,

$$H_k = \lambda V_{k-1} + \eta X_k, \qquad 0 < \lambda < 1. \tag{1.20}$$

Il fattore di decadimento λ indica quale frazione del potenziale sopravvive da un passo al successivo, e il coefficiente η stabilisce quanto pesi l'ingresso. Con $\lambda = 0{,}5$, in assenza di ingresso, il potenziale si dimezza a ogni passo. Il neurone IF si ritrova ponendo $\lambda = 1$ ed $\eta = 1$. Quando l'ingresso compare con coefficiente unitario, come nell'equazione (1.16), si sottintende che η sia già stato assorbito nella definizione di X_k o nei pesi della rete.

La perdita cambia in modo sostanziale il comportamento del neurone, come mostra la figura 1.4. Un neurone con soglia 1 riceve tre ingressi da 0,6, una volta in tre passi consecutivi e una volta a tre passi di distanza l'uno dall'altro. Il neurone IF emette uno spike in entrambi i casi, non appena il secondo ingresso porta il potenziale a 1,2. Il neurone LIF con $\lambda = 0{,}5$ ed $\eta = 1$, quando gli ingressi sono ravvicinati, sale a 0,6, poi a 0,9 e infine a 1,05, e supera la soglia al terzo passo. Quando invece sono distanziati, tra un ingresso e l'altro il potenziale ha il tempo di scendere, non va mai oltre 0,68 e lo spike non arriva. A parità di ingresso complessivo, il LIF risponde agli ingressi che arrivano vicini e lascia cadere quelli isolati, e si comporta in questo senso come un rilevatore di coincidenze.

1.2.2 Decadimento discreto e scala temporale fisica

Il fattore λ è definito per passo, mentre la memoria di un neurone, dal punto di vista della scena osservata, si misura in millisecondi. Per collegare le due cose

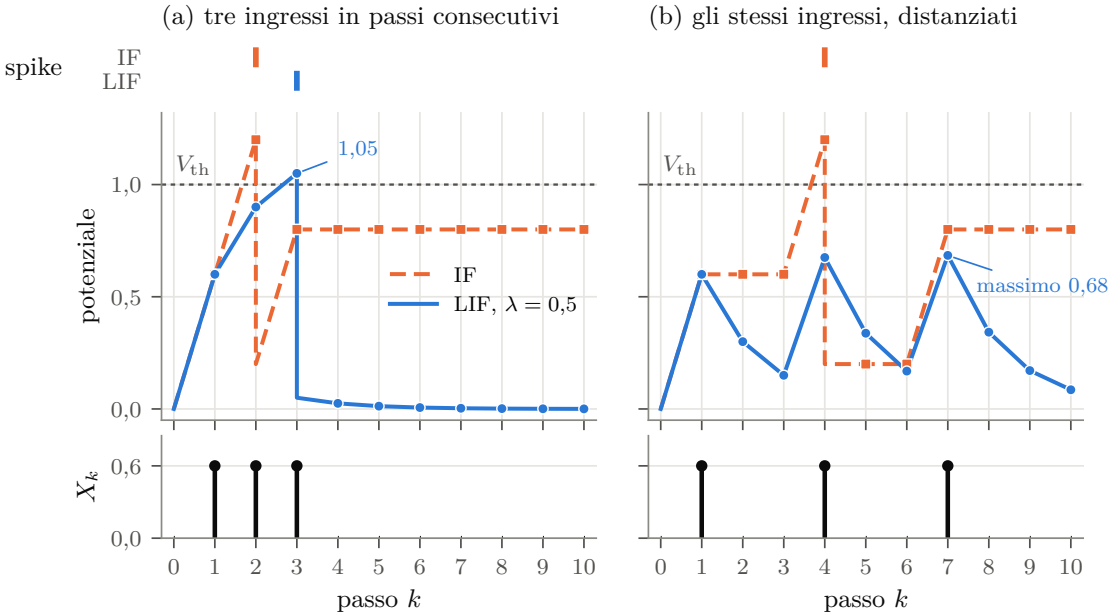


Figura 1.4: Integrazione, soglia e soft reset in un neurone IF (tratteggio arancione) e in un neurone LIF con $\lambda = 0{,}5$ (linea blu), entrambi con soglia $V_{\text{th}} = 1$. I due neuroni ricevono tre ingressi da 0,6 (in basso), consecutivi in (a) e distanziati in (b). I marcatori indicano il potenziale H_k appena caricato, i tratti verticali il soft reset che segue uno spike, la fila in alto gli spike emessi. Il neurone IF, che non dimentica, emette uno spike in entrambi i casi. Il LIF lo emette soltanto quando gli ingressi sono abbastanza ravvicinati da sommarsi prima che il potenziale decada.

bisogna risalire al modello da cui il LIF discende. Nella formulazione originale, a tempo continuo, la membrana si comporta come un condensatore che si scarica lentamente attraverso una resistenza,

$$\tau_m \frac{\mathrm{d}V(t)}{\mathrm{d}t} = -\Big(V(t) - V_{\text{rest}}\Big) + RI(t), \tag{1.21}$$

dove $I(t)$ è la corrente d'ingresso, R la resistenza di membrana, V_{rest} il potenziale di riposo e τ_m la costante di tempo di membrana [7]. Il primo termine a destra riporta il potenziale verso il riposo con una spinta proporzionale alla distanza, il secondo lo sposta in base all'ingresso. Se l'ingresso si annulla e non vengono emessi spike, il ritorno al riposo segue un'esponenziale,

$$V(t) - V_{\text{rest}} = \Big(V(t_0) - V_{\text{rest}}\Big) \exp\left[-\frac{t - t_0}{\tau_m}\right]. \tag{1.22}$$

Trascorso un tempo pari a τ_m, la distanza dal riposo si è ridotta a circa il 37% del valore iniziale, poiché $e^{-1} \approx 0{,}37$. La costante τ_m misura dunque, in unità di

tempo, per quanto il neurone ricorda un ingresso. La legge vale però soltanto al di sotto della soglia, perché ogni spike interrompe il decadimento con un reset.

Per simulare questa dinamica a passi di durata Δt occorre discretizzarla, e il risultato ha proprio la forma dell'equazione (1.20), con riferimento nullo. Il modo di discretizzare determina però i valori dei due coefficienti. Se l'ingresso $X_k = RI_k$ resta costante per tutta la durata del passo, l'equazione continua si può integrare esattamente, e si ottiene

$$\lambda = \exp\left(-\frac{\Delta t}{\tau_m}\right), \qquad \eta = 1 - \lambda. \tag{1.23}$$

Molte implementazioni, tra cui la libreria SpikingJelly [8], adottano invece la discretizzazione di Eulero, che sostituisce la derivata con un rapporto incrementale,

$$H_k = V_{k-1} + \frac{X_k - V_{k-1}}{\tau_d}, \qquad \lambda = 1 - \frac{1}{\tau_d}, \quad \eta = \frac{1}{\tau_d}. \tag{1.24}$$

Qui la costante τ_d, che deve superare 1 perché λ resti positivo, non è un tempo ma un numero puro, espresso in passi. Interpretandola come discretizzazione della stessa equazione continua vale $\tau_d = \tau_m/\Delta t$, ma le due regole non producono lo stesso decadimento. Con τ_m pari a due passi l'integrazione esatta dà $\lambda = e^{-1/2} \approx 0{,}61$, mentre Eulero con $\tau_d = 2$ dà $\lambda = 0{,}5$, e dopo quattro passi senza ingresso resta circa il 14% del potenziale nel primo caso e poco più del 6% nel secondo. Quando un lavoro riporta una costante di tempo, quindi, il solo valore numerico non basta. Bisogna sapere se si tratti di un tempo fisico o di un numero di passi, e con quale discretizzazione venga usata.

Srotolando la ricorrenza, cioè sostituendo a ritroso ogni potenziale con la sua espressione, si vede che cosa il neurone conservi della propria storia. In assenza di spike e reset, con coefficienti costanti, si ottiene

$$V_k = \lambda^k V_0 + \eta \sum_{j=1}^{k} \lambda^{k-j} X_j. \tag{1.25}$$

Il potenziale è la somma di tutti gli ingressi ricevuti, ciascuno moltiplicato per λ tante volte quanti sono i passi trascorsi dal suo arrivo. Con $\lambda = 0{,}5$, rispetto all'ingresso appena arrivato, quello del passo precedente pesa la metà, quello di tre passi prima un ottavo e quello di dieci passi prima meno di un millesimo. Gli ingressi passati, però, non vi compaiono come osservazioni separate. Sono fusi in un unico numero, e su questa proprietà si tornerà alla fine della sezione.

La stessa espressione suggerisce come tradurre λ in un tempo fisico. Un ingresso arrivato n passi prima, cioè $n\Delta t$ secondi prima, pesa λ^n. Cercando la costante τ_{eq} per cui $\lambda^n = \exp(-n\Delta t/\tau_{\text{eq}})$, come nel rilassamento continuo, si trova

$$\tau_{\text{eq}} = -\frac{\Delta t}{\ln \lambda}. \tag{1.26}$$

È la costante di tempo che un neurone continuo dovrebbe avere per dimenticare alla stessa velocità di quello discreto. Con $\lambda = 0{,}5$ vale circa $1{,}44\,\Delta t$, quindi poco più di 14 ms con bin da 10 ms e poco più di 140 ms con bin da 100 ms.

Il punto essenziale è che τ_{eq} dipende dalla durata del bin, cioè dal clock dell'input. Lo stesso neurone, con lo stesso λ, dimentica due volte più in fretta in tempo reale se la durata dei bin si dimezza. Per conservare la stessa memoria fisica bisognerebbe adeguare λ al clock. Nemmeno questo, però, renderebbe equivalenti i due modelli, perché cambiando i bin cambiano anche i conteggi che ciascuno contiene, la scala degli ingressi e il numero di occasioni in cui il neurone può emettere spike ed essere resettato. Con bin di durata variabile, come nelle partizioni a eventi costanti, un coefficiente fisso per passo non corrisponde infine ad alcun decadimento uniforme nel tempo fisico. Un passo lungo mezzo secondo e uno lungo cinque millisecondi fanno perdere al potenziale la stessa frazione. In alcuni modelli λ è un parametro appreso, diverso da strato a strato [9], ma il legame con il clock resta lo stesso.

Queste relazioni descrivono un singolo neurone isolato, al di sotto della soglia. La memoria di una rete, che combina molti neuroni, trasformazioni apprese e interazioni tra strati, non ha un orizzonte massimo così rigidamente definito. Il singolo integratore resta però il mattone elementare da cui quella memoria è costruita.

1.2.3 Addestramento con surrogate gradient

Le reti neurali si addestrano con la retropropagazione dell'errore, che calcola per ogni peso di quanto cambierebbe la loss modificandolo di poco, e lo corregge di conseguenza. La soglia che genera gli spike rende questo calcolo inutilizzabile. La funzione Θ è piatta ovunque tranne che nell'origine, dove salta da 0 a 1. Se il potenziale cambia di poco lo spike resta quello che era, e la derivata è quindi nulla quasi ovunque, mentre nell'unico punto in cui lo spike cambia la derivata non è definita. In entrambi i casi nessun segnale di errore riesce ad attraversare la soglia e a raggiungere i pesi degli strati precedenti.

I *surrogate gradient* aggirano l'ostacolo con un compromesso [5]. Nel passaggio in avanti (forward pass), quello che produce le uscite della rete, lo spike resta esattamente quello dell'equazione (1.17). Nel passaggio all'indietro (backward pass), quello che calcola i gradienti, la sua derivata viene sostituita da una funzione regolare,

$$\left.\frac{\partial S_k}{\partial H_k}\right|_{\text{backward}} := g_\alpha(H_k - V_{\text{th}}). \tag{1.27}$$

Una scelta diffusa è la derivata di una sigmoide. Con $\sigma(u) = (1 + e^{-u})^{-1}$ si pone

$$g_\alpha(z) = \alpha\,\sigma(\alpha z)\big[1 - \sigma(\alpha z)\big], \qquad \alpha > 0, \tag{1.28}$$

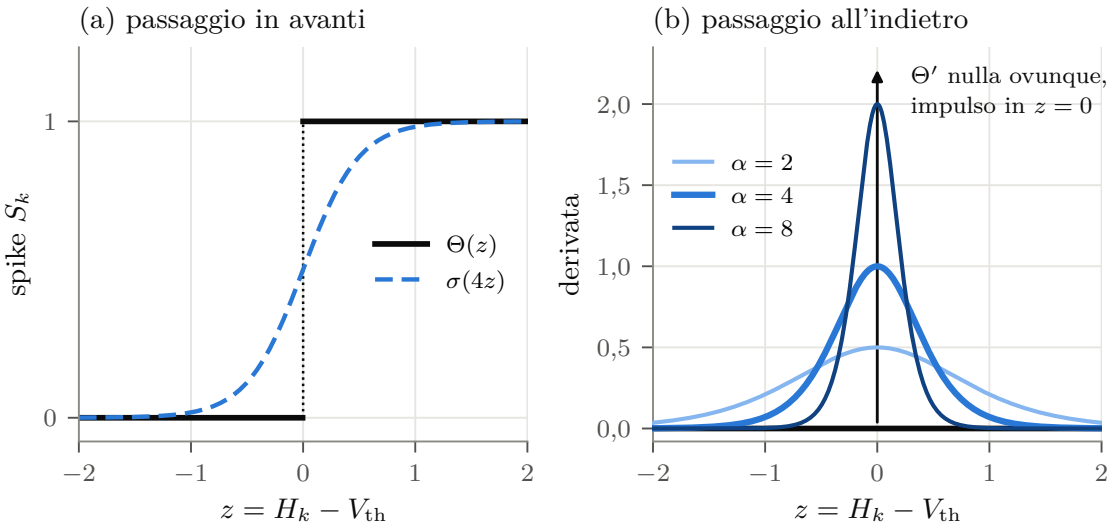


Figura 1.5: Principio del surrogate gradient, in funzione della distanza $z = H_k - V_{\text{th}}$ tra potenziale e soglia. (a) Nel passaggio in avanti lo spike è prodotto dal gradino Θ (linea nera). La sigmoide $\sigma(4z)$ (tratteggio) non viene mai calcolata, ma è la funzione di cui il passaggio all'indietro usa la derivata. (b) La derivata del gradino è nulla ovunque tranne nell'origine, dove non è definita (freccia), e non trasmette alcun gradiente. La surrogata g_α la sostituisce con una campana centrata sulla soglia, tanto più alta e stretta quanto maggiore è α. Il valore evidenziato, $\alpha = 4$, è quello adottato in questo lavoro.

che è proprio la derivata di $\sigma(\alpha z)$. In pratica la rete calcola gli spike con il gradino ma, al momento di calcolare i gradienti, si comporta come se il gradino fosse stato una sigmoide (figura 1.5).

Il parametro α regola la ripidità della sigmoide, e quindi l'altezza e la larghezza del gradiente surrogato. La funzione g_α raggiunge il massimo, pari ad $\alpha/4$, sulla soglia e si annulla rapidamente allontanandosene. Con $\alpha = 4$ vale 1 quando il potenziale è esattamente sulla soglia ed è già inferiore a 0,1 a distanza 1. Ricevono quindi un gradiente apprezzabile soltanto i neuroni il cui potenziale era vicino alla soglia, quelli cioè per cui una piccola modifica dei pesi avrebbe potuto cambiare la decisione di emettere o meno lo spike. Un α più grande avvicina la surrogata al gradino ma concentra il gradiente su pochi neuroni, uno più piccolo lo distribuisce su un intervallo più ampio a prezzo di un'approssimazione più grossolana. In ogni caso la sostituzione fornisce all'ottimizzatore un gradiente utilizzabile senza rendere differenziabile la funzione di soglia originale.

La soglia non è l'unico ostacolo. Poiché il potenziale collega ogni passo al successivo, la loss dipende dai pesi anche attraverso il tempo, e il gradiente deve percorrere la sequenza a ritroso. La Backpropagation Through Time (BPTT)

affronta il problema srotolando la ricorrenza [10]. La rete viene vista come una catena di T copie di sé stessa, una per ciascun passo, che condividono gli stessi pesi e sono collegate dagli stati che ciascuna passa alla successiva. Su questa catena, che non contiene più cicli, la retropropagazione ordinaria si applica senza difficoltà, e l'errore si propaga in due direzioni, attraverso gli strati e all'indietro nel tempo.

Per seguire il gradiente lungo il tempo si indichino con $J_{k+1} = \partial h_{k+1}/\partial h_k$ lo Jacobiano della transizione, che misura quanto lo stato successivo sia sensibile a quello corrente, e con $\boldsymbol{g}_k$ il contributo diretto della loss allo stato h_k. Il gradiente $\boldsymbol{\delta}_k$ accumulato sullo stato soddisfa allora la ricorrenza

$$\boldsymbol{\delta}_k = \boldsymbol{g}_k + J_{k+1}^{\top} \boldsymbol{\delta}_{k+1}, \tag{1.29}$$

che si calcola dall'ultimo passo verso il primo, a partire da $\boldsymbol{\delta}_T = \boldsymbol{g}_T$. Lo stato al passo k riceve dunque due segnali d'errore, quello che la loss gli attribuisce direttamente e quello che torna indietro dal passo successivo, che h_k ha contribuito a determinare. Nel calcolo effettivo gli Jacobiani contengono le derivate surrogate.

Un caso semplice rende visibile la conseguenza. Molte implementazioni escludono il termine di reset dal calcolo del gradiente, trattandolo come una costante. Il reset viene eseguito normalmente nel passaggio in avanti, e il neurone si comporta esattamente come prima, ma nel passaggio all'indietro il potenziale successivo dipende da quello corrente soltanto attraverso il termine λV_k. Per un singolo neurone LIF lo Jacobiano si riduce allora al fattore λ, e un errore che nasce n passi più avanti arriva al passo k moltiplicato per λ^n, lo stesso fattore con cui il neurone dimentica. Un neurone che dimentica in fretta, in altre parole, impara poco dalle conseguenze lontane dei propri stati. È una forma del più generale problema della scomparsa del gradiente nelle reti ricorrenti [11], e un collegamento esplicito tra istanti distanti, come i ritardi discussi nella sezione 1.5, offre anche al gradiente una strada più breve. La scelta di escludere o meno il reset dal gradiente non cambia dunque il comportamento della rete in inferenza, ma cambia il modo in cui viene addestrata, e va dichiarata.

Il prezzo della BPTT è la memoria. Per percorrere la catena all'indietro servono i valori calcolati in avanti in ogni passo, che vanno quindi conservati fino alla fine della sequenza. Per N_l unità nello strato l, il solo stato neuronale da memorizzare lungo T passi cresce come

$$\mathcal{O}\left(T \sum_l N_l\right), \tag{1.30}$$

a cui si sommano le altre attivazioni del grafo di calcolo [5]. Raddoppiare la lunghezza delle sequenze raddoppia quindi la memoria richiesta dall'addestramento. In inferenza la situazione è molto diversa. Per avanzare di un passo, a una rete di soli neuroni LIF basta conoscere il potenziale attuale di ciascun neurone, cioè $\sum_l N_l$ valori in tutto, qualunque sia T. Le due grandezze vanno tenute distinte,

perché un modello può richiedere pochissimo stato durante l'esecuzione e, al tempo stesso, un grafo molto esteso durante l'ottimizzazione.

In questo lavoro la rete spiking viene addestrata direttamente, con surrogate gradient e BPTT. È un paradigma diverso dalla conversione ANN-SNN, in cui una rete convenzionale viene prima addestrata e poi tradotta in una rete spiking che ne approssima le attivazioni con la frequenza degli spike. La dinamica discreta, la funzione surrogata e il trattamento del reset effettivamente adottati sono specificati nella sezione 2.3.1.

1.2.4 Memoria neuronale e relazioni tra feature

Si torni all'equazione (1.25). Per un neurone LIF privo di ulteriori stati interni, tutto ciò che la storia degli ingressi può trasmettere al futuro passa per un unico numero, il potenziale V_k,

$$(X_1, \ldots, X_k) \longrightarrow V_k. \tag{1.31}$$

L'intera sequenza viene compressa in un solo valore, e storie diverse possono lasciare lo stesso potenziale. Con $\lambda = 0{,}5$ ed $\eta = 1$, partendo da un potenziale nullo, un ingresso pari a 0,4 arrivato al passo precedente e un ingresso pari a 0,2 arrivato adesso lasciano entrambi $V_k = 0{,}2$. Da quel momento, se gli ingressi successivi coincidono, il neurone si comporta in modo identico nei due casi, e non ha più modo di sapere se abbia visto qualcosa di intenso poco fa o qualcosa di debole in questo istante. La rete nel suo complesso può comunque conservare la differenza in altre unità o in altri moduli, sicché la compressione operata da un singolo potenziale non implica di per sé una perdita equivalente a livello dell'intera rete.

Si delineano così due capacità diverse. La prima è la *persistenza*, cioè la capacità di conservare a lungo una traccia del passato. La seconda è l'*accesso*, cioè la possibilità di recuperare separatamente ciò che è accaduto in un istante preciso per confrontarlo con il presente. Si potrebbe pensare di migliorare la memoria di un neurone avvicinando λ a 1, ma la modifica agisce soltanto sulla prima. I contributi passati sopravvivono più a lungo, e proprio per questo un numero maggiore di ingressi si mescola nella stessa somma, che diventa ancora più difficile da scomporre. Si pensi alla barra della sezione 1.1.1, la cui direzione è scritta nell'ordine con cui si attivano pixel vicini. Per riconoscerla non basta sapere che entrambi i pixel sono stati attivi di recente, informazione che un potenziale conserva bene. Bisogna sapere quale dei due si è attivato prima, cioè mettere a confronto due istanti distinti, e un'unica somma pesata può farlo soltanto in modo indiretto.

Un'ultima avvertenza riguarda la causalità. Che ogni neurone dipenda soltanto dal proprio passato non garantisce che lo faccia l'intera rete. Se una normalizzazione calcola le proprie statistiche su tutti i passi della sequenza, come accade spesso durante l'addestramento, il valore normalizzato a un passo dipende anche dai passi successivi. Il caso della rete usata in questo lavoro è discusso nella sezione 2.4.2.

La distinzione tra persistenza e accesso motiva l'interesse per operatori capaci di combinare o confrontare esplicitamente feature diverse. La sezione successiva descrive come l'attenzione sia stata integrata nelle architetture spiking, e sarà allora necessario distinguere le relazioni costruite all'interno di uno stesso passo da quelle che collegano istanti diversi.

1.3 Dalle SNN agli Spiking Transformer

La dinamica neuronale offre alle SNN un modo naturale di trasportare informazione da un timestep al successivo. Resta però aperto un problema di altra natura, quello di mettere in relazione le numerose feature che compongono una rappresentazione visiva. Un neurone LIF integra nel tempo il proprio ingresso, ma nulla nella sua dinamica gli permette di stabilire quali regioni della scena siano rilevanti l'una per l'altra, né di collegare feature spazialmente lontane.

Le architetture convoluzionali affrontano tradizionalmente questo problema componendo operatori locali. Ogni convoluzione combina l'informazione di un intorno limitato, e soltanto impilando molti strati, che allargano progressivamente il campo recettivo, la rete arriva a costruire relazioni spaziali estese. Su questo principio si fonda l'efficacia delle CNN nella visione artificiale, ed è su di esso che sono state costruite molte delle prime SNN profonde.

Il Transformer adotta una prospettiva diversa. Anziché affidare la costruzione delle dipendenze a una lunga catena di operatori locali, permette agli elementi di una rappresentazione di interagire direttamente attraverso il meccanismo di self-attention [12]. Trasferita alla visione, l'idea consiste nel suddividere l'immagine in porzioni, le patch, che dopo la proiezione nello spazio delle feature prendono il nome di token e possono influenzarsi reciprocamente in base al contenuto che rappresentano [13].

Data una matrice di token

$$X \in \mathbb{R}^{N \times D}, \tag{1.32}$$

dove N è il numero dei token e D la dimensione della rappresentazione di ciascuno, la self-attention convenzionale ne ricava tre proiezioni lineari,

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \tag{1.33}$$

denominate rispettivamente Query, Key e Value. Il prodotto tra Query e Key misura la compatibilità tra ogni coppia di token,

$$A = \frac{QK^\top}{\sqrt{d}}, \tag{1.34}$$

dove d è la dimensione su cui viene calcolata l'attenzione. Normalizzata riga per riga mediante softmax, questa matrice fornisce i pesi con cui combinare i Value,

$$Y = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V. \tag{1.35}$$

Al di là della formulazione compatta, il meccanismo ricorda la consultazione di un archivio. Ogni token formula una richiesta, la propria Query, la confronta con le descrizioni offerte dagli altri token, le loro Key, e aggiorna la propria rappresentazione combinandone i contenuti, i Value, in proporzione alla compatibilità riscontrata. Ne deriva una capacità che la convoluzione non possiede nella stessa forma. Due regioni lontane nello spazio possono interagire direttamente all'interno di un singolo blocco, senza che l'informazione debba attraversare una lunga successione di operatori locali.

Questa libertà ha un prezzo. Poiché $QK^\top$ confronta ogni token con tutti gli altri, la matrice di attenzione ha dimensione $N \times N$ e il costo del prodotto cresce quadraticamente con il numero dei token, approssimativamente come

$$\mathcal{O}(N^2 D), \tag{1.36}$$

mentre la memoria necessaria a materializzare la matrice cresce come $\mathcal{O}(N^2)$.

Già in un Transformer convenzionale questa complessità costituisce un vincolo importante. In una SNN se ne aggiunge un secondo, perché le stesse operazioni vanno ripetute a ogni timestep. Per un ingresso $X \in \mathbb{R}^{T\times N\times D}$ l'attenzione viene tipicamente applicata, istante per istante, alle feature di ciascun passo della simulazione. La dimensione temporale, che finora era apparsa come la sorgente della memoria neuronale, diventa così anche un moltiplicatore del costo.

L'incontro tra SNN e Transformer nasce da questa tensione. La self-attention offre una notevole capacità relazionale, ma nella sua forma convenzionale si basa su attivazioni continue, prodotti tra matrici dense e normalizzazioni, operazioni che non traggono alcun vantaggio dalla natura binaria e sparsa degli spike. Sostituire le attivazioni di un Transformer con neuroni LIF non basta dunque a ottenere uno Spiking Transformer. Occorre ripensare almeno in parte il meccanismo di attenzione, altrimenti la rappresentazione spiking verrebbe subito riconvertita in una computazione del tutto analoga a quella di una rete convenzionale (ANN). Spikformer nasce proprio da questo problema.

Con Spikformer, Zhou et al. propongono nel 2023 la Spiking Self-Attention (SSA), mostrando che Query, Key e Value possono essere espressi in forma spiking e che la softmax, cuore dell'attenzione convenzionale, può essere eliminata [14]. Il passaggio è importante perché il lavoro non si limita a inserire neuroni spiking in un Transformer esistente, ma interviene sul nucleo stesso dell'attenzione per renderlo compatibile con rappresentazioni binarie.

Lo schema generale resta riconoscibile. Dalle feature spiking si ricavano tre rappresentazioni binarie,

$$Q_s,\ K_s,\ V_s \in \{0,1\}^{N\times D}, \tag{1.37}$$

la cui interazione continua a svolgere il ruolo dell'attenzione. In forma semplificata,

$$\mathrm{SSA}(Q_s, K_s, V_s) = \mathrm{SN}\left(Q_s K_s^\top V_s\, \alpha\right), \tag{1.38}$$

dove $\mathrm{SN}(\cdot)$ indica lo strato di neuroni spiking che riporta il risultato in forma binaria e α è un fattore di scala. I dettagli cambiano da una variante all'altra, ma due idee restano centrali, la rappresentazione di Q, K e V mediante spike e la rinuncia alla softmax. Lavori successivi riconoscono proprio in questi due elementi i principi fondativi della prima generazione di attenzione spike-based.

La rinuncia alla softmax ha un peso particolare. La softmax richiede esponenziali, divisioni e una normalizzazione che coinvolge tutti i coefficienti di una riga, operazioni estranee a un paradigma che punta su comunicazioni binarie e su una computazione guidata dagli eventi.

Spikformer dimostra così che self-attention e SNN sono compatibili, ma porta alla luce due questioni che in un Transformer convenzionale restano in secondo piano. La prima riguarda la natura effettiva delle operazioni. Che Query, Key e Value siano spike non garantisce che l'intero blocco esegua soltanto operazioni economiche. I prodotti tra matrici binarie si possono realizzare con accumuli condizionati, ma i risultati intermedi sono conteggi e non più valori binari. Le connessioni residue e le trasformazioni successive possono poi reintrodurre valori a più bit, e con essi operazioni che non beneficiano del paradigma spike-driven.

La seconda questione riguarda la scalabilità. Il prodotto $QK^\top$ conserva una matrice di correlazione tra tutte le coppie di token, e anche con dati sparsi la dimensione logica dell'operazione resta legata a N^2, un costo che diventa critico al crescere del numero dei token.

Da questi due problemi sono nate linee di ricerca diverse e complementari. La prima si chiede quanta parte della rete sia davvero spike-driven, cioè quanta computazione venga effettivamente innescata dalla presenza di uno spike e risparmiata in sua assenza.

Spike-driven Transformer affronta direttamente questa domanda [15]. Gli autori osservano che diverse architetture precedenti mescolano semplici accumuli (AC), innescati dagli spike, con moltiplicazioni seguite da accumulo (MAC) ereditate dal Transformer, dando luogo a una computazione ibrida. Il costo dei due tipi di operazione sarà discusso nella sezione 1.6. L'obiettivo diventa allora non soltanto rendere spiking Q, K e V, ma mantenere il più possibile binaria la comunicazione tra i blocchi, in modo che l'assenza di uno spike coincida davvero con l'assenza di calcolo.

Il meccanismo proposto, la Spike-Driven Self-Attention (SDSA), abbandona il prodotto completo $QK^\top$. In una delle sue formulazioni, Query e Key vengono moltiplicate elemento per elemento e sommate lungo la dimensione dei token, e il risultato attraversa uno strato spiking,

$$A_s = \mathrm{SN}\left(\mathrm{SUM}_N\left(Q_s \odot K_s\right)\right), \tag{1.39}$$

dopodiché il vettore binario così ottenuto viene applicato ai Value,

$$Y_s = A_s \odot V_s, \tag{1.40}$$

dove $\odot$ indica il prodotto elemento per elemento. Quando gli operandi sono binari, questo prodotto equivale a un mascheramento, perché ogni elemento viene semplicemente conservato oppure azzerato.

Rispetto all'equazione (1.35) il cambiamento è sostanziale. Non viene costruita alcuna matrice $N \times N$ di relazioni tra coppie di token. L'interazione tra gli spike viene compressa, attraverso mascheramento e somma, in un vettore binario che seleziona le feature rilevanti. Per questo SDSA può essere interpretata come una forma particolare di attenzione lineare, la cui complessità cresce linearmente con il numero di token e di canali anziché quadraticamente con il numero di token [15]. Le due filosofie possono essere confrontate schematicamente,

$$\begin{aligned} &\text{VSA:} && QK^\top \longrightarrow N \times N \text{ attention map,} \\ &\text{SDSA:} && Q \odot K \longrightarrow \text{summation} \longrightarrow \text{binary attention vector.} \end{aligned} \tag{1.41}$$

Si rinuncia così a parte della ricchezza delle correlazioni tra coppie, in cambio di un'operazione molto più adatta alla computazione sparsa. Gli autori mostrano che le interazioni tra Query, Key e Value in forma spiking possono essere realizzate mediante maschere e accumuli sparsi, senza costruire la matrice di attenzione convenzionale [15].

Spike-driven Transformer interviene anche su un elemento meno appariscente, ma importante per comprendere gli sviluppi successivi, cioè la posizione delle connessioni residue. Una connessione residua somma l'ingresso di un blocco alla trasformazione che il blocco stesso produce, e nelle reti profonde facilita la propagazione dell'informazione e dei gradienti. In una SNN, però, conta dove la somma viene eseguita, se tra spike oppure tra potenziali di membrana.

Nella connessione residua Spike-Element-Wise (SEW) adottata da Spikformer si sommano direttamente due tensori di spike. Il risultato può valere 0, 1 o 2, e l'intervallo si allarga ulteriormente a ogni blocco attraversato. Il segnale che percorre il ramo residuo smette così di essere binario. Quando raggiunge uno strato pesato non basta più selezionare i pesi corrispondenti agli spike attivi, perché ciascun peso va moltiplicato per un valore intero, e ricompaiono moltiplicazioni tra interi e numeri in virgola mobile.



Figura 1.6: Progressiva riformulazione della self-attention nel dominio spiking. Spikformer introduce Query, Key e Value in forma spiking ed elimina la softmax. Spike-driven Transformer sostituisce inoltre la correlazione tra coppie di token con operazioni di mascheramento e accumulo, privilegiando una computazione lineare e spike-driven.

Spike-driven Transformer sposta invece la somma residua sui potenziali di membrana e colloca il neurone spiking dopo di essa, cosicché ciò che viene trasmesso al blocco successivo resta binario [15]. Il dettaglio contiene una lezione progettuale di portata più generale. Contare i neuroni spiking presenti in un'architettura dice poco sulla sua natura event-driven, che dipende piuttosto dalla posizione di quei neuroni rispetto alle operazioni lineari e ai percorsi residui.

Spikingformer approfondisce lo stesso punto in una linea di ricerca parallela [16]. Gli autori analizzano le connessioni residue di Spikformer e mostrano che la variante SEW genera valori a più bit lungo i collegamenti diretti, mentre una Membrane Shortcut (MS), che somma i potenziali anziché gli spike, preserva ingressi binari per gli operatori successivi. Combinando questa soluzione con una self-attention globale, il modello mira a conservare insieme comunicazione spike-driven e capacità di modellazione globale.

Il confronto mette in luce un compromesso che accompagna l'intera evoluzione degli Spiking Transformer. Semplificare l'attenzione in modo aggressivo ne riduce il costo, ma impoverisce le relazioni che il modello può rappresentare. Conservare una self-attention più globale offre maggiore espressività, ma obbliga a intervenire altrove per preservare l'efficienza spike-driven.

La storia di queste architetture non va letta, allora, come una successione di modelli via via migliori. È piuttosto l'esplorazione di uno spazio definito da almeno tre assi, la capacità relazionale, la scalabilità e l'aderenza al paradigma spike-driven, lungo i quali ciascun lavoro sceglie la propria posizione. Spikformer dimostra che la self-attention può essere resa spiking. Spike-driven Transformer mostra che l'attenzione può essere riformulata in profondità, fino a eliminare gran parte delle operazioni non guidate dagli eventi. Spikingformer chiarisce che anche i percorsi residui fanno parte del problema e che la modellazione globale è conciliabile con la comunicazione binaria.

Resta aperta una questione di altro genere, che riguarda il modo in cui scalare l'architettura stessa. Un Vision Transformer tradizionale mantiene spesso la stessa risoluzione lungo gran parte dell'encoder, e molte delle prime architetture Spiking Transformer seguono la stessa strategia. Un modulo iniziale produce la sequenza di token e i blocchi successivi operano prevalentemente sulla stessa granularità spaziale. Spikformer, per esempio, usa uno Spiking Patch Splitting per convertire l'ingresso in N token di dimensione D, elaborati poi dai blocchi SSA e MLP.

La soluzione è semplice, ma si discosta dalla struttura gerarchica che caratterizza molte CNN e numerosi Vision Transformer moderni. In una rete gerarchica le prime fasi operano su molte posizioni con pochi canali, mentre quelle profonde lavorano su pochi token descritti da feature più ricche. Schematicamente,

$$(N_1, D_1) \longrightarrow (N_2, D_2) \longrightarrow (N_3, D_3), \tag{1.42}$$

con $N_1 > N_2 > N_3$ e $D_1 < D_2 < D_3$. Questa progressione presenta due vantaggi. Preserva il dettaglio spaziale nelle prime fasi mentre costruisce rappresentazioni via via più astratte, e riduce il numero dei token proprio dove la dimensione delle feature, e con essa il costo dei blocchi Transformer, tende a crescere.

Da questa esigenza nasce QKFormer [17]. Gli autori individuano due limiti congiunti delle architetture precedenti, la complessità quadratica della SSA rispetto al numero dei token e l'elevato costo di memoria che deriva dal combinare la dimensione spaziale con la simulazione temporale. Per superarli introducono una struttura gerarchica e un nuovo meccanismo, la Q-K Attention, che sostituisce la matrice di attenzione con vettori binari capaci di esprimere l'importanza dei token oppure dei canali.

Rispetto a SDSA l'idea compie un ulteriore passo. Oltre a chiedersi come rendere più economico il prodotto tra Q, K e V, QKFormer si domanda se tutte e tre le componenti siano davvero necessarie per selezionare l'informazione. La Q-K Attention ne usa soltanto due, Query e Key. A seconda della variante, una delle due viene aggregata lungo i canali oppure lungo i token, trasformata da un neurone spiking in un vettore binario e usata per mascherare l'altra. Nel primo caso la Q-K Token Attention stabilisce quali token siano rilevanti, nel secondo la Q-K Channel Attention stabilisce quali canali lo siano. In entrambi i casi l'attenzione si riduce a un vettore anziché a una matrice densa di relazioni tra coppie di token. La complessità diventa lineare nella dimensione su cui opera il meccanismo, e l'elaborazione continua a basarsi su spike binari, somme e mascheramenti.

QKFormer combina questa attenzione con una struttura a più stadi. A partire da patch relativamente piccole, il numero dei token viene ridotto progressivamente da moduli di Spiking Patch Embedding with Deformed Shortcut (SPEDS), mentre il numero dei canali cresce negli stadi più profondi. Nella configurazione descritta dagli autori la risoluzione spaziale segue approssimativamente la progressione

$$\frac{H}{4} \times \frac{W}{4} \longrightarrow \frac{H}{8} \times \frac{W}{8} \longrightarrow \frac{H}{16} \times \frac{W}{16}. \tag{1.43}$$

Poiché il rapporto tra token e canali cambia lungo la rete, non è detto che lo stesso tipo di attenzione convenga a ogni livello. La Q-K Token Attention si adatta bene alle prime fasi, dove i token sono numerosi e i canali relativamente pochi, mentre nelle fasi profonde possono subentrare la Q-K Channel Attention o la SSA.

Questo percorso spiega perché QKFormer rappresenti il punto di partenza naturale per il lavoro della presente tesi. Oltre ad adattare un Transformer al dominio spiking, il modello riunisce tre proprietà preziose nel regime compatto studiato più avanti, cioè un'attenzione di costo lineare, una rappresentazione gerarchica e una trasmissione residua che attraversa i cambi di risoluzione.

Altrettanto importante è osservare quale problema QKFormer non si proponga di risolvere. Le operazioni di attenzione descritte finora costruiscono relazioni tra



Figura 1.7: Principali passaggi concettuali nell'evoluzione degli Spiking Transformer rilevanti per questa tesi. I diversi lavori affrontano progressivamente compatibilità della self-attention con gli spike, purezza event-driven, residual learning e scalabilità del meccanismo di attenzione.

token o canali all'interno delle feature di un singolo timestep. Data una sequenza $X_1, X_2, \ldots, X_T$, l'attenzione applicata all'istante t riceve tipicamente la sola X_t e ne modella le relazioni interne. Il collegamento tra X_{t-1} e X_t resta invece affidato principalmente agli stati dei neuroni spiking che attraversano il tempo.

Non si tratta di una critica, poiché QKFormer si concentra su efficienza, scalabilità e qualità della rappresentazione spaziale. È però proprio qui che il percorso della letteratura incontra il problema affrontato in questa tesi. L'evoluzione esaminata ha migliorato progressivamente il modo in cui le feature interagiscono all'interno di X_t e ha ridotto il costo di queste interazioni. Resta da capire se la dinamica neuronale descritta nella sezione 1.2 basti a governare anche l'interazione tra X_t e X_{t-k}.

La domanda merita di essere posta in modo esplicito, perché l'espressione *self-attention* suggerisce, in modo intuitivo ma impreciso, un accesso globale all'intera sequenza. La globalità dell'attenzione va invece sempre riferita alla dimensione lungo cui l'operatore agisce. Uno Spiking Transformer può disporre di un'attenzione globale sui token e, insieme, di una comunicazione temporale prevalentemente ricorrente e locale. Questa separazione tra relazioni globali nello spazio delle feature e interazioni lungo il tempo motiva l'analisi, nella sezione 1.5, delle architetture progettate per l'attenzione spazio-temporale e per il mixing mediante ritardi.

Prima di affrontare questo problema conviene però soffermarsi sul punto di arrivo dell'evoluzione descritta. QKFormer, oltre a essere tra i lavori più performanti della famiglia, combina struttura gerarchica e Q-K Attention in un modo che determina direttamente la baseline da cui muove la parte sperimentale della tesi. La sezione successiva ne analizza quindi il funzionamento in dettaglio.

1.4 QKFormer come riferimento architetturale

Le architetture esaminate avevano già mostrato che la self-attention può essere trasferita nel dominio spiking e che buona parte della computazione può essere riformulata in termini di spike binari, mascheramenti e accumuli. Restava difficile, però, combinare questa idea con una struttura davvero gerarchica, per ragioni soprattutto computazionali. Quando i token sono numerosi, un'attenzione basata su correlazioni tra coppie richiede memoria e calcolo che crescono rapidamente, e in una SNN questo costo si moltiplica lungo la dimensione temporale. QKFormer nasce dall'esigenza di rendere possibile una rappresentazione gerarchica senza rinunciare alla natura spike-driven dell'attenzione [17].[1]

[1]Gli autori segnalano nel repository ufficiale un errore nell'ordine di reshape e LIF della proiezione SSA, con possibile sovrastima dei risultati ImageNet. I punteggi del paper vanno quindi distinti da una riproduzione verificata dell'implementazione corretta [18].

Tabella 1.2: Evoluzione dei meccanismi di attenzione spiking rilevanti per il lavoro.

Architettura	**Problema affrontato**	**Operatore**	**Ambito**	**Costo**	**Spike-driven**
Transformer / ViT	Relazioni globali tra token	VSA + softmax	Token–token	Quadratica in N	No
Spikformer	Rendere spiking la self-attention	SSA con Q, K, V spiking	Token–token	Quadratica in N	Parziale
Spike-driven Transformer	Eliminare operazioni non spike-driven e ridurre il costo	SDSA, mask + sparse additions	Selezione di feature/canali	Lineare	Sì
Spikingformer	Preservare global attention mantenendo residual spike-driven	Self-attention + MS residual	Token–token	Dipendente dalla SSA	Sì
QKFormer	Scalabilità e rappresentazione gerarchica	QKTA / QKCA + SPEDS	Token o channel importance	Lineare nella dimensione	Sì

Nota. «Spike-driven» indica qui la proprietà architetturale dichiarata dai rispettivi lavori. Il costo effettivo dipende comunque da firing rate, operatori periferici, rappresentazione degli input e piattaforma hardware, e verrà discusso nella sezione 1.6.

Il modello proposto da Zhou et al. può essere letto come l'integrazione di tre idee [17]. La prima è la Q-K Attention, che sostituisce la matrice completa delle relazioni tra token con un vettore binario di importanza. La seconda è una struttura gerarchica a più stadi, nella quale la risoluzione spaziale si riduce progressivamente mentre aumenta il numero dei canali. La terza è lo Spiking Patch Embedding with Deformed Shortcut (SPEDS), che accompagna i cambi di risoluzione senza interrompere il flusso residuo dell'informazione. I tre elementi si sostengono a vicenda. L'attenzione lineare rende praticabile la gerarchia, e SPEDS ne attraversa i cambi di dimensione senza rinunciare all'apprendimento residuo.

Conviene partire dalla Q-K Attention. Tanto la self-attention convenzionale quanto la SSA di Spikformer impiegano Query, Key e Value [14]. QKFormer elimina invece il Value dal calcolo dell'attenzione e costruisce due sole proiezioni spiking [17],

$$Q = \mathrm{SN}_Q\left(\mathrm{BN}(XW_Q)\right), \qquad K = \mathrm{SN}_K\left(\mathrm{BN}(XW_K)\right), \tag{1.44}$$

con $Q, K \in \{0,1\}^{T\times N\times D}$, dove T indica il numero dei timestep, N quello dei token e D la dimensione delle feature. Le proiezioni lineari, seguite da batch normalization (BN) e da un neurone spiking, restano apprendibili, ma il meccanismo di selezione che segue opera su spike binari.

La differenza rispetto alla self-attention tradizionale va oltre la rimozione di V. QKFormer non costruisce alcuna matrice $A \in \mathbb{R}^{N\times N}$ che descriva la relazione di ogni token con tutti gli altri, e si pone una domanda più economica, cioè quali token, o quali canali, meritino di attraversare il blocco di attenzione. Dalle due possibili risposte derivano le varianti Q-K Token Attention e Q-K Channel Attention.

Nella Q-K Token Attention (QKTA) si sommano, per ciascun token, gli spike della sua Query lungo i canali. Considerando per semplicità un solo timestep, con $Q \in \{0,1\}^{N\times D}$, si ottiene

$$r_n = \sum_{d=1}^{D} Q_{n,d}, \qquad n = 1, \ldots, N, \tag{1.45}$$

e r_n misura quanta attività sia presente nella Query del token n. Il vettore così ottenuto attraversa un neurone spiking,

$$A_t = \mathrm{SN}\left(\sum_{d=1}^{D} Q_{:,d}\right), \qquad A_t \in \{0,1\}^{N\times 1}, \tag{1.46}$$

che lo trasforma in un vettore binario di attenzione sui token, il *token attention vector*. Il pedice t si riferisce qui ai token e non al tempo. Ogni componente di A_t decide se il token corrispondente debba passare oppure essere mascherato.

Il vettore viene poi applicato alla Key mediante un prodotto elemento per elemento (prodotto di Hadamard), replicato su tutti i canali,

$$X' = A_t \odot K. \tag{1.47}$$

Essendo A_t binario, non serve alcuna moltiplicazione vera e propria. Per ciascun token il risultato si riduce a

$$X'_n = \begin{cases} K_n, & A_{t,n} = 1, \\ 0, & A_{t,n} = 0. \end{cases} \tag{1.48}$$

Il meccanismo si configura così come una selezione dei token guidata dagli spike.

Il cambio di prospettiva è netto. Nella self-attention convenzionale ogni token costruisce la propria uscita come combinazione pesata di molti altri. QKTA rinuncia a rappresentare questa struttura di relazioni tra coppie e si limita a usare l'attività della Query per decidere quali token della Key siano rilevanti. Da questa rinuncia deliberata alla matrice $N \times N$ deriva la riduzione della complessità.

Occorre però distinguere, come fanno gli autori, il costo dell'intero blocco da quello dell'operazione di attenzione in senso stretto [17]. Memorizzare Q e K richiede comunque spazio dell'ordine di ND, e le proiezioni lineari restano. Il meccanismo che costruisce l'attenzione, tuttavia, ha bisogno soltanto di un vettore $N \times 1$. Grazie al mascheramento diretto, l'attenzione QKTA ha complessità lineare nella dimensione dei canali su cui avviene la riduzione e richiede soltanto $\mathcal{O}(N)$ memoria aggiuntiva per il vettore di selezione, contro la matrice $\mathcal{O}(N^2)$ della SSA. Gli autori riportano per QKTA una complessità dell'attenzione $\mathcal{O}(D)$, a fronte di $\mathcal{O}(N^2 D)$ per la self-attention convenzionale e per la SSA, fermo restando il costo delle trasformazioni che circondano l'operatore [17].

La Q-K Channel Attention (QKCA) applica lo stesso principio all'altro asse. Per stabilire quali canali siano importanti somma la Query lungo i token,

$$c_d = \sum_{n=1}^{N} Q_{n,d}, \qquad d = 1, \ldots, D, \tag{1.49}$$

e ne ricava il vettore binario

$$A_c = \mathrm{SN}\left(\sum_{n=1}^{N} Q_{n,:} \right), \qquad A_c \in \{0,1\}^{1 \times D}, \tag{1.50}$$

che, applicato alla Key,

$$X' = A_c \odot K, \tag{1.51}$$

conserva o elimina interi canali della rappresentazione.

Le due varianti applicano dunque lo stesso principio su assi diversi. QKTA si chiede quali posizioni spaziali siano rilevanti, QKCA quali canali di feature. Proprio questa simmetria rende la Q-K Attention adatta a una rete gerarchica, nella quale il rapporto tra N e D cambia lungo il backbone.

Al mascheramento segue comunque una trasformazione lineare dell'uscita [17],

$$X'' = \mathrm{SN}\left(\mathrm{BN}\left(\mathrm{Linear}(X')\right)\right). \tag{1.52}$$



Figura 1.8: Principio della Q-K Attention. QKTA comprime la Query lungo i canali e costruisce una maschera binaria sui token, mentre QKCA la comprime lungo i token e costruisce una maschera sui canali. In entrambi i casi l'attenzione evita la costruzione della matrice $N \times N$ delle relazioni tra coppie di token.

Il blocco non si riduce quindi a una maschera. Quest'ultima seleziona i token o i canali, e la proiezione successiva ne ricombina le feature.

Un'ulteriore semplificazione riguarda il fattore di scala. Nella self-attention convenzionale la varianza di $QK^\top$ cresce con la dimensione delle feature, e il fattore $1/\sqrt{d}$ serve a contenerla. Anche nella SSA, pur senza softmax, la somma di molti prodotti binari può produrre valori abbastanza grandi da richiedere un fattore di scala. Gli autori di QKFormer mostrano, analiticamente e sperimentalmente, che l'aggregazione della Q-K Attention produce una distribuzione con varianza molto più contenuta, e nella loro analisi la varianza massima teorica di QKTA risulta di gran lunga inferiore a quella della SSA [17]. Il fattore di scala può così essere eliminato dal percorso di attenzione.

L'insieme di queste semplificazioni rende praticabile il secondo elemento caratteristico del modello, la struttura gerarchica. Per un ingresso neuromorfico, QKFormer parte da una rappresentazione

$$X_0 \in \mathbb{R}^{T\times H\times W\times n}, \tag{1.53}$$

dove T è il numero dei timestep e $n = 2$ corrisponde ai due canali di polarità [17]. Per le immagini statiche gli autori usano invece $T_0 = 1$ e $n = 3$, i tre canali di colore.

Il primo modulo di patch embedding, SPEDS-1, opera su patch spaziali 4×4 [17], e la rappresentazione prodotta dal primo stage ha dimensione

$$T \times \frac{H}{4} \times \frac{W}{4} \times C, \tag{1.54}$$

con un numero di token pari a

$$N_1 = \frac{H}{4}\frac{W}{4}. \tag{1.55}$$

SPEDS-2 applica un ulteriore sottocampionamento 2×2 e porta la rappresentazione a

$$T \times \frac{H}{8} \times \frac{W}{8} \times 2C, \tag{1.56}$$

mentre SPEDS-3 produce

$$T \times \frac{H}{16} \times \frac{W}{16} \times 4C. \tag{1.57}$$

A ogni sottocampionamento il numero delle posizioni spaziali si riduce di un fattore quattro e quello dei canali raddoppia.

La gerarchia serve a qualcosa di più che risparmiare calcolo. Gli stage iniziali conservano una rappresentazione spaziale relativamente fine, mentre quelli successivi sacrificano gradualmente la localizzazione precisa in favore di feature più

numerose e semanticamente più elaborate. È il principio che ha reso efficaci le architetture gerarchiche nella visione convenzionale, qui realizzato mantenendo una rappresentazione spiking lungo tutto il backbone.

Il variare del rapporto N/D incide direttamente sulla scelta dell'attenzione più conveniente. Nel primo stage è plausibile che valga $N \gg D$, con molti token spaziali e relativamente pochi canali. In questo regime QKTA è la scelta naturale, perché comprime la dimensione dei canali per decidere sui token. Nei livelli profondi la situazione tende a rovesciarsi, con meno token e più canali, e un unico meccanismo di attenzione non è più necessariamente ottimale in tutti gli stage.

Per questo QKFormer adotta una Mixed Spiking Attention Integration [17]. Nella configurazione predefinita QKTA viene impiegata negli stage iniziali, mentre nell'ultimo viene reintrodotta la SSA. QKCA rappresenta un'alternativa per gli stadi in cui i canali sono numerosi e i token pochi.

La scelta trova conferma negli esperimenti di ablazione [17]. Usare esclusivamente QKTA o esclusivamente QKCA dà risultati inferiori alle combinazioni miste, mentre configurazioni come QKTA+SSA, QKCA+SSA o QKTA+QKCA raggiungono prestazioni paragonabili alla sola SSA con meno parametri e, soprattutto, con meno memoria dedicata all'attenzione. Su CIFAR10-DVS, per esempio, la configurazione QKTA+SSA raggiunge l'84,0% con circa 1,50 milioni di parametri, mentre le varianti basate su un solo tipo di Q-K Attention scendono all'80,7% [17].

Ne emerge un principio che tornerà utile nel resto della tesi. L'operatore più efficiente non va necessariamente applicato in modo uniforme a tutta la rete, perché in un'architettura gerarchica la natura della rappresentazione cambia con la profondità, e stage diversi possono trarre vantaggio da operatori diversi.

Nonostante queste modifiche, un blocco QKFormer resta vicino all'encoder di un Transformer. Indicando con X_{l-1} il suo ingresso,

$$X'_l = \mathrm{QKA}(X_{l-1}) + X_{l-1}, \tag{1.58}$$

$$X_l = \mathrm{SMLP}(X'_l) + X'_l, \tag{1.59}$$

dove QKA indica QKTA o QKCA e SMLP è lo Spiking Multi-Layer Perceptron [17]. Entrambi i sottoblocchi sono accompagnati da un collegamento residuo.

La figura rende visibile un aspetto che tra le equazioni rischia di sfuggire. La gerarchia di QKFormer riguarda lo spazio e i canali, non il tempo. Prima di sviluppare questo punto conviene però esaminare SPEDS, il componente che rende possibile il passaggio da uno stage all'altro.

Un collegamento residuo ordinario presuppone che ingresso e uscita abbiano le stesse dimensioni. Un patch embedding gerarchico modifica però sia il numero dei token sia quello dei canali, e l'ingresso non può essere sommato direttamente all'uscita del sottocampionamento. QKFormer risolve il problema inserendo una proiezione leggera W_d sul ramo residuo [17],

$$Y = \mathcal{F}(X, \{W_i\}) + \mathrm{SN}(W_d X). \tag{1.60}$$



Figura 1.9: Struttura gerarchica di QKFormer. La risoluzione spaziale viene progressivamente ridotta attraverso SPEDS, mentre aumenta la dimensionalità dei canali. Nella configurazione predefinita QKTA viene utilizzata negli stage ad alta numerosità di token e SSA nello stage finale. La dimensione temporale T viene preservata lungo il backbone.

Nel lavoro originale W_d è una convoluzione 1×1 con passo (stride) maggiore di uno, che adegua in un solo colpo risoluzione e numero dei canali. Il ramo principale $\mathcal{F}$ è invece una sequenza di convoluzioni, batch normalization, neuroni spiking e max-pooling, di cui gli autori propongono due possibili ordinamenti, entrambi pensati per raggiungere la nuova risoluzione conservando un percorso residuo attraverso il sottocampionamento [17].

Da qui il nome di *deformed shortcut*. Il collegamento diretto non può più essere una semplice identità e va «deformato» per portare l'ingresso nello stesso spazio dell'uscita. La motivazione è semplice. Senza collegamento diretto, ogni patch embedding diventa un collo di bottiglia attraverso cui l'intera rappresentazione deve essere ricostruita, mentre con SPEDS una parte dell'informazione attraversa il cambio di scala per una via diretta.

Il beneficio è confermato sperimentalmente. Negli esperimenti originali, sostituire SPEDS con il precedente Spiking Patch Splitting degrada le prestazioni, mentre inserire SPEDS in Spikformer le migliora sia su CIFAR100 sia su CIFAR10-DVS, dove l'accuratezza passa dall'80,90% all'82,20% [17].

SPEDS è dunque un elemento importante del successo di QKFormer. Porta con sé, però, una conseguenza che diventerà rilevante quando l'architettura verrà ridotta a un regime molto più compatto, perché il patch embedding non è un componente gratuito. La struttura gerarchica sposta una parte consistente dell'elaborazione nei moduli di transizione tra le scale, dove ridurre la risoluzione richiede convoluzioni, normalizzazioni, pooling, neuroni spiking e un ramo residuo proiettato. Analisi successive della stessa famiglia architetturale hanno evidenziato che, in QKFormer, gli stage di patch embedding possono rappresentare una quota importante del costo energetico complessivo. L'aspetto sarà ripreso nella sezione 1.6. Per ora basta osservare che una struttura pensata per ridurre il costo dell'attenzione può spostare parte del budget verso il front-end gerarchico, un compromesso che diventa evidente quando si passa da modelli con milioni o decine di milioni di parametri a modelli con poche centinaia di migliaia.

I risultati del lavoro originale aiutano a misurare la distanza tra i due regimi. Per i task neuromorfici Zhou et al. impiegano una configurazione che chiamano mini QKFormer, con circa 1,50 milioni di parametri, blocchi distribuiti sui tre stage secondo lo schema $\{0, 1, 1\}$ e dimensione massima di embedding pari a 256. Con 16 timestep raggiunge il 98,6% su DVS128-Gesture e l'84,0% su CIFAR10-DVS [17].

Questa configurazione non va confusa con la Mini-QKFormer sviluppata nella presente tesi e descritta nella metodologia. Dietro la somiglianza dei nomi si celano due regimi diversi, perché la baseline di questo lavoro scende a circa 5×10^5 parametri, un terzo della già ridotta configurazione neuromorfica originale. In un regime simile, componenti che in QKFormer occupano una frazione modesta del budget possono diventare dominanti. Per questo QKFormer è un riferimento prezioso ma non può essere ridotto meccanicamente. Una versione compatta

deve preservarne la gerarchia, l'attenzione spike-driven efficiente e la progressiva riduzione dei token, rivalutando però il peso relativo di ogni componente quando larghezza e profondità diminuiscono.

Resta infine la questione più importante per il percorso di questa tesi, cioè il ruolo assegnato al tempo. Nella formulazione di QKFormer, Q e K hanno dimensione $T \times N \times D$, e si potrebbe pensare che l'attenzione operi congiuntamente sui timestep. Non è così.

Gli autori precisano che la dimensione T resta separata soltanto negli strati neuronali, mentre negli altri operatori viene di fatto fusa con quella del batch [17]. Per questi operatori, in pratica, ogni timestep è un campione a sé. A ciascun istante t la Q-K Attention riceve quindi una rappresentazione $X_t \in \mathbb{R}^{N \times D}$ e costruisce il proprio vettore di attenzione sulla sola attività di quel timestep,

$$A_t = \mathcal{A}(Q_t), \tag{1.61}$$

senza che compaia un termine della forma

$$\mathcal{A}\left(Q_t, Q_{t-1}, Q_{t-2}, \dots\right). \tag{1.62}$$

Il legame tra X_t e le rappresentazioni precedenti è affidato principalmente agli stati dei neuroni LIF distribuiti lungo la rete.

La divisione dei ruoli è netta. La Q-K Attention mette in relazione token e canali all'interno di X_t attraverso la selezione binaria, mentre il legame tra X_{t-k} e X_t passa principalmente per lo stato neuronale. QKFormer possiede una struttura fortemente gerarchica nello spazio, ma nessuna gerarchia esplicita corrispondente lungo il tempo.

Ciò non significa che il modello ignori l'informazione temporale. Gli esperimenti originali mostrano anzi che il numero dei timestep influenza le prestazioni. Su CIFAR100 l'accuratezza cresce dal 78,51% con $T = 1$ all'81,15% con $T = 4$, fino all'81,30% con $T = 6$ [17]. Tra le limitazioni del lavoro, gli autori riconoscono inoltre il costo associato ai timestep e indicano come direzione futura la ricerca di architetture efficienti capaci di operare con meno passi temporali [17].

QKFormer, in altre parole, utilizza il tempo, ma l'interazione temporale non è l'oggetto del suo meccanismo di attenzione. Per questa tesi la distinzione è essenziale. QKFormer fornisce un backbone in cui l'elaborazione spaziale e di canale è già stata progettata con cura per essere gerarchica ed efficiente, e consente così di porre una domanda più specifica di quella affrontata dal lavoro originale.

Una volta portata l'architettura in un regime fortemente compatto, la sola memoria LIF è il modo più efficace di spendere il budget destinato alla dimensione temporale?

Il lavoro originale non offre elementi per rispondere, e la risposta va cercata sperimentalmente.

La baseline compatta adottata in questo lavoro nasce da una scelta conservativa. Mantiene i principi di QKFormer con fedeltà sufficiente a costituire un riferimento riconoscibile, ma ne riduce drasticamente la scala. Soltanto dopo averne caratterizzato il comportamento diventa possibile decidere quali componenti preservare, quali semplificare e quali sostituire.

La letteratura più recente rafforza la legittimità di questa domanda. STAtten mostra che sostituire in QKFormer l'attenzione puramente spaziale con un meccanismo capace di correlare feature appartenenti a blocchi di timestep può migliorare le prestazioni sui benchmark neuromorfici [19]. Più di recente, il Multi-Delay Mixer ha mostrato che arricchire QKFormer con informazioni provenienti da ritardi diversi migliora sia l'interazione temporale misurata sia l'accuratezza [20].

Questi risultati non indicano ancora quale soluzione adottare, ma delineano con chiarezza il passo successivo. QKFormer offre una forte gerarchia spaziale e un'attenzione efficiente, e affida la memoria temporale ai neuroni. Resta da capire come arricchirne la capacità temporale senza sacrificarne la compattezza. La sezione successiva affronta questa domanda, passando dalle architetture generali alle strategie progettate specificamente per modellare il tempo.

Tabella 1.3: Evoluzione delle dimensioni della rappresentazione lungo QKFormer. La gerarchia riduce progressivamente il numero dei token e aumenta la dimensionalità dei canali, mentre il tipo di attenzione della configurazione standard varia in funzione del regime N/D. Le dimensioni e la strategia di mixed attention seguono la formulazione originale di QKFormer [17].

Stage	**Output**	**Risoluzione relativa**	**Canali**	**Attenzione predefinita**	**Ruolo principale**
Stage 1	$T \times H/4 \times W/4 \times C$	Alta	C	QKTA	Preservare dettaglio spaziale e selezionare token
Stage 2	$T \times H/8 \times W/8 \times 2C$	Intermedia	$2C$	QKTA	Costruire rappresentazioni gerarchiche
Stage 3	$T \times H/16 \times W/16 \times 4C$	Bassa	$4C$	SSA (o QKCA nelle varianti)	Modellare feature profonde con meno token

1.5 Strategie per la modellazione temporale esplicita

L'analisi di QKFormer lascia aperta una questione precisa. La rete possiede una dimensione temporale reale, perché i neuroni conservano uno stato da un timestep

al successivo e ciò che accade all'istante $t-k$ può quindi influenzare il calcolo all'istante t. L'attenzione, però, opera prevalentemente sulle relazioni interne alla rappresentazione corrente. Memoria e attenzione svolgono così funzioni distinte. La prima porta nel presente una sintesi del passato, la seconda organizza le relazioni tra le feature disponibili nel presente.

La stessa separazione ricorre in numerosi Spiking Transformer. Questi modelli non sono privi di temporalità, ma la affidano in larga parte, e implicitamente, alla dinamica neuronale. Lavori recenti hanno cominciato a mettere in discussione questa scelta. STAtten osserva che l'attenzione puramente spaziale cattura efficacemente le relazioni tra feature dello stesso timestep, senza però costruire direttamente relazioni tra feature appartenenti a tempi diversi [19]. Il Multi-Delay Mixer compie un passo ulteriore e introduce una metrica dedicata, il Temporal Interaction Coefficient (TIC), che descrive come la rilevanza si distribuisca tra i timestep. Con essa mostra che in diversi meccanismi di attenzione spiking la dipendenza è fortemente concentrata sul timestep corrente, mentre i contributi storici restano molto più deboli [20].

Occorre dunque distinguere la persistenza dell'informazione nello stato neuronale dall'accesso esplicito a feature di istanti diversi. Di seguito si esaminano due approcci che perseguono questo accesso, l'attenzione spazio-temporale a blocchi e il mixing mediante ritardi. La risoluzione temporale con cui il segnale viene fornito al modello resta una scelta separata, già discussa nella sezione 1.1.

STAtten applica l'attenzione a blocchi temporali [19]. Partendo da feature spiking $X \in \mathbb{R}^{T\times N\times D}$, anziché calcolare la self-attention separatamente per ciascun t raggruppa timestep consecutivi in blocchi di dimensione B e considera congiuntamente le feature comprese tra iB e $(i+1)B-1$. All'interno di un blocco, token e istanti vengono disposti lungo un unico asse, come se ogni coppia formata da una posizione e da un istante costituisse un token a sé. La correlazione coinvolge così rappresentazioni che differiscono sia per posizione sia per istante,

$$\mathrm{STAtten}(X[b]) = \mathrm{LIF}\left(Q[b]K[b]^{\top}V[b]\alpha\right), \tag{1.63}$$

con $[b] = [iB : (i+1)B]$. Con $T = 4$ e $B = 2$, per esempio, vengono elaborati insieme i timestep (0,1) e (2,3). L'informazione del primo elemento di un blocco non deve più sopravvivere nello stato neuronale fino al secondo, perché le feature di entrambi partecipano direttamente alla stessa operazione di attenzione.

La scelta di procedere a blocchi ha due motivazioni. Una correlazione completa tra tutti i timestep sarebbe possibile concatenando l'intera sequenza lungo la dimensione dei token, ma richiederebbe di conservare e correlare tutte le feature temporali, con un costo di memoria elevato. Inoltre, osservano gli autori, con rappresentazioni binarie la similarità e il numero di neuroni attivi nelle operazioni QKV tendono a diminuire al crescere della distanza temporale, e correlare indiscriminatamente istanti molto lontani diventa poco produttivo [19].



Figura 1.10: Tre modalità differenti di integrare il passato. La dinamica neuronale comprime ricorsivamente la storia nello stato corrente, l'attenzione spazio-temporale confronta direttamente feature appartenenti a più timestep, il temporal mixing mediante ritardi accede selettivamente a istanti passati diversi senza costruire una correlazione spazio-temporale completa.

La dimensione B del blocco definisce così un orizzonte di interazione esplicita. Al suo interno l'attenzione vede insieme spazio e tempo, mentre oltre il blocco la relazione torna a essere affidata ad altri meccanismi, a cominciare dalla dinamica neuronale.

STAtten dimostra che attenzione temporale e spaziale possono essere integrate restando nella stessa classe di complessità che gli autori dichiarano per l'attenzione puramente spaziale, $\mathcal{O}(TND^2)$, grazie alla riorganizzazione a blocchi e all'assenza della softmax [19]. Il costo pratico, tuttavia, non resta invariato sotto ogni aspetto. Le feature di più timestep devono essere disponibili contemporaneamente e la pressione sulla memoria aumenta, tanto che lo stesso lavoro indica proprio la memoria temporale tra le ragioni per evitare una correlazione globale sull'intera sequenza.

Per un modello fortemente compatto la considerazione pesa in modo particolare. La capacità temporale non si misura soltanto in parametri, e un modulo che non aggiunge quasi alcun peso può comunque richiedere buffer di dimensioni significative.

Ci si può chiedere allora se sia davvero necessario aprire l'attenzione a più timestep contemporaneamente, o se basti portare nel presente alcune informazioni storiche selezionate. Il Multi-Delay Mixer (MD-Mixer) esplora la seconda possibilità, e il suo punto di partenza è molto vicino al problema affrontato in questa tesi. Prima di proporre il nuovo modulo, Shi et al. cercano di misurare quanto i meccanismi di self-attention esistenti dipendano dal passato [20]. Le misure ottenute con il TIC mostrano una rilevanza temporale concentrata attorno al timestep corrente, che gli autori interpretano come segno di una scarsa varietà dei pattern temporali rappresentati dalle attenzioni puramente spaziali.

La soluzione proposta non aggiunge una seconda attenzione lungo T, ma modifica le feature che raggiungono quella spaziale. Per ciascun canale i, MD-Mixer considera K rami temporali con ritardi $d_i^{(1)}, \ldots, d_i^{(K)}$ e coefficienti apprendibili $\alpha_i^{(1)}, \ldots, \alpha_i^{(K)}$, e calcola la feature arricchita come

$$\widetilde{X}_{t,i} = \sum_{k=1}^{K} \alpha_i^{(k)} X_{t-d_i^{(k)},i}. \tag{1.64}$$

Ogni canale può così attingere contemporaneamente a più istanti della propria storia, e canali diversi possono imparare ritardi diversi.

L'operazione cambia la forma stessa della memoria. In un LIF i contributi passati si sovrappongono secondo un decadimento geometrico, $X_t + \lambda X_{t-1} + \lambda^2 X_{t-2} + \ldots$, mentre il mixing a ritardi multipli può costruire combinazioni del tipo

$$\alpha_1 X_{t-1} + \alpha_2 X_{t-4} + \alpha_3 X_{t-7}, \tag{1.65}$$

senza che X_{t-4} debba sopravvivere indirettamente a quattro aggiornamenti successivi del potenziale. La differenza è qualitativa. Nel LIF è la distanza temporale a

fissare implicitamente il peso di ogni contributo, attraverso il decadimento. Nel mixing a ritardi è la rete a stabilire quanta importanza attribuire a ciascun ritardo.

I ritardi stessi, inoltre, vengono appresi anziché fissati a mano. Poiché uno spostamento discreto nel tempo non è derivabile rispetto all'indice del ritardo, gli autori adottano una procedura *soft-to-hard*. All'inizio dell'addestramento ogni ritardo è una combinazione morbida, e quindi differenziabile, di più spostamenti possibili, che una temperatura progressivamente ridotta rende via via più netta fino a ricondurla a una scelta discreta [20].

L'ablazione sui ritardi è istruttiva. Su CIFAR10-DVS, ritardi casuali e fissi producono soltanto un incremento modesto rispetto alla baseline, mentre ritardi appresi portano un beneficio sensibilmente maggiore [20]. Il vantaggio sembra dunque derivare dalla capacità di scegliere quale passato rendere disponibile, più che dalla semplice iniezione di attività passata.

Nell'architettura completa il mixer viene usato per arricchire temporalmente Key e Value, mentre la Query resta allineata all'istante corrente. La Query interroga dunque il presente, ma le Key e i Value a cui si rivolge, $\widetilde{K}_t$ e $\widetilde{V}_t$, contengono anche informazioni selezionate dal passato. L'attenzione spaziale successiva opera così su feature che incorporano già una forma di mixing temporale.

Per un sistema con risorse hardware limitate questa architettura è interessante perché separa nettamente due funzioni, prima il mixing temporale e poi l'attenzione spaziale. Per ampliare l'accesso alla storia non serve rendere spazio-temporale l'intera attenzione. Il prezzo da pagare è un buffer sufficiente a recuperare le feature ritardate. Se il ritardo massimo è $d_{\max}$, un'implementazione in streaming, che elabora la sequenza un passo alla volta man mano che arriva, deve disporre almeno concettualmente di una memoria capace di restituire gli stati richiesti entro quell'orizzonte. Il costo non scompare, ma cambia forma, passando da una correlazione temporale globale a una memoria temporale più strutturata e selettiva.

Il principio tornerà nei capitoli successivi. Un meccanismo temporale può risultare conveniente non perché elimini lo stato storico, ma perché rende esplicito, e limitato, quale stato occorra conservare.

I due approcci offrono dunque due forme diverse di accesso esplicito alla storia. L'attenzione a blocchi confronta congiuntamente più rappresentazioni, il mixing a ritardi ne seleziona alcune e le combina con le feature correnti. Entrambi affiancano la memoria neuronale, con esigenze diverse di conservazione e trasformazione delle feature, e per confrontarli occorre guardare, oltre che alle prestazioni, a parametri, operazioni e stato persistente. La sezione seguente introduce le grandezze necessarie a descrivere questi costi, distinguendo le stime algoritmiche dalle misure su dispositivo.

1.6 Efficienza neuromorfica e vincoli hardware

Nelle sezioni precedenti l'efficienza è comparsa come un vincolo che accompagna la progettazione, ed è ora il momento di chiarire che cosa significhi in concreto. Un modello spiking viene spesso definito efficiente dal punto di vista energetico perché comunica mediante spike binari e presenta attivazioni sparse. Queste proprietà sono il presupposto di una computazione a basso consumo, ma non la garantiscono. Il vantaggio si materializza soltanto quando algoritmo, flusso dei dati e hardware sono progettati in modo che l'assenza di attività si traduca in lavoro effettivamente evitato. È questa la differenza tra efficienza teorica ed efficienza di sistema.

In una rete convenzionale una trasformazione lineare

$$\mathbf{y} = W\mathbf{x}, \tag{1.66}$$

si calcola con una successione di operazioni di moltiplicazione e accumulo (multiply-and-accumulate, MAC). Per ciascun elemento non nullo dell'ingresso si moltiplica l'attivazione per il peso corrispondente e si somma il risultato all'accumulatore,

$$a \leftarrow a + w_i x_i. \tag{1.67}$$

Se invece l'attivazione presinaptica è uno spike binario $s_i \in \{0,1\}$, il prodotto si riduce a

$$w_i s_i = \begin{cases} w_i, & s_i = 1, \\ 0, & s_i = 0. \end{cases} \tag{1.68}$$

Quando $s_i = 1$ basta accumulare il peso, quando $s_i = 0$ idealmente non occorre fare nulla. La computazione sinaptica si riconduce così a un semplice accumulo (accumulate, AC),

$$a \leftarrow a + w_i, \tag{1.69}$$

eseguito soltanto in presenza dello spike.

In questo consiste il significato hardware più concreto del paradigma spike-driven. Il punto non è che il tensore contenga soltanto zeri e uni, ma che la sparsità delle attivazioni possa tradursi in un numero minore di operazioni effettivamente eseguite.

Spike-driven Transformer porta questo principio all'interno dell'attenzione [15]. La comunicazione binaria tra neuroni consente di sostituire diverse moltiplicazioni con mascheramenti e addizioni sparse, e gli autori formulano esplicitamente l'obiettivo di una computazione in cui nessuna operazione venga attivata in assenza di un evento. Nei loro esperimenti il basso tasso di attivazione (firing rate) di Query e Key è uno dei fattori principali della riduzione stimata del costo della self-attention, che nelle configurazioni analizzate risulta molto ampia rispetto all'attenzione convenzionale [15].

Il legame tra sparsità e costo si può rendere esplicito. Se uno strato richiederebbe F_l operazioni in forma densa e presenta un firing rate medio

$$\rho_l = \frac{\text{numero di spike attivi}}{\text{numero totale di possibili attivazioni}}, \tag{1.70}$$

una stima diffusa del numero di operazioni sinaptiche (synaptic operations, SOP) che esegue in forma spiking è

$$\text{SOP}_l \approx \rho_l \, T \, F_l, \tag{1.71}$$

dove T è il numero dei timestep. Le operazioni dense vengono cioè ripetute a ogni passo, ma soltanto la frazione corrispondente agli spike effettivamente emessi dà luogo a calcolo.

QKFormer adotta questa impostazione nella propria analisi energetica, con

$$\text{SOP}_l = fr_l \, T \, \text{FLOP}_l, \tag{1.72}$$

dove fr_l è il firing rate dello strato e FLOP_l il numero di operazioni in virgola mobile che esso richiederebbe in forma densa [17]. Per tradurre i conteggi in una stima teorica del consumo, gli autori assegnano a ogni operazione un costo di riferimento, 4,6 pJ per una MAC a 32 bit e 0,9 pJ per una AC, valori relativi a una tecnologia a 45 nm [17]. In forma semplificata, l'energia stimata è

$$E_{\text{SNN}} \approx E_{\text{MAC}} N_{\text{MAC}} + E_{\text{AC}} N_{\text{AC}}, \tag{1.73}$$

con $N_{\text{AC}} \simeq \sum_l \text{SOP}_l$.

La formula è utile per confrontare rapidamente configurazioni diverse, purché la si legga per quello che è, un modello energetico algoritmico e non la misura del consumo di un sistema fisico. Non tiene necessariamente conto dell'accesso alla memoria, del trasferimento dei dati, dei buffer, dell'interconnessione, della logica di controllo, della rete di distribuzione del clock o del lavoro necessario a individuare gli spike attivi, né del costo di mantenere gli stati neuronali. Due modelli con la stessa stima secondo l'equazione (1.73) possono quindi comportarsi in modo molto diverso una volta realizzati. Per questo, nella presente tesi, i conteggi di MAC, AC e SOP verranno usati come indicatori comparativi (proxy) finché non sarà disponibile una sintesi hardware completa, e non saranno presentati come misure fisiche di energia.

La prima grandezza da affiancare al costo computazionale è proprio il firing rate. La natura binaria degli spike rende economica la singola operazione, ma il vantaggio svanisce se quasi tutti i neuroni emettono spike quasi sempre. Per un'attività binaria $S \in \{0,1\}^{T \times N}$ il firing rate globale vale

$$\rho = \frac{1}{TN} \sum_{t=1}^{T} \sum_{i=1}^{N} S_{t,i}. \tag{1.74}$$

Una rete con $\rho = 0{,}05$ attiva in media il 5% dei neuroni a ogni timestep. Una rete con $\rho = 0{,}60$ resta formalmente spiking, ma offre molte meno occasioni di saltare le operazioni associate agli zeri (*zero-skipping*).

Il risparmio energetico potenziale richiede dunque il concorso di tre condizioni, ossia spike binari, attività sparsa e un hardware capace di sfruttarla, e basta che ne venga meno una perché il vantaggio si riduca drasticamente. La terza condizione è spesso sottovalutata. Una GPU è estremamente efficiente nell'elaborare in parallelo tensori densi, ma la presenza di molti zeri non comporta automaticamente l'eliminazione dei corrispondenti thread, accessi o moltiplicazioni. La survey di Karamimanesh et al. sottolinea proprio questo limite. Le ampie regioni inattive dei tensori spiking non vengono sfruttate naturalmente dall'architettura GPU, e il modello di von Neumann aggiunge un costo considerevole dovuto al continuo trasferimento dei dati tra memoria e unità di calcolo [21].

Ne segue che il tempo di inferenza di una SNN misurato in PyTorch su GPU può dare un'idea fuorviante della sua efficienza neuromorfica. Una rete algoritmicamente sparsa può benissimo essere eseguita da un backend che tratta gran parte del calcolo come denso.

L'hardware dedicato cerca invece di rendere la sparsità visibile al datapath, cioè all'insieme delle unità che trasportano ed elaborano i dati. Un esempio diretto è l'acceleratore sparso per Spike-driven Transformer proposto da Li et al. [22]. Gli autori non si limitano a memorizzare matrici binarie dense, ma codificano la posizione degli spike validi, e le computazioni successive operano sugli indirizzi degli elementi attivi ignorando quelli nulli. La stessa rappresentazione serve operatori diversi, dai layer lineari al max-pooling fino alla SDSA, e si basa principalmente su addizioni e confronti anziché su moltiplicazioni [22]. L'idea si coglie con un esempio elementare. In forma densa il segnale [0,0,1,0,0,1,0,0] viene elaborato come un vettore di otto elementi, mentre una rappresentazione per indirizzi lo descrive con le sole posizioni degli spike attivi, [2,5].

Il vantaggio è massimo quando la rappresentazione è molto sparsa e si assottiglia man mano che gli spike diventano frequenti, perché il costo della codifica, degli indirizzi e della logica di controllo finisce per eguagliare quello del calcolo evitato. È il compromesso messo in evidenza dalla survey sulle implementazioni FPGA. Sfruttare la sparsità conviene soltanto finché i circuiti necessari a gestirla, come gating, multiplexer, codifica sparsa, code FIFO e logica di controllo, costano meno delle operazioni e degli accessi in memoria che permettono di eliminare [21].

La sparsità non è quindi una grandezza da massimizzare in sé. Un firing rate più basso può richiedere una logica di controllo più complessa, una rappresentazione molto irregolare può lasciare inattive parte delle unità parallele, e la codifica degli indirizzi può occupare memoria che una rappresentazione densa non richiederebbe. Il valore della sparsità dipende in definitiva dalla granularità con cui l'hardware è in grado di sfruttarla.



Figura 1.11: La sparsità delle attivazioni produce un beneficio hardware soltanto quando l'architettura evita realmente le operazioni associate agli elementi nulli. Il vantaggio netto dipende dal rapporto tra computazione risparmiata e overhead introdotto dalla gestione della rappresentazione sparsa.

Accanto alle attivazioni occorre considerare una seconda forma di memoria, che nelle reti feed-forward convenzionali non compare nella stessa misura, cioè lo stato temporale. Ogni neurone LIF deve conservare almeno il proprio potenziale di membrana. Indicando con N_l il numero di unità dello strato l e con b_l i bit usati per ciascuno stato, l'occupazione minima dovuta ai potenziali è approssimativamente

$$M_{\text{state}} = \sum_l N_l b_l. \tag{1.75}$$

Questa memoria, però, non viene usata una volta sola. Lo stato $V_t^{(l)}$ deve persistere fino al passo successivo, dove diventa $V_{t+1}^{(l)}$, e durante un'inferenza in streaming deve quindi risiedere in una memoria con banda sufficiente per essere letto, aggiornato e riscritto a ogni timestep.

Una capacità temporale esplicita può accrescere ulteriormente questo costo. Un mixer temporale che legge gli ultimi $d_{\max}$ timestep richiede un buffer storico (*history buffer*) che, per una feature di N elementi rappresentati con b bit, occupa

$$M_{\text{history}} \propto d_{\max} N b. \tag{1.76}$$

Un'attenzione temporale che elabora insieme B timestep richiede invece che più feature siano disponibili contemporaneamente. È anche per questo che STAtten procede a blocchi invece di correlare l'intera sequenza, contenendo l'occupazione di memoria rispetto a un'attenzione temporale completa [19].

Per queste ragioni, nella presente tesi, gli elementi di stato e i buffer temporali sono considerati metriche a pieno titolo, al pari dei parametri. Un modulo temporale con pochi parametri può rivelarsi poco conveniente se costringe a conservare una grande quantità di feature storiche. Il numero dei parametri indica quanto spazio occupino i pesi, lo stato indica quanto contesto debba restare vivo mentre il modello opera, e le due grandezze non sono intercambiabili. Un modello con 400 mila parametri ma milioni di stati persistenti può essere più difficile da collocare interamente sul chip di uno con più pesi e uno stato dinamico molto più contenuto.

Da questo punto di vista le FPGA (Field-Programmable Gate Array) costituiscono una piattaforma di notevole interesse. Mettono a disposizione risorse logiche e di memoria riconfigurabili, che possono essere organizzate attorno al flusso computazionale della specifica applicazione. A differenza di una CPU general-purpose, non costringono ogni operazione a passare per una pipeline di istruzioni fissa, e rispetto a una GPU consentono di costruire datapath specializzati per precisioni numeriche, accessi alla memoria e operatori irregolari. Le risorse principali sono di quattro tipi. Le Look-Up Table (LUT) realizzano gran parte della logica combinatoria, i flip-flop (FF) conservano stati e registri, le Block RAM (BRAM) forniscono memoria sul chip e i blocchi DSP eseguono in modo efficiente le operazioni aritmetiche, in particolare moltiplicazioni e accumuli.

Quanto ciascuna risorsa venga sollecitata dipende dalla natura del modello. Un'architettura dominata dalle moltiplicazioni mette sotto pressione i DSP, un modello a bassa precisione e ricco di mascheramenti sposta il costo verso le LUT e il routing, una SNN con grandi feature map può essere limitata soprattutto dalla BRAM che ospita potenziali di membrana e buffer. Affermare che un modello richiede meno operazioni in virgola mobile non equivale quindi a descriverne il costo hardware.

La survey sulle implementazioni FPGA documenta infatti una grande eterogeneità, perché modelli neuronali, precisione numerica, strategie di parallelizzazione e organizzazione della memoria impiegano combinazioni molto diverse di LUT, FF e DSP [21]. Gli autori sottolineano inoltre che la valutazione di un'implementazione deve includere, oltre all'accuratezza e alla fedeltà con cui viene approssimata la dinamica neuronale, le risorse occupate, la frequenza operativa e il tempo di esecuzione [21].



Figura 1.12: Possibile decomposizione funzionale di uno Spiking Transformer su FPGA. Oltre alle unità di calcolo, il costo hardware comprende memoria dei pesi, stato neuronale persistente, buffer temporali, routing degli spike e logica necessaria a sfruttarne la sparsità.

Un datapath specializzato può produrre benefici considerevoli. Du et al. realizzano un acceleratore per Spike Transformer su una Xilinx Zynq UltraScale+ ZCU102, con unità dedicate alle addizioni sparse, ai neuroni LIF e alle operazioni lineari [23]. Il progetto non utilizza alcun DSP e impiega principalmente LUT, FF e BRAM, per un totale di 90 552 LUT, 62 216 FF e 158 BRAM e una potenza complessiva di circa 1,2 W. Nelle condizioni considerate raggiunge 986,3 GOP/s e 758,6 GOP/J, valori superiori a quelli delle altre implementazioni FPGA con cui gli autori si confrontano [23].

Ai fini di questa tesi conta meno il valore assoluto, che dipende da modello, piattaforma e definizione di operazione, e più ciò che il risultato dimostra. Una

struttura spike-driven può condurre a un datapath profondamente diverso da quello di un Transformer convenzionale, perché l'assenza di moltiplicazioni sposta la realizzazione dall'uso massiccio dei DSP a una logica specializzata per accumulo e controllo.

Gli stessi risultati suggeriscono anche una cautela di metodo. Throughput, potenza ed efficienza energetica si possono confrontare correttamente soltanto a parità di piattaforma, precisione, frequenza, modello e carico di lavoro, e un valore espresso in GOP/J non è una proprietà intrinseca dell'architettura neurale.

Questi esempi illustrano una regola generale della progettazione congiunta di algoritmo e hardware (*co-design*). Ottimizzare un operatore non significa ottimizzare il sistema. Una moltiplicazione resa quasi gratuita non rende gratuito portare gli operandi dove servono, una memoria temporale molto espressiva smette di essere economica se richiede trasferimenti continui, e un'attenzione sparsa perde il suo vantaggio se la gestione degli indici finisce per dominare il datapath.

Per questo le trasformazioni architetturali studiate nella tesi verranno valutate anche in termini di movimento dei dati potenziale. Un profiler software non può restituire il costo esatto del routing o degli accessi alla BRAM in una futura implementazione, ma grandezze come la dimensione delle feature, il numero degli stati e la profondità dei buffer sono indicatori molto più informativi del solo numero di parametri.

La precisione numerica aggiunge un'ulteriore dimensione al problema. Durante lo sviluppo le reti vengono di norma addestrate in virgola mobile, mentre una FPGA trae grande vantaggio da rappresentazioni a precisione ridotta. Quantizzare i P pesi di un modello da 32 a b bit ne riduce idealmente l'occupazione da $32P$ a bP bit, con un fattore di compressione

$$C = \frac{32}{b}. \tag{1.77}$$

Con pesi a 4 bit, per esempio, il guadagno teorico rispetto a FP32 è pari a $8\times$. Una precisione minore consente inoltre di aumentare la densità delle operazioni aritmetiche e di ridurre la banda richiesta dagli accessi in memoria.

Le SNN presentano però un'asimmetria peculiare. Le attivazioni possono essere già binarie, mentre pesi e potenziali di membrana restano normalmente a più bit. Quantizzare una SNN non significa quindi produrre un generico «modello a bassa precisione», ma intervenire su componenti con distribuzioni e funzioni molto diverse. Ridurre la precisione può alterare la rappresentazione costruita dall'attenzione e la dinamica con cui vengono generati gli spike, e richiede perciò di verificare le prestazioni del modello quantizzato [24]. Pesi, correnti e stati vanno trattati separatamente, perché svolgono funzioni diverse e non condividono necessariamente la stessa precisione.

In una SNN con dinamiche temporali complesse il costo complessivo può essere visto, in prima approssimazione, come funzione di molte variabili,

$$C_{\mathrm{HW}} = f\left(P, b_w, b_s, S_{\mathrm{state}}, T, \rho, N_{\mathrm{MAC}}, N_{\mathrm{AC}}, M_{\mathrm{buffer}}, D_{\mathrm{move}}\right), \tag{1.78}$$

dove b_w e b_s indicano le precisioni di pesi e stati e D_{move} il volume di dati da movimentare. L'espressione non pretende di essere un modello fisico. Serve a ricordare che nessuna metrica, da sola, può rappresentare l'efficienza di un'architettura.

Per QKFormer questa osservazione ha un significato particolare. La Q-K Attention riduce la memoria associata alle relazioni tra token, ma il costo dell'architettura comprende anche i patch embedding, le trasformazioni delle feature e i percorsi residui [17], e la valutazione deve quindi abbracciare l'intero backbone. La conseguenza per questa tesi è diretta. In un regime molto compatto l'attenzione non è necessariamente la voce di costo principale, e intervenire soltanto su di essa rischierebbe di mancare il bersaglio. Un front-end spaziale che conserva feature ad alta risoluzione su molti canali e molti timestep può dominare sia il numero delle operazioni sia la quantità di stato.

Ne discende la regola progettuale che guida il resto del lavoro. La capacità di un modello va giudicata per il luogo in cui viene spesa, non soltanto per la sua quantità, ed è questa regola, più che un generico vincolo di risorse imposto dall'esterno, a legare la prospettiva hardware alla ricerca architetturale. Ridurre i canali delle prime feature map può liberare meno parametri di una modifica profonda del classificatore, ma ridurre in misura molto maggiore gli stati di membrana e le operazioni ripetute a ogni timestep. Allo stesso modo, un piccolo mixer temporale negli stage profondi aggiunge pochi parametri e richiede un buffer storico ridotto, e può risultare più conveniente di un'attenzione temporale applicata alle feature ad alta risoluzione. In questo consiste, concretamente, un'allocazione della capacità consapevole dell'hardware (*hardware-aware capacity allocation*).

In vista di una futura implementazione su FPGA, quattro domande diventano centrali.

1. I pesi possono risiedere prevalentemente sul chip?
2. Gli stati neuronali e temporali trovano posto nella BRAM disponibile?
3. Le operazioni possono essere parallelizzate senza saturare LUT e DSP?
4. La sparsità è sufficiente a giustificare la logica necessaria a sfruttarla?

Se la risposta alla prima o alla seconda è negativa, il sistema deve ricorrere più spesso alla memoria esterna, con costi di latenza ed energia che possono superare quelli dell'aritmetica. Se è negativa la terza, il throughput resta limitato dalle risorse di calcolo. Se lo è la quarta, un'implementazione sparsa può risultare meno conveniente di un datapath regolare.

In questo lavoro la caratterizzazione computazionale comprende parametri, stato persistente, firing rate, operazioni e buffer temporali. Una futura realizzazione su FPGA dovrebbe affiancare a queste grandezze le misure ottenute dalla sintesi e dall'esecuzione, ovvero l'occupazione di LUT, FF, BRAM e DSP, la frequenza di clock, la latenza, il throughput, la potenza e l'energia per inferenza. Disporre delle prime non significa avere già ottenuto le seconde, e il testo manterrà sempre esplicita la distinzione tra una stima energetica e un'energia misurata sull'hardware. Un modello energetico può orientare correttamente il progetto, ma soltanto la realizzazione fisica rende osservabili nella loro forma effettiva costi come il routing, il traffico verso la memoria e la logica di controllo.

Tabella 1.4: Metriche utilizzabili ai differenti livelli di valutazione.

Livello	**Metrica**	**Che cosa misura**	**Perché è rilevante**
Architettura	Parametri P	Dimensione dei pesi	Storage e bandwidth
Dinamica	State elements	Quantità di stato persistente	Footprint delle membrane
Temporale	History buffer	Feature storiche da conservare	BRAM e data movement
Attività	Firing rate ρ	Densità degli spike	Opportunità di sparse execution
Computazione	MAC	Operazioni dense/multibit	Pressione su DSP/arithmetic units
Computazione	AC / SOP	Accumuli spike-triggered	Costo event-driven potenziale
Precisione	bit-width	Rappresentazione di pesi e stati	Memoria, bandwidth, datapath
Sintesi FPGA	LUT / FF	Logica e registri	Capacità implementativa
Sintesi FPGA	BRAM	Memoria on-chip	Pesi, membrane, buffer
Sintesi FPGA	DSP	Moltiplicatori/accumulatori dedicati	Costo delle parti non spike-driven
Runtime	Latency / throughput	Tempo e rate di elaborazione	Requisiti real-time
Energia	Power / energy per sample	Costo fisico dell'inferenza	Efficienza applicativa

La tabella 1.4 mostra quanto il numero dei parametri, pur importante, offra una visione parziale del problema. In un sistema temporale la memoria che resta viva durante l'inferenza può pesare quanto quella dei pesi. Il costo fisico è il risultato di una catena di scelte. L'architettura determina i pattern di attività, questi si riflettono sulle operazioni e sullo stato richiesto, e il mapping sul dispositivo ne stabilisce le conseguenze in termini di energia e latenza.

Ogni scelta temporale lascia una traccia sul sistema. Lo stato LIF richiede memoria persistente, un mixer a ritardi aggiunge buffer storici, un'attenzione spazio-temporale impone che più feature siano disponibili insieme. Due strategie con lo stesso guadagno predittivo possono quindi avere conseguenze hardware molto diverse, e nel regime compatto studiato in questa tesi il loro confronto fa parte della domanda scientifica, anziché essere rimandato a una successiva fase di ingegnerizzazione. L'obiettivo non è costruire il modello con il minor numero di operazioni, né quello più accurato a qualunque costo, ma trovare una soluzione in cui l'informazione temporale aggiuntiva valga le risorse necessarie a rappresentarla.

Si chiude così il percorso teorico del capitolo, che ha seguito l'informazione dal segnale prodotto dalla event camera alla sua discretizzazione, dalla memoria neuronale all'attenzione spiking, fino alla modellazione temporale esplicita e alle conseguenze hardware delle diverse scelte. La descrizione della baseline e della metodologia sperimentale applicherà questi criteri ai modelli effettivamente implementati, tenendo distinte le scelte relative all'input, i meccanismi architetturali e le condizioni di valutazione.

Capitolo 2

Baseline compatta e metodologia sperimentale

Ogni modifica architetturale discussa nel seguito viene misurata rispetto a un riferimento, e quella misura vale soltanto quanto la precisione con cui il riferimento è stato fissato. Questo capitolo lo definisce in tutte le sue parti. Descrive i quattro dataset su cui si svolge la sperimentazione, il modo in cui il flusso di eventi diventa l'ingresso della rete di partenza, l'architettura della baseline compatta ispirata a QKFormer e la ricetta con cui viene addestrata. Stabilisce poi come i modelli vengono valutati e confrontati, quali tipi di controllo sperimentale vengono impiegati e con quali grandezze se ne caratterizza il costo computazionale.

La baseline non è concepita come un avversario da battere, ma come un punto fisso da cui partono tre percorsi distinti, ciascuno dei quali modifica un solo fattore. Il primo ridistribuisce la capacità all'interno della rete lasciando invariato l'ingresso. Il secondo cambia il clock dell'input tenendo ferma l'architettura. Il terzo arricchisce la memoria temporale a parità di input. Perché i tre percorsi restino leggibili, ciò che in ciascuno di essi rimane costante va dichiarato una volta per tutte, ed è questo il compito delle pagine che seguono.

2.1 Dataset

La sperimentazione si svolge su quattro benchmark neuromorfici di classificazione, scelti perché affidano all'informazione temporale ruoli molto diversi. In DVS-Lip le classi sono parole pronunciate, distinte da movimenti labiali brevi e spesso simili tra loro, e l'ordine in cui si susseguono le configurazioni della bocca è parte essenziale del segnale. DVS128 Gesture raccoglie gesti ampi e in larga parte ripetitivi, di durata variabile, alcuni dei quali si distinguono soltanto per il verso del movimento. CIFAR10-DVS deriva da immagini statiche rese visibili al sensore

attraverso un movimento imposto, e la sua dinamica è prodotta dall'acquisizione più che dal contenuto. THU-EACT-50-CHL, infine, contiene azioni umane complete, riprese da un sensore a risoluzione più elevata e con durate molto variabili da un campione all'altro. Un meccanismo temporale utile in uno di questi scenari non lo è necessariamente negli altri, ed è proprio questa eterogeneità a rendere il confronto informativo.

2.1.1 DVS-Lip

DVS-Lip è un dataset di lettura labiale a eventi, introdotto insieme alla rete MSTP [25]. Raccoglie registrazioni di 100 parole inglesi pronunciate da 40 parlanti, acquisite con una event camera DAVIS346 e rilasciate come regioni di 128×128 pixel attorno alla bocca. La partizione ufficiale assegna al training 30 parlanti, con 14 896 campioni, e al test i restanti 10, con 4 975 campioni [25]. Le registrazioni sono brevi, con una durata mediana di circa 1,08 s e massima di 1,89 s, e relativamente povere di eventi, circa ottomila al secondo in mediana. Il segnale utile si concentra in un'area ristretta dell'immagine e in variazioni di forma molto fini.

Il vocabolario è costruito in modo da includere parole visivamente confondibili. Gli autori ne individuano 25 coppie, per un totale di 50 parole, alle quali si aggiungono le restanti 50 [25]. Questa struttura permette di affiancare all'accuratezza complessiva due accuratezze parziali, Acc1 sul gruppo delle parole confondibili e Acc2 sull'altro, che vengono riportate anche in questo lavoro.

2.1.2 DVS128 Gesture

DVS128 Gesture è stato acquisito con un sensore DVS128, a 128×128 pixel, e comprende 11 classi, dieci gesti di mani e braccia e una classe residua che raccoglie movimenti diversi [26]. Le registrazioni coinvolgono 29 soggetti in diverse condizioni di illuminazione, e la partizione ufficiale riserva al test i soggetti dal 24 al 29 [26]. Ogni registrazione contiene più gesti consecutivi, delimitati da annotazioni temporali. In questo lavoro un campione coincide con un segmento annotato, e la sua origine temporale con l'inizio del segmento. Il training ufficiale ne contiene 1 176, che durano in mediana circa 6,4 s ma in alcuni casi superano i 18 s, con circa cinquantamila eventi al secondo in mediana. Diverse classi differiscono soltanto per il braccio impiegato o per il verso di rotazione, e questa proprietà esclude trasformazioni come il ribaltamento orizzontale, che scambierebbero tra loro classi diverse.

2.1.3 CIFAR10-DVS

CIFAR10-DVS converte in eventi 10 000 immagini del dataset CIFAR-10, mille per ciascuna delle dieci classi [27]. Le immagini, visualizzate su un monitor, vengono fatte muovere con un movimento regolare e ripetuto davanti a un sensore DVS128, così che i loro contorni generino eventi [27]. La risoluzione è di 128×128 pixel e le registrazioni hanno durate molto uniformi, con una mediana di circa 1,30 s e un massimo di 1,46 s. Sono anche le più dense del gruppo, con oltre centocinquantamila eventi al secondo in mediana. Poiché il contenuto è statico, la struttura temporale riflette soprattutto la traiettoria imposta durante l'acquisizione, e il dataset rappresenta un termine di confronto in cui è lecito attendersi che la memoria temporale pesi meno che negli altri casi.

Il dataset non prevede una partizione ufficiale. In questo lavoro i campioni con indice da 0 a 99 di ciascuna classe formano un insieme riservato (holdout) di 1 000 registrazioni, escluso dallo sviluppo, mentre i restanti 9 000 costituiscono l'insieme di sviluppo.

2.1.4 THU-EACT-50-CHL

THU-EACT-50 è un benchmark per il riconoscimento di azioni umane con event camera, articolato in 50 classi [28]. Accanto alla versione principale, gli autori rilasciano THU-EACT-50-CHL, acquisita con un sensore DAVIS346 a 346×260 pixel, che è la versione adottata in questo lavoro. Comprende 2 330 registrazioni di 18 soggetti, ripartite dagli autori tra training e test in modo che nessun soggetto compaia in entrambi. La durata mediana è di circa 2,7 s, ma la distribuzione ha una coda lunga e alcuni campioni si avvicinano ai 20 s, mentre il numero di eventi per registrazione spazia da circa diecimila a oltre quattro milioni. Rispetto agli altri tre dataset, la risoluzione maggiore moltiplica il numero di posizioni spaziali, e dunque di token, che la rete deve elaborare a ogni passo.

2.1.5 Partizioni di sviluppo

Le partizioni di test ufficiali non partecipano allo sviluppo. Tutte le decisioni, dalla configurazione dell'input alla scelta del checkpoint da valutare, si basano su una partizione di validazione ricavata dal solo insieme di training, indicata nel seguito come validazione di sviluppo (*development validation*).

Su DVS-Lip i 14 896 campioni del training ufficiale vengono suddivisi in modo deterministico e stratificato per classe, con 11 901 campioni di addestramento e 2 995 di validazione. Su DVS128 Gesture la validazione è formata dai segmenti di cinque soggetti del training ufficiale, l'8, l'11, il 17, il 19 e il 22, per un totale di 264 segmenti contro i 912 destinati all'addestramento. Su CIFAR10-DVS l'insieme di sviluppo viene diviso in modo stratificato, con 90 campioni per classe in validazione,



Figura 2.1: Esempi di campioni dei quattro dataset. Per ciascuno sono mostrati alcuni frame di conteggio, con gli eventi ON e OFF in colori distinti, e l'andamento nel tempo della frequenza di eventi dell'intera registrazione.

900 in tutto, e 8 100 in addestramento. Su THU-EACT-50-CHL, infine, i soggetti 8, 10 e 14 del training ufficiale formano la validazione, con 399 registrazioni contro le 1 387 di addestramento, mentre le 544 registrazioni del test ufficiale restano riservate.

Le partizioni sono definite una volta per tutte, prima di qualsiasi esperimento, e condivise da tutti i modelli, così che ogni confronto avvenga esattamente sugli stessi campioni. La tabella 2.1 riassume queste informazioni insieme alle principali statistiche delle registrazioni.

Tabella 2.1: Sintesi dei dataset e delle partizioni di sviluppo. La partizione riservata è il test ufficiale, tranne che per CIFAR10-DVS, per il quale coincide con l'insieme di holdout. Le durate sono calcolate sulle sole partizioni di sviluppo e riportano il valore mediano e, tra parentesi, il massimo.

Dataset	**Classi**	**Risoluz.**	**Addestr.**	**Valid.**	**Riservati**	**Durata [s]**
DVS-Lip	100	128×128	11 901	2 995	4 975	1,08 (1,89)
DVS128 Gesture	11	128×128	912	264	288	6,36 (18,46)
CIFAR10-DVS	10	128×128	8 100	900	1 000	1,30 (1,46)
THU-EACT-50-CHL	50	346×260	1 387	399	544	2,71 (19,91)

2.2 Rappresentazione dell'input nella baseline

La baseline riceve il flusso di eventi sotto forma di frame di conteggio a finestra fissa (*fixed-window count frames*), la più diretta tra le rappresentazioni introdotte nella sezione 1.1. A partire dall'origine temporale del campione si considera una finestra $[0, D_{\text{win}})$, la si suddivide in T bin consecutivi della stessa durata secondo l'equazione (1.5), e in ciascun bin si contano separatamente, pixel per pixel, gli eventi ON e quelli OFF, come nell'equazione (1.9). Il campione diventa così un tensore

$$X = \big(F_0, \ldots, F_{T-1}\big) \in \mathbb{N}_0^{T \times 2 \times H \times W}, \qquad \Delta t = \frac{D_{\text{win}}}{T}, \tag{2.1}$$

con un canale per ciascuna polarità. Gli istogrammi di eventi a due canali sono tra le rappresentazioni più diffuse nell'apprendimento profondo su dati neuromorfici [29, 2]. La variante adottata qui si caratterizza per due scelte, una durata dei bin comune a tutti i campioni di un dataset e una finestra di durata prefissata, indipendente dalla lunghezza della registrazione. Nel seguito la si indicherà semplicemente come rappresentazione a finestra fissa.

In tutti i dataset la sequenza conta $T = 40$ passi. Si tratta di un budget comune di passi, non di una risoluzione temporale comune, perché la durata della finestra, e con essa quella dei bin, è scelta separatamente per ciascun dataset in base alla

distribuzione delle durate di sviluppo (tabella 2.2). Su DVS-Lip e CIFAR10-DVS una finestra di 2 s con bin da 50 ms contiene per intero anche le registrazioni più lunghe, e lo stesso vale su DVS128 Gesture per una finestra di 20 s con bin da 500 ms. Su THU-EACT-50-CHL la coda lunga delle durate impone un compromesso. Una finestra estesa fino al campione più lungo avrebbe richiesto bin di mezzo secondo, mentre la finestra di 8 s conserva bin da 200 ms e il 99,06% degli eventi di sviluppo, al prezzo di troncare due sole registrazioni della validazione.

I conteggi sono interi esatti, memorizzati su 8 bit. Se un pixel accumulasse in un singolo bin più di 255 eventi la codifica verrebbe interrotta con un errore, anziché saturare silenziosamente il valore, ma nessun campione di sviluppo raggiunge questo limite. Il massimo osservato va da 17 eventi per pixel e per bin su DVS-Lip a 236 su DVS128 Gesture, dove i bin sono dieci volte più lunghi. La rete riceve i conteggi convertiti in virgola mobile, senza alcuna normalizzazione.

Tabella 2.2: Parametri della rappresentazione a finestra fissa. Le ultime due colonne si riferiscono alle partizioni di sviluppo. La frazione di finestra occupata è il rapporto tra la durata mediana delle registrazioni e la durata della finestra.

Dataset	**Finestra** D_{win}	**Bin** Δt	**Passi** T	**Eventi conservati**	**Finestra occupata**
DVS-Lip	2 s	50 ms	40	100%	54%
DVS128 Gesture	20 s	500 ms	40	100%	32%
CIFAR10-DVS	2 s	50 ms	40	100%	65%
THU-EACT-50-CHL	8 s	200 ms	40	99,06%	34%

Una finestra prefissata ha un effetto collaterale che conviene rendere esplicito fin d'ora. Le registrazioni più brevi della finestra lasciano vuoti gli ultimi bin, che non sono padding ma osservazioni valide di un intervallo senza attività, secondo la distinzione discussa a proposito dell'equazione (1.13). La rete li elabora comunque, e il readout descritto più avanti ne include i contributi nella propria media. L'entità del fenomeno varia molto da un dataset all'altro. Rapportando la durata mediana alla finestra, la parte di sequenza effettivamente occupata da eventi è di circa il 65% su CIFAR10-DVS e il 54% su DVS-Lip, ma scende al 34% su THU-EACT-50-CHL e al 32% su DVS128 Gesture, dove un campione tipico trascorre più di due terzi dei passi senza ricevere alcun evento. Le conseguenze di questa scelta, insieme alle rappresentazioni che modificano il clock dell'input, sono analizzate nel capitolo dedicato al clock dell'input. Nei capitoli che precedono quell'analisi la rappresentazione a finestra fissa resta invariata, così che ogni differenza osservata possa essere attribuita all'architettura.



Figura 2.2: Costruzione della rappresentazione a finestra fissa. Il flusso di eventi viene suddiviso in $T = 40$ bin della stessa durata a partire dall'origine del campione, e in ogni bin si contano separatamente gli eventi ON e OFF di ciascun pixel. I bin successivi all'ultimo evento restano vuoti ma fanno parte della sequenza elaborata dalla rete. In basso, la distribuzione delle durate di sviluppo di ciascun dataset rispetto alla finestra adottata.

2.3 Architettura della baseline compatta

La baseline compatta è un'implementazione autonoma, scritta per questo lavoro, che conserva i principi di QKFormer descritti nella sezione 1.4 riducendone drasticamente la scala. Non è una riproduzione fedele del codice originale, e le differenze sono riassunte alla fine della sezione. Il backbone si articola in due stadi gerarchici. Un primo patch embedding porta l'ingresso a un ottavo della risoluzione con 64 canali, un blocco con Q-K Token Attention elabora questa rappresentazione, un secondo patch embedding ne dimezza ancora la risoluzione portando i canali a 128, e un blocco con spiking self-attention completa l'elaborazione prima del readout. La figura 2.3 ne mostra la struttura, la tabella 2.3 le dimensioni.



Figura 2.3: Architettura della baseline compatta. Il primo patch embedding riduce la risoluzione di un fattore otto con un ramo principale di quattro convoluzioni e una scorciatoia diretta dalla prima mappa di feature. Seguono un blocco con Q-K Token Attention, un secondo patch embedding e un blocco con spiking self-attention. Il readout media le feature finali su spazio e tempo prima della testa lineare. Le dimensioni indicate si riferiscono a un ingresso di 128×128 pixel.

2.3.1 Neurone e convenzioni di calcolo

Tutti i neuroni della rete sono LIF, discretizzati secondo lo schema di Eulero dell'equazione (1.24) e dotati di soft reset (1.19). Per un neurone che riceve la corrente X_k al passo k,

$$H_k = V_{k-1} + \frac{X_k - V_{k-1}}{\tau_d}, \qquad S_k = \Theta\big(H_k - V_{\mathrm{th}}\big), \qquad V_k = H_k - S_k V_{\mathrm{th}}, \tag{2.2}$$

con $\tau_d = 2$, $V_{\mathrm{th}} = 1$ e potenziale azzerato all'inizio di ogni campione. Fanno eccezione i neuroni che producono le decisioni dell'attenzione, descritti più avanti,

la cui soglia è dimezzata. Durante l'addestramento la derivata dello spike viene sostituita dalla surrogata sigmoidale dell'equazione (1.28) con $\alpha = 4$, e il termine di reset è trattato come una costante nel passaggio all'indietro.

Con $\tau_d = 2$ il fattore di decadimento vale $\lambda = 1/2$. In assenza di ingresso il potenziale si dimezza a ogni passo, e la costante di tempo fisica equivalente dell'equazione (1.26) vale $\tau_{\text{eq}} = \Delta t / \ln 2 \approx 1{,}44\,\Delta t$. Lo stesso neurone, con gli stessi parametri, dimentica dunque in tempi fisici molto diversi a seconda del dataset, circa 72 ms su DVS-Lip e CIFAR10-DVS, circa 290 ms su THU-EACT-50-CHL e circa 720 ms su DVS128 Gesture. La relazione descrive un integratore isolato al di sotto della soglia, ma basta a mostrare che la scelta del clock fissa anche l'orizzonte della memoria neuronale.

Tutti gli altri operatori, cioè convoluzioni, normalizzazioni e proiezioni lineari, vengono applicati a ogni passo in modo indipendente, come se i T passi fossero campioni distinti. Nella baseline l'unico canale attraverso cui l'informazione passa da un istante al successivo è quindi lo stato dei neuroni, come in QKFormer (1.61). Le convoluzioni sono prive di bias, perché ciascuna è seguita da batch normalization, e i loro pesi, come quelli delle proiezioni lineari, sono inizializzati secondo lo schema di He et al. [30].

I collegamenti residui sommano l'ingresso di un sottoblocco alla sua uscita a valle dei neuroni, e la somma non attraversa un'ulteriore soglia. I tensori scambiati tra un blocco e l'altro contengono perciò piccoli interi, somme di più spike, anziché valori strettamente binari. La proprietà è condivisa con QKFormer, come mostrano le equazioni (1.58) e (1.59), e ha conseguenze dirette sul conteggio delle operazioni discusso nella sezione 2.7.

2.3.2 Primo patch embedding

Il primo patch embedding traduce i conteggi in spike e riduce la risoluzione di un fattore otto. Il ramo principale è una cascata di quattro convoluzioni 3×3, ciascuna seguita da batch normalization e da un neurone LIF. La prima conserva la risoluzione e porta i canali da 2 a 32, le altre tre dimezzano a turno la risoluzione con passo 2 e portano i canali a 64. In parallelo, un ramo di scorciatoia preleva le feature del primo strato e le porta direttamente alla risoluzione finale con una convoluzione 1×1 di passo 8, seguita anch'essa da normalizzazione e neurone. Le due uscite, entrambe spiking, vengono sommate,

$$E_k^{(1)} = M_k^{(1)} + R_k^{(1)} \in \{0,1,2\}^{64 \times \frac{H}{8} \times \frac{W}{8}}, \tag{2.3}$$

dove $M_k^{(1)}$ e $R_k^{(1)}$ sono gli spike emessi al passo k dal ramo principale e dalla scorciatoia. È lo stesso principio della scorciatoia deformata di SPEDS (1.60), con due differenze. Il sottocampionamento avviene con convoluzioni a passo 2

anziché con max-pooling, e la scorciatoia parte dalla prima mappa di feature per attraversare l'intero cambio di scala in un solo salto.

Questo modulo è l'unico della rete a operare a piena risoluzione. Il suo primo strato produce a ogni passo 32 mappe di 128×128 posizioni, e il peso che ne deriva nel bilancio complessivo risulterà evidente tra poco.

2.3.3 Stadio con Q-K Token Attention

Il primo stadio contiene un unico blocco Transformer, in cui il ruolo dell'attenzione è svolto dalla Q-K Token Attention (QKTA) descritta nella sezione 1.4. Per un ingresso di 128×128 pixel il blocco opera su $16 \times 16 = 256$ token con 64 canali. Query e Key sono ottenute con proiezioni 1×1, normalizzazione e neurone LIF, come nell'equazione (1.44), e i 64 canali sono divisi in 8 teste da 8 canali ciascuna. Per ogni testa h e ogni token n, la somma degli spike della Query sui canali della testa attraversa un neurone con soglia 0,5,

$$a_{h,n} = \mathrm{SN}_{0,5}\Big(\sum_{d \in h} Q_{n,d}\Big), \qquad X'_{n,d} = a_{h,n}\, K_{n,d} \quad \text{per } d \in h, \tag{2.4}$$

e la decisione binaria risultante lascia passare o azzera i canali della Key di quella testa per quel token. Rispetto all'equazione (1.46) la somma è calcolata separatamente per ogni testa, e ciascun token dispone così di otto decisioni indipendenti. Anche il neurone che le produce possiede uno stato, sicché la selezione di un token dipende dalla storia recente della sua Query oltre che dal passo corrente. L'uscita mascherata attraversa poi una proiezione lineare con normalizzazione e neurone, come nell'equazione (1.52).

Il blocco si completa secondo lo schema delle equazioni (1.58) e (1.59). L'uscita dell'attenzione viene sommata al suo ingresso, e il risultato attraversa uno spiking MLP con due strati 1×1, che espandono i canali da 64 a 128 e li riportano a 64, ciascuno con normalizzazione e neurone e a sua volta accompagnato da un collegamento residuo.

2.3.4 Secondo patch embedding e stadio con SSA

Il secondo patch embedding segue lo stesso schema a due rami. Il ramo principale applica una convoluzione 3×3 che porta i canali da 64 a 128 e una seconda convoluzione 3×3 con passo 2, la scorciatoia una convoluzione 1×1 con passo 2, e le due uscite spiking vengono sommate come nell'equazione (2.3). La rappresentazione risultante ha 128 canali a un sedicesimo della risoluzione d'ingresso, cioè 64 token per un ingresso di 128×128 pixel.

Il secondo stadio impiega la spiking self-attention (SSA), con Query, Key e Value spiking (1.37) divise in 8 teste, ora da 16 canali. Per ciascuna testa l'uscita

dell'attenzione è

$$Y = \mathrm{SN}_{0,5}\Big(s\,Q(K^\top V)\Big), \qquad s = \frac{1}{\sqrt{16}}, \tag{2.5}$$

seguita, come nel primo stadio, dalla proiezione di uscita, dal collegamento residuo e da uno spiking MLP che espande i canali da 128 a 256 prima di riportarli a 128. In assenza di softmax il prodotto è associativo, e l'implementazione calcola prima $K^\top V$, una matrice 16×16 per ogni testa, e poi il suo prodotto con la Query. Il risultato coincide con quello di $(QK^\top)V$ nell'equazione (1.38), ma la matrice $N \times N$ delle relazioni tra token non viene mai costruita, e il costo dell'attenzione cresce linearmente con il numero dei token.

2.3.5 Readout

Il readout riduce la rappresentazione finale a un vettore, mediandola prima sulle posizioni spaziali e poi sui passi temporali,

$$z = \frac{1}{T\,N_2} \sum_{k=0}^{T-1} \sum_{n=1}^{N_2} Z^{(2)}_{k,n}, \qquad \hat{y} = W_c\, z + b_c, \tag{2.6}$$

dove $Z^{(2)}_{k,n} \in \mathbb{R}^{128}$ è l'uscita del secondo stadio al passo k e nella posizione n, N_2 il numero dei suoi token, W_c e b_c i parametri di una testa lineare con un'uscita per classe. Le feature mediate sono quelle del flusso residuo, e non sono quindi binarie. La media temporale comprende tutti i T passi della finestra, inclusi quelli successivi all'ultimo evento. La decisione non dipende così da un istante privilegiato, ma da un riassunto dell'intera sequenza in cui ogni passo pesa allo stesso modo.

2.3.6 Bilancio di parametri e stato

La tabella 2.3 riporta, blocco per blocco, le dimensioni delle uscite, i parametri e lo stato persistente, cioè il numero di potenziali di membrana che devono sopravvivere da un passo al successivo. Il totale dei parametri dipende dal numero delle classi e va da 489 098 su CIFAR10-DVS a 500 708 su DVS-Lip, mentre il backbone ne conta 487 808 in tutti i casi. Su THU-EACT-50-CHL, con un ingresso di 346×260 pixel, i due stadi operano rispettivamente su $33 \times 44 = 1\,452$ e $17 \times 22 = 374$ token, e lo stato cresce in proporzione al numero delle posizioni.

Parametri e stato si distribuiscono in modo quasi speculare. I due patch embedding contengono circa due terzi dei parametri, e il secondo da solo quasi la metà, mentre i due blocchi di attenzione e i relativi MLP ne occupano poco meno di un terzo. Lo stato è invece dominato dal primo patch embedding, che lavora sulle mappe a risoluzione più alta e ne ospita oltre l'80%, e il solo primo strato ne contiene quasi la metà. Con circa 1,1 milioni di elementi, lo stato supera di

Tabella 2.3: Struttura della baseline compatta per un ingresso di 128×128 pixel. Le uscite sono indicate come canali $\times$ altezza $\times$ larghezza per ciascun passo. Lo stato conta i potenziali di membrana di tutti i neuroni del blocco. I parametri della testa dipendono dal numero C delle classi, e il totale si riferisce a DVS-Lip.

Blocco	**Operazioni principali**	**Uscita**	**Parametri**	**Stato**
Primo patch embedding	4 conv. 3×3 ($2 \to 32 \to 64 \to 64 \to 64$), scorciatoia 1×1 a passo 8	$64 \times 16 \times 16$	95 360	884 736
Stadio 1	QKTA a 8 teste, MLP $64 \to 128 \to 64$	$64 \times 16 \times 16$	29 440	100 352
Secondo patch embedding	conv. 3×3 ($64 \to 128$), conv. 3×3 a passo 2, scorciatoia 1×1 a passo 2	$128 \times 8 \times 8$	230 144	49 152
Stadio 2	SSA a 8 teste, MLP $128 \to 256 \to 128$	$128 \times 8 \times 8$	132 864	65 536
Readout e testa	media su spazio e tempo, lineare $128 \to C$	C	$129\,C$	0
Totale	DVS-Lip, $C = 100$		500 708	1 099 776

oltre il doppio il numero dei parametri. È un primo indizio di quanto, nel regime compatto, capacità e costo si concentrino lontano dall'attenzione, un aspetto che il capitolo successivo esamina nel dettaglio.

2.3.7 Differenze rispetto a QKFormer

Rispetto alla configurazione neuromorfica di QKFormer descritta nella sezione 1.4, la baseline conserva la sequenza di un blocco QKTA seguito da un blocco SSA, il patch embedding gerarchico con scorciatoia e la separazione tra memoria neuronale e interazioni all'interno del singolo passo. Cambiano invece la scala e alcuni dettagli costruttivi, riassunti nella tabella 2.4. Il backbone è organizzato in due stadi anziché tre, la larghezza massima è dimezzata da 256 a 128 canali, e il numero dei parametri scende a circa un terzo. Il sottocampionamento è affidato a convoluzioni con passo, e la sequenza elaborata conta 40 passi, contro i 16 degli esperimenti originali su dati neuromorfici.

Un dettaglio merita una menzione esplicita. Prima di attraversare il proprio neurone, la proiezione di uscita della SSA viene riportata alla disposizione con il tempo come primo asse, così che lo stato di membrana non mescoli mai campioni diversi del batch. È il punto in cui gli autori di QKFormer hanno segnalato un errore nell'implementazione ufficiale [18], e nella baseline il problema non si presenta.

Tabella 2.4: Confronto tra la configurazione neuromorfica di QKFormer, secondo la descrizione della sezione 1.4, e la baseline compatta.

Aspetto	**QKFormer, configurazione neuromorfica**	**Baseline compatta**
Stadi e blocchi	tre stadi, blocchi {0,1,1}	due stadi, un blocco ciascuno
Attenzione	QKTA, poi SSA	QKTA, poi SSA
Larghezza massima	256 canali	128 canali
Parametri	circa 1,50 milioni	da 489 098 a 500 708
Sottocampionamento	convoluzione, BN, neurone e max-pooling	convoluzioni a passo 2
Passi temporali	16	40

2.4 Protocollo di addestramento

2.4.1 Ricetta di ottimizzazione

Tutti i modelli di questo lavoro vengono addestrati da zero con la stessa ricetta, riassunta nella tabella 2.5. L'ottimizzatore è AdamW [31], con un tasso di apprendimento di 3×10^{-4} e un weight decay di 5×10^{-4}. Il tasso cresce linearmente nelle prime 4 epoche, a partire da un centesimo del valore nominale, e decresce poi con un profilo cosinusoidale [32] fino a 10^{-6} alla fine delle 128 epoche previste. La funzione di perdita è l'entropia incrociata con label smoothing pari a 0,1 [33], la norma globale del gradiente viene limitata a 1 [11] e i calcoli avvengono in precisione mista automatica [34].

L'unica augmentation è un ribaltamento orizzontale con probabilità 0,5 su DVS-Lip. Sugli altri dataset non si applica alcuna trasformazione, e per DVS128 Gesture il ribaltamento sarebbe comunque scorretto, perché scambierebbe tra loro classi direzionali. La ricetta non viene adattata ai singoli modelli né ai singoli dataset. Quando una campagna richiede di modificarla, la modifica viene dichiarata e applicata in modo identico a tutti i modelli del confronto.

Tabella 2.5: Ricetta di addestramento comune a tutti i modelli e a tutti i dataset.

Componente	**Scelta**
Ottimizzatore	AdamW, tasso di apprendimento 3×10^{-4}, weight decay 5×10^{-4}
Schedule	warmup lineare di 4 epoche da 0,01 volte il valore nominale, poi decadimento cosinusoidale fino a 10^{-6}
Durata	128 epoche
Funzione di perdita	entropia incrociata con label smoothing 0,1
Stabilizzazione	limite di 1 sulla norma globale del gradiente
Precisione	precisione mista automatica su GPU
Batch effettivo	32 campioni, ottenuti per accumulo su più microbatch
Augmentation	ribaltamento orizzontale con probabilità 0,5, solo su DVS-Lip
Seed	42 per lo screening, 43 e 44 per le repliche
Selezione del checkpoint	epoca con la massima Macro-F1 sulla validazione di sviluppo

2.4.2 Microbatch e batch normalization

Il batch effettivo di 32 campioni si ottiene accumulando i gradienti di più microbatch, la cui dimensione dipende dalla memoria richiesta da ciascun dataset. Negli addestramenti con la rappresentazione a finestra fissa si accumulano 2 microbatch

da 16 campioni su DVS-Lip e CIFAR10-DVS, 4 da 8 su DVS128 Gesture e 8 da 4 su THU-EACT-50-CHL, le cui mappe sono le più grandi. Le campagne successive dichiarano il proprio frazionamento, sempre identico tra i modelli di uno stesso confronto.

L'accumulo rende equivalente il gradiente delle componenti lineari, ma non il comportamento della batch normalization [35], che durante l'addestramento normalizza con le statistiche del solo microbatch corrente. Poiché gli operatori sono applicati passo per passo con il tempo fuso nella dimensione del batch, quelle statistiche aggregano tutti i passi e tutti i campioni del microbatch, come avviene comunemente nelle SNN addestrate direttamente [36]. Ne discendono due conseguenze. Quattro microbatch da 8 campioni non equivalgono a un batch da 32, e il frazionamento diventa perciò parte del protocollo, da mantenere fisso in ogni confronto. Inoltre, durante l'addestramento ciascun passo viene normalizzato con statistiche che dipendono anche dai passi successivi, sicché la rete non è strettamente causale in questa fase, pur essendolo ogni suo operatore. In inferenza la normalizzazione usa statistiche accumulate e fisse, e le feature di ogni passo tornano a dipendere soltanto dai passi precedenti.

2.4.3 Verifiche preliminari

Nelle campagne multi-dataset e in quelle successive, ogni nuova configurazione affronta una serie di verifiche automatiche prima dell'addestramento completo. Un audit delle partizioni di sviluppo controlla la conservazione degli eventi e il rispetto dei limiti di conteggio nella rappresentazione scelta. Un passaggio in avanti e all'indietro, in singola precisione e in precisione mista, verifica le forme dei tensori, la finitezza di uscite e gradienti e la compatibilità con il microbatch configurato.

Segue un test di sovradattamento controllato. Il modello viene addestrato, senza augmentation né precisione mista, su quattro campioni per ciascuna di al più 16 classi, e deve mantenere per cinque epoche consecutive un'accuratezza superiore al 95% su quello stesso sottoinsieme, con una loss inferiore a 1,5, entro un massimo di 500 epoche e 1 000 passi di ottimizzazione. Il test non dice nulla sulla capacità di generalizzare, ma dimostra che la pipeline è in grado di apprendere e intercetta, a un costo trascurabile, errori che altrimenti emergerebbero soltanto al termine di un addestramento completo. Superate le verifiche, l'addestramento vero e proprio riparte da pesi nuovi.

2.5 Metriche e valutazione

2.5.1 Macro-F1 e metriche complementari

La metrica principale è la Macro-F1, media non pesata degli F1 delle singole classi [37]. Per una classe c, indicati con TP_c, FP_c e FN_c i veri positivi, i falsi positivi e i falsi negativi,

$$P_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c}, \qquad R_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c}, \qquad \mathrm{F1}_c = \frac{2P_cR_c}{P_c + R_c}, \tag{2.7}$$

e la metrica è la media sulle C classi,

$$\text{Macro-F1} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{F1}_c. \tag{2.8}$$

A differenza dell'accuratezza, la Macro-F1 attribuisce a tutte le classi lo stesso peso, indipendentemente dalla loro numerosità, e penalizza un modello che raggiunga un buon risultato complessivo trascurando alcune classi.[1] Accanto ad essa si riportano l'accuratezza top-1 e, su DVS-Lip, le accuratezze Acc1 e Acc2 dei due gruppi di parole descritti nella sezione 2.1.1.

2.5.2 Selezione del checkpoint e incertezza dei confronti

Ogni modello viene valutato sulla validazione di sviluppo al termine di ciascuna epoca, e il checkpoint conservato è quello con la Macro-F1 più alta. Selezione e misura avvengono quindi sugli stessi dati, e il valore riportato è una stima ottimistica della prestazione su dati nuovi, tanto più quanto più la curva di validazione oscilla da un'epoca all'altra. Per questo, a partire dalle campagne multi-dataset, il riepilogo di ogni addestramento registra anche media e deviazione standard della Macro-F1 nelle ultime 16 epoche, un indicatore di stabilità meno esposto alla scelta del massimo.

Quando due modelli vengono confrontati sulla stessa validazione, le loro predizioni sono appaiate campione per campione, e questo consente due strumenti complementari. Il test esatto di McNemar considera soltanto i campioni su cui i due modelli discordano e verifica se gli errori corretti dall'uno e quelli introdotti

[1] Su THU-EACT-50-CHL la media è calcolata su tutte le 50 classi della testa di classificazione, anche se quattro di esse compaiono soltanto in soggetti del training e mancano sia nella validazione di sviluppo sia nel test ufficiale. Per queste classi l'F1 è nullo per costruzione, e il valore massimo raggiungibile della Macro-F1 è pari al 92%. Il vincolo è identico per tutti i modelli e non altera i confronti, ma va tenuto presente nella lettura dei valori assoluti.

dall'altro siano sbilanciati [38, 39]. Il bootstrap appaiato ricampiona con reinserimento i campioni della validazione, ricalcola su ciascuna delle 2 000 repliche la differenza della metrica tra i due modelli e ne ricava un intervallo di confidenza al 95% [40]. I valori di p non sono corretti per i confronti multipli, e vanno letti come indicazioni piuttosto che come verdetti.

Entrambi gli strumenti quantificano l'incertezza dovuta alla dimensione finita della validazione, a checkpoint fissati. Nessuno dei due misura la variabilità dell'addestramento, cioè quanto cambierebbe il risultato ripetendo l'addestramento con un seed diverso, una fonte di variazione che può essere dello stesso ordine delle differenze tra i modelli confrontati [41]. Per questo medie e deviazioni standard vengono riportate soltanto quando esistono repliche indipendenti della stessa configurazione, e le differenze ottenute con un solo seed vengono trattate come indicazioni esplorative.

2.5.3 Valutazione sui prefissi

Su DVS-Lip e DVS128 Gesture la valutazione comprende anche una misura di come la decisione si formi nel tempo. Lo stesso checkpoint viene valutato su prefissi della sequenza, facendo elaborare alla rete soltanto i primi bin e calcolando il readout sui soli passi osservati. I prefissi sono definiti in tempo assoluto, a 250, 500, 1 000, 1 500 e 2 000 ms su DVS-Lip e a 2, 5, 10, 15 e 20 s su DVS128 Gesture, oppure come frazioni della durata di ciascun campione. Un indice sintetico, indicato come PrefixAUC, integra con la regola dei trapezi l'accuratezza sui prefissi assoluti e la normalizza sull'intervallo coperto, che su DVS-Lip va da 250 a 2 000 ms.

I prefissi relativi richiedono di conoscere in anticipo la fine della registrazione, e non corrispondono quindi ad alcuna politica di decisione anticipata realizzabile. Entrambe le misure sono diagnostiche. Non intervengono nella selezione dei modelli e non autorizzano di per sé a ridurre la finestra di ingresso, perché un modello addestrato su sequenze complete non è stato ottimizzato per decidere su sequenze troncate.

2.5.4 Uso delle partizioni riservate

Tutte le metriche riportate nei capitoli successivi sono misurate sulla validazione di sviluppo, salvo indicazione esplicita. Le partizioni riservate, cioè i test ufficiali di DVS-Lip, DVS128 Gesture e THU-EACT-50-CHL e l'holdout di CIFAR10-DVS, restano escluse dallo sviluppo, dalla selezione dei checkpoint e dalla caratterizzazione computazionale. L'unica eccezione riguarda un insieme di analisi diagnostiche, presentate separatamente, in cui alcuni checkpoint già selezionati su DVS-Lip e THU-EACT-50-CHL sono stati valutati in sola inferenza sul test ufficiale, senza

alcun addestramento né selezione basati su quei dati. Quei risultati descrivono il comportamento di modelli già scelti e non vengono utilizzati per sceglierne altri.



Figura 2.4: Uso delle partizioni nel protocollo sperimentale. Addestramento e selezione del checkpoint impiegano esclusivamente le partizioni di sviluppo, ricavate dal training ufficiale. La partizione riservata resta esclusa dallo sviluppo e viene impiegata soltanto per analisi diagnostiche a posteriori su checkpoint già selezionati.

2.6 Disegno dei controlli sperimentali

Il principio che governa i confronti di questo lavoro è semplice da enunciare. Due modelli confrontati differiscono per un solo elemento, e tutto il resto, dati, partizione, rappresentazione dell'input, ricetta, seed, frazionamento in microbatch e regola di selezione, resta identico. Quando l'oggetto dell'esperimento è l'input, è l'architettura a essere tenuta fissa. Una differenza di prestazione può allora essere attribuita all'elemento modificato, entro i limiti di incertezza discussi nella sezione precedente. Lo stesso modulo, però, può essere interrogato in modi diversi, che rispondono a domande diverse (figura 2.5).

Un'ablazione riaddestrata rimuove o sostituisce il modulo e addestra il modello da zero con la stessa ricetta. Misura che cosa riesca a ottenere l'architettura senza quel componente, ed è la sola prova che il componente serva davvero a raggiungere un certo risultato.

Un intervento sul checkpoint modifica invece un modello già addestrato soltanto in inferenza, lasciando invariati pesi e statistiche di normalizzazione. Rientrano in questa categoria la disattivazione di un ramo, l'eliminazione di singoli ritardi in un modulo temporale, la permutazione dell'ordine dei passi in ingresso e la valutazione sui prefissi. Un intervento di questo tipo misura quanto la soluzione appresa dipenda dal componente, non se il componente sia necessario. Una rete addestrata con un modulo può essersi adattata alla sua presenza e degradare vistosamente quando lo si rimuove, anche se un'architettura che ne è priva, addestrata da zero, avrebbe

raggiunto un risultato analogo. Il ruolo dell'attenzione nel secondo stadio viene esaminato proprio combinando le due prospettive. Un test di necessità disattiva la SSA in inferenza sul checkpoint addestrato, lasciando attivi collegamento residuo e MLP, mentre un test di sostituibilità la rimpiazza con un mixer spaziale locale di capacità comparabile e addestra il modello da zero.

Un controllo di capacità, infine, separa l'effetto di un operatore da quello delle risorse che aggiunge. Ogni nuovo modulo porta con sé parametri e operazioni, e un miglioramento potrebbe dipendere semplicemente da questi. Il confronto con alternative di costo comparabile ma prive della struttura in esame, come un allargamento uniforme della rete, un percettrone multistrato con lo stesso numero di parametri o una variante che conserva la forma del modulo eliminandone la proprietà ipotizzata, permette di attribuire il guadagno alla struttura e non alla quantità.



Figura 2.5: Tipologie di controllo sperimentale. L'ablazione riaddestrata confronta due modelli addestrati da zero che differiscono per un solo componente. L'intervento sul checkpoint modifica in sola inferenza un modello già addestrato. Il controllo di capacità sostituisce il componente con un'alternativa di costo comparabile ma priva della struttura in esame.

Perché il confronto sia davvero appaiato occorre anche controllare l'inizializzazione. Nelle varianti che aggiungono un modulo a un'architettura di riferimento, i parametri del nuovo componente vengono generati in un flusso pseudo-casuale separato, oppure i pesi comuni vengono copiati dalla rete di riferimento costruita con lo stesso seed. In questo modo, a parità di seed, i pesi condivisi partono dagli stessi valori e la sequenza dei dati non viene perturbata dalla costruzione del nuovo modulo. L'accorgimento non rende identiche le traiettorie di addestramento, che divergono non appena il modulo modifica i gradienti, ma elimina una fonte di variabilità estranea al confronto.

L'esplorazione procede con un solo seed, il 42, che permette di esaminare molte varianti a costo contenuto. Le configurazioni destinate a diventare un riferimento

vengono replicate con i seed 43 e 44, e solo per queste si riportano medie e deviazioni standard. Nei capitoli successivi i risultati di screening e quelli confermati restano sempre distinti.

2.7 Caratterizzazione computazionale

Il costo di ciascun modello viene caratterizzato in inferenza sul checkpoint selezionato, usando 64 campioni della validazione di sviluppo scelti in modo deterministico alternando le classi, gli stessi per tutti i modelli di un dataset. Ogni profilo è legato al proprio checkpoint attraverso un'impronta crittografica e non viene mai trasferito da un modello a un altro. Le grandezze raccolte riprendono quelle introdotte nella sezione 1.6 e sono riassunte nella tabella 2.6.

Tabella 2.6: Grandezze raccolte nella caratterizzazione computazionale di ciascun checkpoint.

Grandezza	**Che cosa conta**
Parametri	pesi addestrabili, testa inclusa, e occupazione in bit alla precisione di esecuzione
Stato persistente	potenziali di membrana e buffer temporali che devono sopravvivere da un passo al successivo
MAC	moltiplicazioni e accumuli con operandi non binari, come i conteggi in ingresso, le somme residue e le correnti dei moduli temporali
AC potenziali	accumuli con ingresso binario, contati come se ogni possibile spike fosse presente
AC ad attività	accumuli pesati per la densità di spike osservata, nell'ipotesi di un salto ideale degli zeri
SOP	AC potenziali più le operazioni dell'attenzione
Firing rate	frazione di spike emessi da ciascuno strato sui campioni profilati

La distinzione tra MAC e AC segue il tipo di operando, non il tipo di strato. Uno strato che legge spike binari esegue accumuli, mentre lo stesso strato applicato a conteggi o a somme residue esegue moltiplicazioni vere e proprie. Per questo la proprietà dei collegamenti residui descritta nella sezione 2.3.1 pesa sul bilancio, perché una parte delle proiezioni della rete riceve valori interi e non binari.

Per riassumere i conteggi in un'unica grandezza si adotta la stima aritmetica dell'equazione (1.73), già impiegata da QKFormer, con i costi di riferimento per operazioni in virgola mobile a 32 bit in tecnologia a 45 nm riportati da Horowitz [42], cioè 4,6 pJ per una MAC, 0,9 pJ per un'addizione e 3,7 pJ per una moltiplicazione. La stima viene calcolata in due varianti, una densa, che conta tutti gli accumuli

potenziali, e una pesata per l'attività osservata. Restano escluse per costruzione molte voci rilevanti, dagli accessi in memoria al routing, dalla logica di controllo alle correnti di dispersione, dall'aggiornamento dei neuroni alle non linearità e alle divisioni di normalizzazione. Il risultato è dunque una proxy parziale e comparativa del solo costo aritmetico, utile per confrontare architetture a parità di input, e in nessun caso una misura dell'energia consumata. Per la stessa ragione i tempi di esecuzione su GPU non vengono usati come indicatore di efficienza.

Lo stato persistente viene riportato sempre accanto ai parametri. Come si è visto per la baseline, le due grandezze possono differire di un fattore superiore a due e dipendono da scelte architetturali diverse, e un modulo che aggiunge pochi parametri può richiedere un buffer temporale molto più grande.

2.8 Sintesi del protocollo di riferimento

La tabella 2.7 raccoglie gli elementi del protocollo che restano fissi nei capitoli successivi, salvo quando sono essi stessi l'oggetto dell'esperimento. Ogni risultato citato corrisponde a un addestramento identificato univocamente, di cui sono conservati la configurazione completa, l'ambiente software, la versione del codice con l'indicazione di eventuali modifiche non registrate, l'impronta del checkpoint selezionato, la storia dell'addestramento e le metriche di validazione. I nomi con cui modelli e varianti compaiono nel testo sono descrittivi e non coincidono con gli identificatori interni degli esperimenti. La corrispondenza tra i due è riportata in appendice, così che ogni numero possa essere ricondotto alla propria esecuzione.

Con il riferimento così fissato, il capitolo successivo può affrontare la prima delle tre domande sperimentali. Si tratta di capire dove spendere un budget di capacità tanto ridotto, partendo dall'osservazione che, nella baseline, la parte più costosa della rete non è quella che il nome dell'architettura metterebbe in primo piano.

Tabella 2.7: Protocollo di riferimento della sperimentazione.

Elemento	**Riferimento**
Dataset	DVS-Lip, DVS128 Gesture, CIFAR10-DVS, THU-EACT-50-CHL
Partizioni	validazione di sviluppo ricavata dal training ufficiale, partizioni riservate escluse dallo sviluppo
Input	frame di conteggio ON e OFF a finestra fissa, $T = 40$, durata dei bin specifica per dataset
Architettura	baseline compatta a due stadi, QKTA e SSA, circa 0,5 milioni di parametri
Neuroni	LIF con $\tau_d = 2$, $V_{\text{th}} = 1$, soft reset, surrogata sigmoidale con $\alpha = 4$
Readout	media su spazio e tempo, testa lineare
Ricetta	AdamW, 128 epoche, batch effettivo 32, frazionamento in microbatch dichiarato
Metrica	Macro-F1 sulla validazione di sviluppo, con accuratezza e, su DVS-Lip, Acc1 e Acc2
Seed	42 per lo screening, 43 e 44 per le repliche
Costo	parametri, stato persistente, MAC, AC, SOP, firing rate e proxy aritmetica parziale

Capitolo 3

Riallocazione del budget nella baseline compatta

Questo capitolo sarà redatto al termine delle riesecuzioni che isolano i singoli interventi architetturali. La traccia seguente ne definisce i contenuti.

- **Diagnosi della baseline.** Distribuzione di operazioni e stato tra i blocchi, controlli di capacità a larghezza 192 e 256, rimozione della memoria tra passi e momento del readout.
- **Pyramidal spiking patch embedding.** Ridistribuzione del budget nel primo patch embedding.
- **Causal multi-lag channel mixer.** Formulazione, punto di inserimento e rimozione dei ritardi in inferenza come diagnostica di co-adattamento, con quattro ritardi fin dall'inizio.
- **Depthwise spatial mixer.** Sostituzione della Q-K Token Attention a parità di ritardi, controllo con ritardi apprendibili.
- **Stabilità e trasferimento.** Repliche con tre seed per la baseline e per la configurazione finale, confronto sui quattro dataset, controlli di larghezza.
- **Ruolo dell'attenzione.** Test di necessità e di sostituibilità della SSA.
- **Uso condizionale dei ritardi.** Variante con gate di contenuto, controllo statico riaddestrato e trasferimento tra dataset.

Capitolo 4

Il clock dell'input come variabile sperimentale

Nei capitoli precedenti l'ingresso della rete è rimasto fermo. La rappresentazione a finestra fissa, introdotta insieme alla baseline, ha fatto da sfondo a ogni modifica architetturale, e proprio per questo nessuna di quelle modifiche poteva dire qualcosa sull'ingresso stesso. Questo capitolo inverte i ruoli. L'architettura viene congelata, e a cambiare è soltanto il modo in cui il flusso di eventi viene suddiviso in passi.

La questione può sembrare un dettaglio di preprocessing, ma tocca il cuore del problema. La rete conta il tempo in passi. I neuroni perdono una frazione fissa del proprio potenziale a ogni passo, il mixer temporale guarda indietro di un numero fisso di passi, il readout media sui passi. Che cosa sia un passo, però, lo decide il clock dell'input, e con esso si decide che cosa la rete possa davvero ricordare e confrontare. Le pagine che seguono mostrano che questa scelta pesa sulle prestazioni in misura paragonabile a una modifica dell'architettura, e che i suoi effetti cambiano da un dataset all'altro in modi che la sola accuratezza finale non lascerebbe intuire.

Per tutto il capitolo la rete è il modello di riferimento con cui si chiude il capitolo 3. Rispetto alla baseline compatta, il primo patch embedding ha una struttura piramidale, la Q-K Token Attention del primo stadio è sostituita da un mixer spaziale depthwise 3×3, e in entrambi i patch embedding un mixer causale a più ritardi (*causal multi-lag channel mixer*, CMLCM) somma alla corrente z_t di uno strato combinazioni lineari tra canali delle correnti di alcuni passi precedenti,

$$u_t = z_t + \sum_{k=1}^{4} W_k \, z_{t-d_k}, \qquad (d_1, d_2, d_3, d_4) = (1,2,4,8). \tag{4.1}$$

Il secondo stadio conserva la SSA, il readout resta la media su spazio e tempo, e il numero dei parametri rimane attorno al mezzo milione. Di questa architettura

interessa qui un solo aspetto. I ritardi d_k, come la costante τ_d dei neuroni, sono espressi in passi, e il loro significato fisico dipende interamente dal clock.

4.1 I limiti della finestra fissa

La rappresentazione a finestra fissa ha due pregi evidenti, la semplicità e la confrontabilità. Ogni campione produce esattamente 40 passi, ogni passo corrisponde alla stessa durata fisica, e il modello può essere addestrato a batch senza alcun mascheramento. La stessa rigidità che rende comodo questo schema produce però tre effetti collaterali di natura molto diversa. Il primo riguarda ciò che accade dopo la fine della registrazione, il secondo le registrazioni più lunghe della finestra, il terzo ciò che si perde dentro un singolo bin quando il movimento è più rapido del clock.

4.1.1 Una coda che non è silenzio

Ogni registrazione più breve della finestra lascia dopo l'ultimo evento una sequenza di bin vuoti, già incontrata nella sezione 2.2. Misurata sul modello di riferimento, questa coda occupa in mediana il 45% dei passi su DVS-Lip, circa un terzo su CIFAR10-DVS e circa due terzi su DVS128 Gesture e THU-EACT-50-CHL (figura 4.1a). In un campione tipico di DVS128 Gesture la rete trascorre più tempo ad aspettare che a osservare.

Aspettare, però, non significa restare inerti. I potenziali di membrana non si azzerano quando gli eventi finiscono, il mixer temporale continua a restituire le correnti di alcuni passi prima e i collegamenti residui propagano tutto ciò che resta. La figura 4.1b dà la misura del fenomeno. Le operazioni dense, che si ripetono a ogni passo indipendentemente dall'attività, seguono esattamente la quota dei passi vuoti. Ma anche le operazioni sinaptiche innescate dagli spike restano consistenti, tra il 10% e il 28% del totale a seconda del dataset. Una parte non trascurabile del calcolo viene quindi spesa elaborando il silenzio.

Resta da capire se quel calcolo serva a qualcosa, e un intervento sul checkpoint permette di rispondere almeno in parte. Lo stesso modello, addestrato con la finestra fissa, viene valutato interrompendo l'elaborazione al bin che contiene l'ultimo evento e calcolando il readout sui soli passi osservati. Se la coda fosse un riempitivo innocuo, l'accuratezza non dovrebbe cambiare. Accade invece l'opposto (figura 4.2). Su DVS-Lip l'accuratezza crolla di 17,2 punti e su THU-EACT-50-CHL di 14,8, mentre su DVS128 Gesture e CIFAR10-DVS la perdita si ferma a 2,3 e 3,4 punti.

Il modello ha dunque imparato a usare la coda. Le sue decisioni continuano a maturare dopo la fine dell'ingresso, mentre la dinamica interna si assesta e il readout accumula evidenza. Va però ricordato che si tratta di un intervento sul checkpoint,

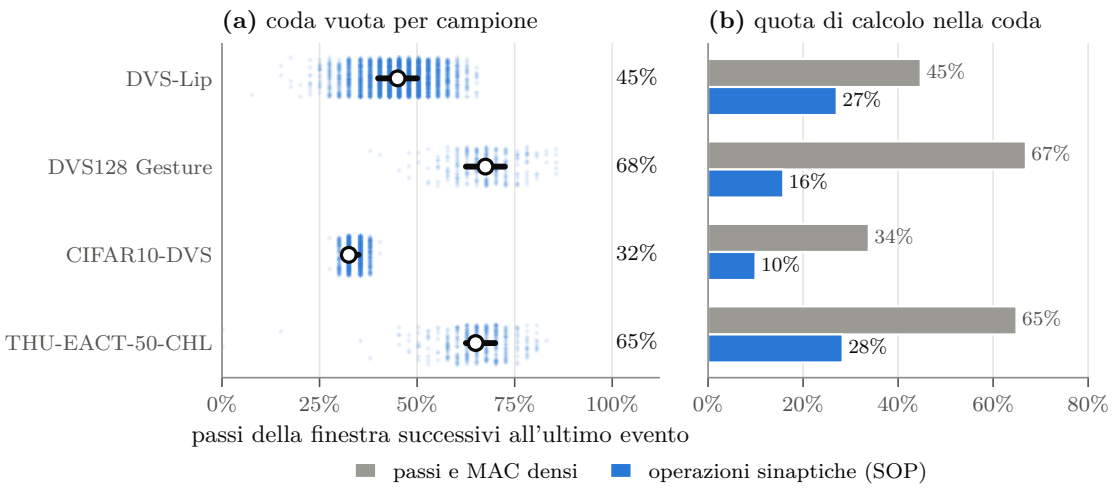


Figura 4.1: Coda vuota della rappresentazione a finestra fissa per il modello di riferimento, sulla validazione di sviluppo. (a) Frazione dei 40 passi successivi al bin che contiene l'ultimo evento, per ciascun campione, con mediana e intervallo interquartile. (b) Quota dei passi, e quindi delle MAC dense, e quota delle operazioni sinaptiche spese dopo l'ultimo evento, sommate sull'intera validazione. Checkpoint con seed 42, valutati in singola precisione.

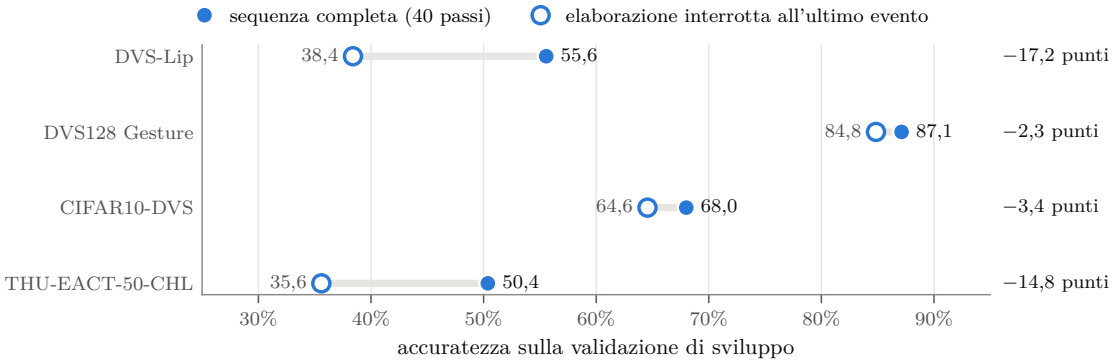


Figura 4.2: Dipendenza dalla coda appresa dal modello addestrato con la finestra fissa. Ogni riga confronta l'accuratezza dello stesso checkpoint quando elabora l'intera finestra e quando l'elaborazione si interrompe al bin che contiene l'ultimo evento. A destra la differenza in punti percentuali. Seed 42, validazione di sviluppo, valutazione in singola precisione.

che misura una dipendenza appresa e non la necessità della coda (sezione 2.6). Interrompere l'elaborazione cambia anche il denominatore della media del readout, e una rete addestrata fin dall'inizio senza coda potrebbe imparare a decidere in modo diverso. È esattamente ciò che verifica la rappresentazione a lunghezza variabile introdotta nella sezione 4.2.

4.1.2 Registrazioni più lunghe della finestra

Il problema opposto, quello delle registrazioni che eccedono la finestra, riguarda soltanto THU-EACT-50-CHL, dove la finestra di 8 s tronca due registrazioni della validazione (sezione 2.2). Anche nell'ipotesi più favorevole, in cui la registrazione completa bastasse a correggere entrambe le predizioni, il guadagno non supererebbe mezzo punto di accuratezza. Il troncamento esiste e va dichiarato, ma non può spiegare le differenze discusse più avanti.

4.1.3 Quando il movimento è più rapido del clock

Il terzo effetto è il più sottile, e lo si osserva con chiarezza sulle rotazioni di DVS128 Gesture. Quattro classi del dataset descrivono un braccio che ruota, il destro o il sinistro, in senso orario o antiorario, e si distinguono soltanto per il verso. Con bin da 500 ms e una rotazione che compie un giro in poco più di un secondo, tra il centro di un bin e quello del successivo il braccio percorre circa mezzo giro. A quel punto due posizioni consecutive sono compatibili tanto con mezzo giro in un verso quanto con mezzo giro nell'altro, e il verso diventa ambiguo. È la ragione per cui una ruota ripresa con troppi pochi fotogrammi al secondo sembra girare al contrario, un caso di aliasing temporale che compare quando il campionamento non è almeno due volte più rapido del moto osservato [43].

La figura 4.3 rende il fenomeno concreto su una registrazione della validazione, una rotazione antioraria del braccio sinistro. La posizione del braccio è approssimata dal baricentro degli eventi di ciascun bin, e l'angolo attorno al centro della scena ne ricostruisce la rotazione. Con bin da 20 ms la traccia di riferimento compie quasi sette giri sempre nello stesso verso. Con i bin da 500 ms della finestra fissa, invece, la metà degli intervalli tra bin consecutivi raggiunge o supera il mezzo giro, e la rotazione ricostruita procede nel verso opposto. Le due rappresentazioni alternative della sezione 4.2, che usano passi più brevi, restano quasi sempre sotto la soglia. Il modello addestrato con la finestra fissa classifica proprio questo campione come una rotazione oraria dello stesso braccio.

Il caso è stato scelto a posteriori, e il baricentro è un'approssimazione grossolana della posizione. Inoltre ogni bin accumula conteggi lungo tutta la sua durata, anziché campionare una singola posizione, e l'esempio non dimostra che i tensori delle due classi siano indistinguibili. Il comportamento dei modelli sull'intera validazione va

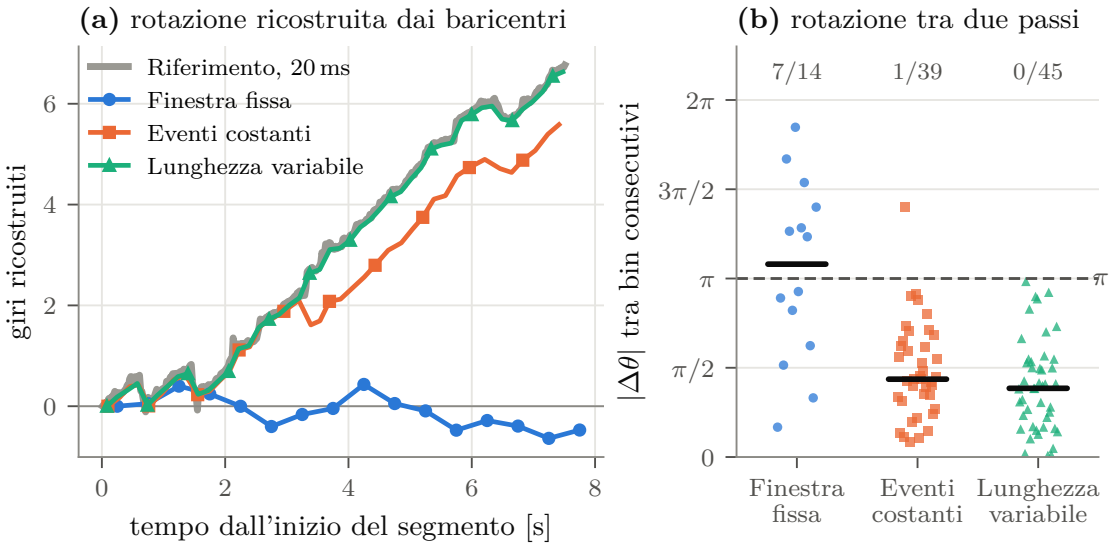


Figura 4.3: Aliasing temporale su una rotazione antioraria del braccio sinistro di DVS128 Gesture (validazione di sviluppo). (a) Giri ricostruiti dall'angolo del baricentro degli eventi di ciascun passo, per una traccia di riferimento a 20 ms e per i tre clock. (b) Variazione angolare della traccia di riferimento tra i centri di passi consecutivi. In alto, il numero di intervalli che raggiungono o superano il mezzo giro. Il baricentro è un'approssimazione della posizione del braccio e il campione è stato scelto a posteriori, sicché la figura illustra il meccanismo senza stimarne la frequenza.

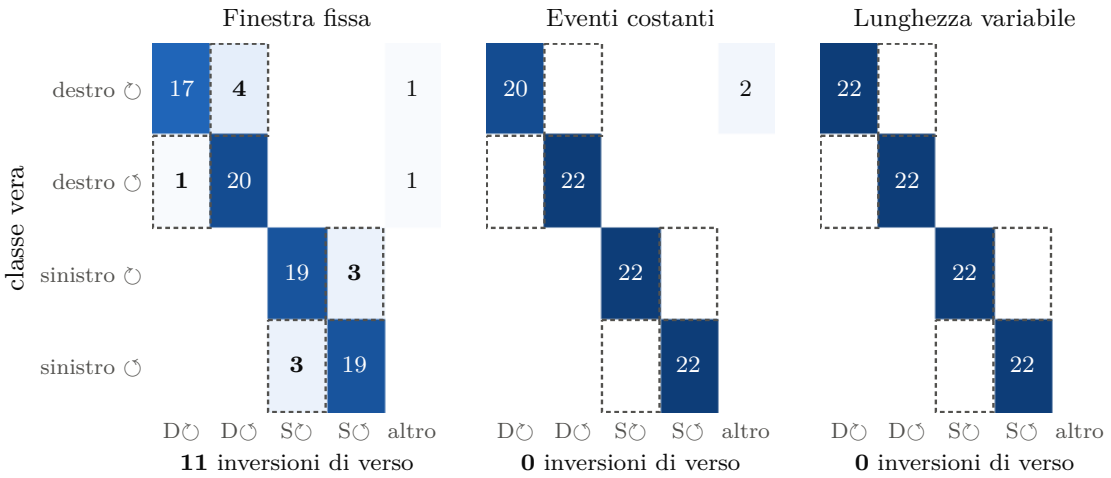


Figura 4.4: Matrici di confusione ristrette alle quattro classi di rotazione di DVS128 Gesture, per il modello di riferimento addestrato con i tre clock (validazione di sviluppo, seed 42). Le righe indicano la classe vera, le colonne la classe predetta, con D e S per il braccio destro e sinistro e le frecce per il verso. Le celle tratteggiate corrispondono alle inversioni di verso a parità di braccio, la colonna *altro* alle predizioni di classi diverse dalle rotazioni.

però nella stessa direzione (figura 4.4). Sugli 88 campioni di rotazione il modello addestrato con la finestra fissa commette 13 errori, e 11 di questi scambiano il verso a parità di braccio. Con le due rappresentazioni a passo più breve le inversioni scompaiono del tutto. Su questo dataset, circa un terzo di tutti gli errori della finestra fissa è un errore di verso.

4.2 Tre clock a confronto

Le tre criticità hanno origini diverse ma una radice comune. La finestra fissa stabilisce la durata e il numero dei passi senza guardare il campione. Le due alternative esaminate in questa sezione allentano questo vincolo in modi opposti. La prima rinuncia al tempo fisico e taglia la registrazione secondo il numero di eventi. La seconda conserva il tempo fisico ma lascia che il numero dei passi segua la durata della registrazione. La figura 4.5 mostra lo stesso gesto suddiviso secondo i tre schemi, la tabella 4.2 ne confronta le proprietà.

4.2.1 Frame a eventi costanti

La rappresentazione a eventi costanti (*fixed event-count frames*) divide l'intera registrazione in 40 gruppi consecutivi che contengono lo stesso numero di eventi, a

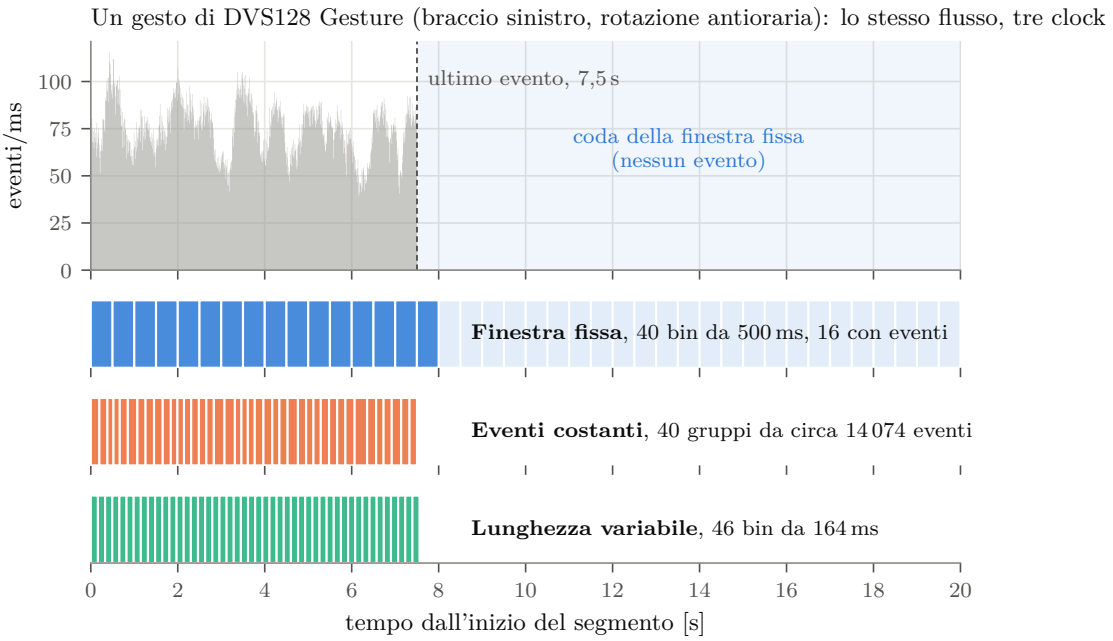


Figura 4.5: La registrazione di DVS128 Gesture della figura 4.3, circa 7,5 s di rotazione, suddivisa secondo i tre clock. In alto, la frequenza degli eventi su intervalli di 10 ms. Sotto, i passi delle tre rappresentazioni, con i passi privi di eventi in tinta chiara. La finestra fissa usa 40 bin da 500 ms, dei quali soltanto 16 contengono eventi. I gruppi a eventi costanti e i bin a lunghezza variabile coprono esattamente la registrazione.

meno di uno, secondo l'equazione (1.8), e in ciascun gruppo conta gli eventi ON e OFF di ogni pixel come la finestra fissa. Suddivisioni di questo tipo sono diffuse tanto nella visione a eventi, dove le fette a numero costante di eventi si affiancano a quelle a durata costante [44, 2], quanto nelle librerie per le SNN [8].

Il vantaggio è evidente. Ogni passo riceve la stessa quantità di attività, la coda vuota scompare e una fase molto intensa viene distribuita su più passi. Il prezzo è altrettanto chiaro. La durata fisica di un passo dipende dal campione e, all'interno del campione, dall'intensità dell'attività, sicché un ritardo di 8 passi nel mixer temporale può corrispondere, a seconda del dataset e del tratto di registrazione, a poche decine di millisecondi oppure a interi secondi. Inoltre, per dividere la registrazione in 40 parti uguali occorre conoscere in anticipo il numero totale degli eventi, e la rappresentazione è perciò applicabile soltanto a registrazioni già concluse. Negli esperimenti di questo capitolo il modello e i suoi ritardi restano identici a quelli usati con la finestra fissa.

4.2.2 Durata fissa e lunghezza variabile

La rappresentazione a lunghezza variabile (*fixed-duration bins with variable sequence length*) conserva invece il tempo fisico. Tutti i bin di un dataset hanno la stessa durata Δ, ma la sequenza parte dal primo evento e si ferma al bin che contiene l'ultimo, secondo la regola dell'equazione (1.6). Per un campione i con timestamp interi in microsecondi, la lunghezza è

$$T_i = \left\lceil \frac{D_i}{\Delta} \right\rceil, \qquad D_i = t_{\text{last},i} - t_{\text{first},i} + 1, \tag{4.2}$$

dove il microsecondo aggiunto garantisce che anche un evento caduto sul bordo di un bin trovi posto nella sequenza. Il numero dei passi varia così da un campione all'altro, mentre il loro significato fisico resta lo stesso. Non c'è coda vuota, nessun evento viene scartato, e i silenzi interni alla registrazione restano visibili come bin validi privi di eventi.

Resta da scegliere Δ. Per conservare lo stesso budget medio di passi della finestra fissa, la durata dei bin viene calibrata sul solo insieme di addestramento, come il più piccolo intero per cui la lunghezza media non supera 40,

$$\Delta = \min\Big\{ d \in \mathbb{N}^+ \;\Big|\; \frac{1}{|\mathcal{D}_{\text{tr}}|} \sum_{i \in \mathcal{D}_{\text{tr}}} \left\lceil \frac{D_i}{d} \right\rceil \le 40 \Big\}, \tag{4.3}$$

dove $\mathcal{D}_{\text{tr}}$ è l'insieme di addestramento. La tabella 4.1 riporta i valori ottenuti. La stessa Δ viene applicata senza modifiche alla validazione, dove la lunghezza media resta vicina a 40 ma i singoli campioni si distribuiscono su intervalli ampi, da 16 a 74 passi su DVS128 Gesture e fino a 285 per la registrazione più lunga di THU-EACT-50-CHL.

Tabella 4.1: Calibrazione della rappresentazione a lunghezza variabile. La durata Δ è calibrata sul solo addestramento e applicata invariata alla validazione. I ritardi del mixer temporale sono riscalati per conservare gli orizzonti fisici della finestra fissa, indicati nell'ultima colonna.

Dataset	**Δt fissa**	**Δ variabile**	**Passi in validazione**	**Ritardi**	**Orizzonti [ms]**
DVS-Lip	50 ms	27,42 ms	da 25 a 67	2,4,7,15	55,110,192,411
DVS128 Gesture	500 ms	164,36 ms	da 16 a 74	3,6,12,24	493,986,1972,3945
CIFAR10-DVS	50 ms	32,88 ms	da 37 a 44	2,3,6,12	66,99,197,395
THU-EACT-50-CHL	200 ms	70,00 ms	da 20 a 285	3,6,11,23	210,420,770,1610

Sequenze di lunghezza diversa richiedono, all'interno di un batch, passi di padding. Coerentemente con la distinzione discussa a proposito dell'equazione (1.13), questi passi sono esclusi da ogni calcolo. Non contribuiscono alle statistiche della batch normalization, non aggiornano i potenziali di membrana né la memoria del mixer temporale, e non entrano nel readout, che diventa una media sui soli T_i passi validi del campione, nella forma dell'equazione (2.6) con T_i al posto di T. Un bin valido senza eventi, invece, resta un'osservazione e viene elaborato come tale.

Cambiando la durata dei bin cambia anche il significato dei ritardi. Per conservare gli orizzonti fisici del mixer temporale, i ritardi vengono riscalati e arrotondati alla nuova griglia, $d'_k = \mathrm{round}\big(d_k\,\Delta t/\Delta\big)$, dove Δt è la durata dei bin della finestra fissa. Su DVS-Lip, per esempio, i ritardi (1,2,4,8) da 50 ms diventano (2,4,7,15) da 27,4 ms, e il più lungo passa da 400 a 411 ms. La costante τ_d dei neuroni resta invece pari a 2 passi, una scelta su cui si tornerà nella sezione 4.2.4.

4.2.3 Risultati sui quattro dataset

La figura 4.6 riassume il confronto, e la tabella 4.3 ne riporta i valori. Il quadro non è uniforme, ed è proprio la sua disomogeneità a essere istruttiva.[1]

DVS128 Gesture è il caso più netto. Entrambe le alternative superano la finestra fissa di circa otto punti di Macro-F1, 94,8 con gli eventi costanti e 94,2 con la lunghezza variabile contro 86,5, e le repliche con un secondo seed confermano il distacco. Anche la media delle ultime 16 epoche, meno esposta alla scelta del massimo, sale da 81,4 a 93,9 e 93,1. Una parte di questo guadagno si lascia ricondurre direttamente alle rotazioni. La finestra fissa sbaglia 19 campioni in

[1]Le esecuzioni di riferimento non condividono tutte lo stesso frazionamento in microbatch, e su DVS-Lip e DVS128 Gesture il confronto tra finestra fissa e lunghezza variabile ne combina due diversi. Dove esistono esecuzioni che differiscono soltanto per il frazionamento, la Macro-F1 cambia di 0,09 punti su DVS-Lip e di 0,14 su CIFAR10-DVS, molto meno degli effetti discussi nel testo. Su THU-EACT-50-CHL tutte le esecuzioni usano lo stesso frazionamento.

Tabella 4.2: Proprietà dei tre clock dell'input.

	Finestra fissa	**Eventi costanti**	**Lunghezza variabile**
Durata di un passo	fissa per dataset	variabile, segue l'attività	fissa per dataset, calibrata sull'addestramento
Numero di passi	40	40	variabile, 40 in media
Parte di registrazione elaborata	quella entro la finestra	intera	intera
Passi dopo l'ultimo evento	sì, la coda vuota	no	no
Significato fisico di un ritardo	costante	variabile	costante
Informazione necessaria in anticipo	nessuna	numero totale di eventi	nessuna

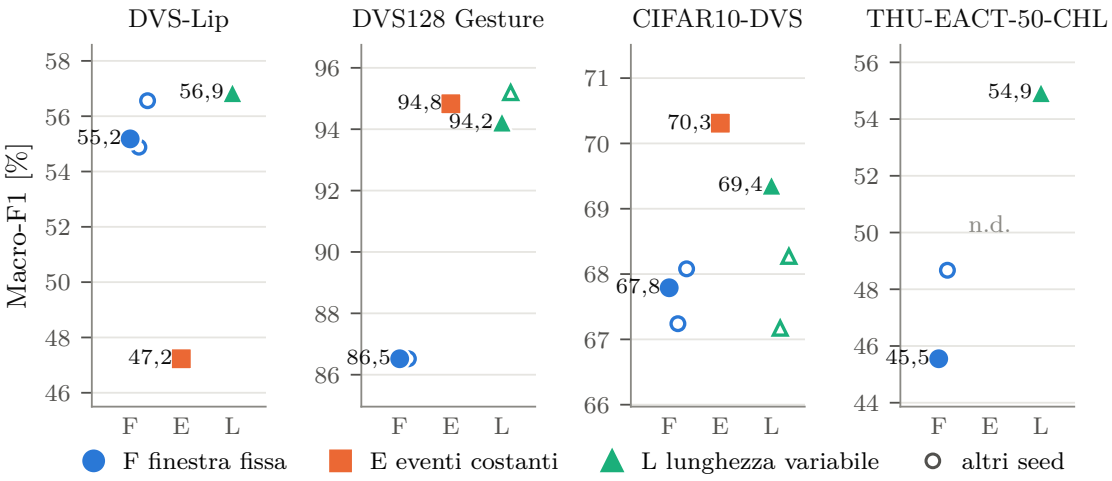


Figura 4.6: Macro-F1 del modello di riferimento con i tre clock, sulla validazione di sviluppo. Il simbolo pieno indica l'esecuzione con seed 42, i simboli vuoti le repliche con altri seed disponibili. Ogni pannello ha una propria scala verticale. Su THU-EACT-50-CHL la rappresentazione a eventi costanti non ha superato le verifiche preliminari.

Tabella 4.3: Accuratezza e Macro-F1 [%] del modello di riferimento con i tre clock, sulla validazione di sviluppo, seed 42. Le repliche disponibili con altri seed danno, in Macro-F1, 54,88 e 56,56 per la finestra fissa su DVS-Lip, 86,52 per la finestra fissa e 95,19 per la lunghezza variabile su DVS128 Gesture, 67,24 e 68,08 per la finestra fissa e 67,17 e 68,27 per la lunghezza variabile su CIFAR10-DVS, 48,67 per la finestra fissa su THU-EACT-50-CHL.

Dataset	**Finestra fissa**	**Eventi costanti**	**Lunghezza variabile**
DVS-Lip	55,53 / 55,18	47,21 / 47,23	56,76 / 56,85
DVS128 Gesture	87,12 / 86,52	95,08 / 94,83	94,32 / 94,23
CIFAR10-DVS	68,00 / 67,79	70,11 / 70,31	69,33 / 69,36
THU-EACT-50-CHL	50,38 / 45,55	non disponibile	61,15 / 54,93

più della lunghezza variabile, e 13 di questi appartengono alle quattro classi di rotazione.

Su THU-EACT-50-CHL la lunghezza variabile porta la Macro-F1 da 45,6 a 54,9 e l'accuratezza da 50,4 a 61,2. Una seconda esecuzione della finestra fissa con un altro seed arriva a 48,7, riducendo il distacco senza annullarlo. La rappresentazione a eventi costanti, invece, non ha superato su questo dataset il test di sovradattamento controllato descritto nella sezione 2.4.3, e non è stata addestrata in forma completa. È un esito da registrare più che da interpretare. Una possibile ragione, non verificata, è la forte variabilità del numero di eventi, che su THU-EACT-50-CHL spazia su oltre due ordini di grandezza e rende i gruppi di campioni diversi molto disomogenei tra loro.

DVS-Lip separa invece le due alternative. Gli eventi costanti peggiorano nettamente il risultato, da 55,2 a 47,2, mentre la lunghezza variabile arriva a 56,9. Quest'ultimo valore supera di poco tutti e tre i seed della finestra fissa, compresi tra 54,9 e 56,6, e il margine va letto come un piccolo vantaggio, non come un salto. Una lettura plausibile del risultato degli eventi costanti chiama in causa la natura del segnale. Nella lettura labiale contano il ritmo e la durata relativa dei movimenti, e una suddivisione che dilata i tratti quieti e comprime quelli concitati altera proprio queste proporzioni.

CIFAR10-DVS, infine, si comporta come ci si poteva attendere da un dataset con dinamica imposta e durate quasi identiche. Con il seed di riferimento le due alternative guadagnano tra un punto e mezzo e due punti e mezzo, 70,3 con gli eventi costanti e 69,4 con la lunghezza variabile contro 67,8. Con tre seed per ciascuna configurazione, però, le medie della finestra fissa e della lunghezza variabile, 67,7 e 68,3, distano poco più di mezzo punto, meno della variabilità tra seed. Su questo dataset il clock conta poco.

Dove nasce il guadagno? La figura 4.7 divide la validazione in quartili di durata

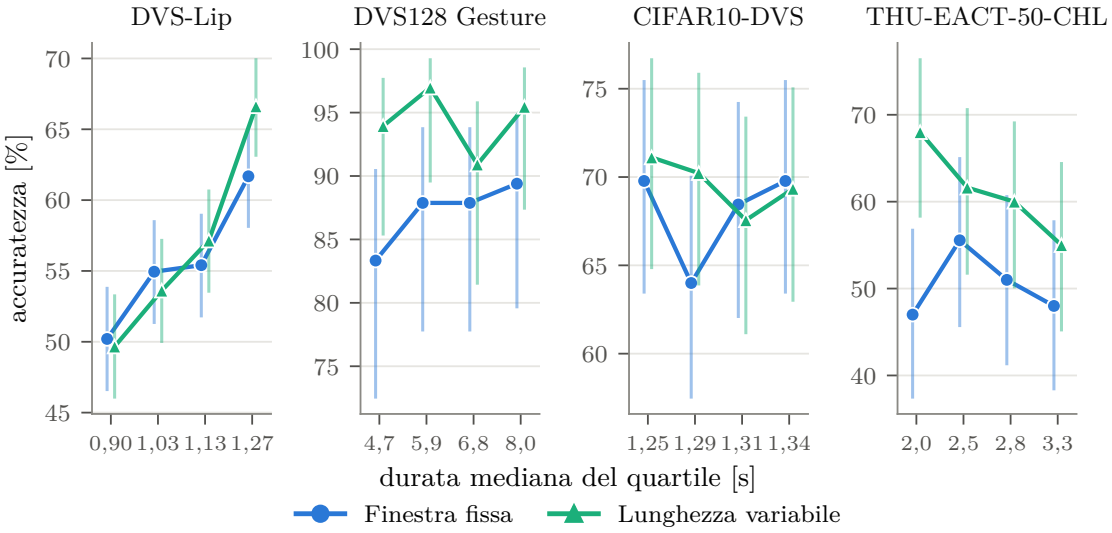


Figura 4.7: Accuratezza del modello di riferimento con la finestra fissa e con la lunghezza variabile, per quartili di durata della registrazione. Le barre verticali indicano l'intervallo di confidenza al 95% di Wilson, ampio dove i quartili contano pochi campioni, come i 66 di DVS128 Gesture. Seed 42, validazione di sviluppo, valutazione in singola precisione.

e confronta, quartile per quartile, i due modelli che lavorano in tempo fisico. Su DVS128 Gesture e THU-EACT-50-CHL la lunghezza variabile è migliore in tutti i quartili, e il vantaggio più ampio si concentra sui campioni più brevi, quelli che nella finestra fissa lasciano la coda più lunga. Sul quartile più breve di THU-EACT-50-CHL l'accuratezza passa dal 47% al 68%. Su DVS-Lip il quadro è quasi rovesciato. I campioni più brevi vengono osservati con più passi utili, in mediana 33 contro i 18 della finestra fissa, ma perdono i passi di assestamento della coda, e il loro risultato non cambia. Il guadagno si concentra nel quartile più lungo, dove l'accuratezza sale dal 61,7% al 66,6%. Su CIFAR10-DVS, con durate quasi identiche, le differenze tra quartili rientrano nell'incertezza.

Il confronto appaiato, campione per campione, aggiunge un'ultima sfumatura. Su DVS-Lip la lunghezza variabile corregge 467 errori della finestra fissa, ma ne introduce 431 nuovi. I due modelli hanno un'accuratezza simile e sbagliano in buona parte campioni diversi, un indizio che il clock non sposta soltanto il livello delle prestazioni ma anche il tipo di informazione che la rete sfrutta. Su DVS128 Gesture e THU-EACT-50-CHL lo sbilanciamento è invece netto, con 25 correzioni contro 6 nuovi errori e 72 contro 29.

4.2.4 Assestamento e costante di tempo

Il passaggio dalla finestra fissa alla lunghezza variabile cambia più cose insieme. Elimina la coda, ma riduce anche la durata dei bin, e poiché τ_d resta di 2 passi accorcia la memoria fisica dei neuroni. Su DVS128 Gesture il bin passa da 500 a 164 ms, e il tempo in cui il potenziale si dimezza si riduce nella stessa proporzione (figura 4.8a). Per capire quale di questi cambiamenti conti, due controlli riaddestrati modificano la rappresentazione a lunghezza variabile un elemento alla volta.

Il primo aggiunge dopo l'ultimo evento 500 ms di passi validi senza eventi, restituendo alla rete un breve intervallo di assestamento. La Macro-F1 resta praticamente invariata, 94,6 contro 94,2. Il secondo conserva l'assestamento e porta τ_d a circa 4,91, il valore per cui in 500 ms il potenziale decade esattamente come con la finestra fissa. La memoria fisica dei neuroni torna così quella originale, e la Macro-F1 scende a 84,0, al di sotto della stessa finestra fissa. Le due esecuzioni di controllo condividono tra loro il frazionamento in microbatch, diverso da quello dell'esecuzione canonica, e differiscono soltanto per τ_d.

Il risultato sposta l'attenzione dalla coda alla scala temporale. Su DVS128 Gesture, restituire ai neuroni la memoria fisica della finestra fissa basta ad annullare il vantaggio della lunghezza variabile, anche con l'intera registrazione a disposizione e passi più fini. La scala della memoria neuronale pesa dunque almeno quanto l'eliminazione della coda. Il dato riguarda un solo dataset e un solo seed e non autorizza una regola generale, ma mostra con chiarezza che il clock dell'input e la dinamica dei neuroni non possono essere progettati separatamente.

4.3 Conseguenze per i confronti successivi

Nessuna delle due alternative prevale ovunque, ma la scelta per i capitoli successivi è netta e cade sulla rappresentazione a lunghezza variabile. Il motivo principale è il clock fisico. Ogni passo ha la stessa durata in tutto il dataset, e ritardi e costanti di tempo conservano un significato preciso, una condizione indispensabile per studiare la memoria della rete. A questo si aggiungono la copertura dell'intera registrazione senza code artificiali e la possibilità di costruire i bin man mano che gli eventi arrivano, senza conoscerne in anticipo il numero totale. Anche sul piano dei risultati la scelta regge il confronto. La lunghezza variabile resta vicina agli eventi costanti su DVS128 Gesture e CIFAR10-DVS, li supera di circa 9,6 punti di Macro-F1 su DVS-Lip e funziona su THU-EACT-50-CHL, dove gli eventi costanti non hanno superato le verifiche preliminari.

Da qui in avanti, il termine di confronto di ogni modifica della memoria è quindi il modello di riferimento addestrato con la rappresentazione a lunghezza variabile, con una Macro-F1 di 56,85 su DVS-Lip, 94,23 su DVS128 Gesture, 69,36

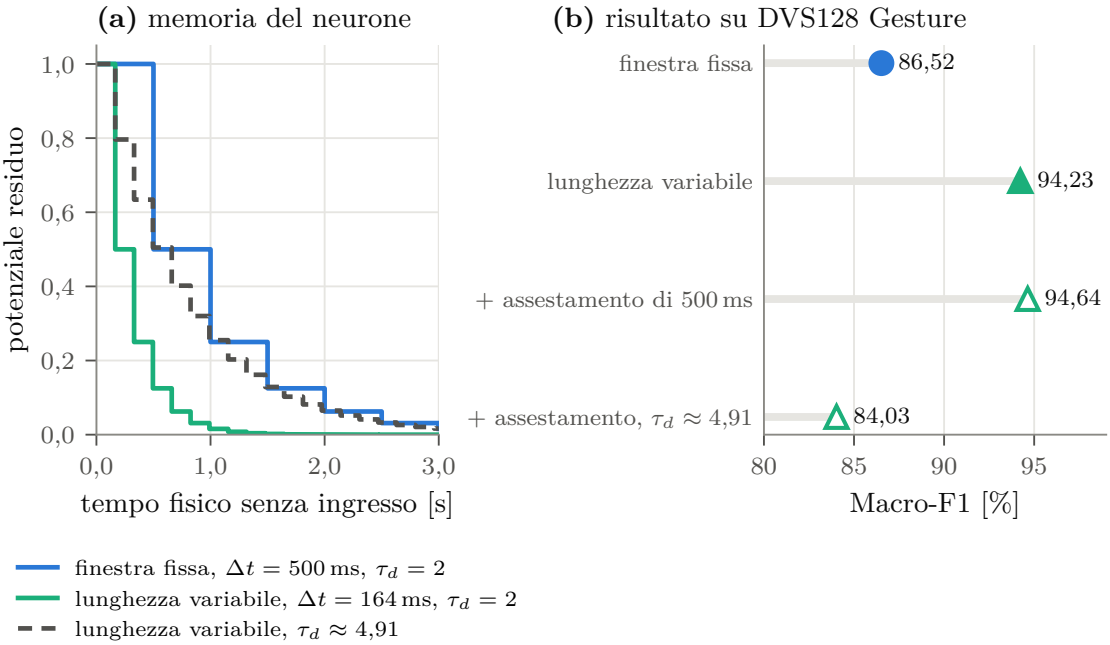


Figura 4.8: Assestamento e costante di tempo su DVS128 Gesture. (a) Frazione del potenziale che sopravvive in assenza di ingresso, in funzione del tempo fisico, per un neurone isolato al di sotto della soglia. Con $\tau_d = 2$ il potenziale si dimezza a ogni passo e, con i bin da 164 ms, svanisce circa tre volte più in fretta che con quelli da 500 ms. Con $\tau_d \approx 4{,}91$ la curva torna a coincidere con quella della finestra fissa ogni 500 ms. (b) Macro-F1 della finestra fissa, della lunghezza variabile e dei due controlli riaddestrati, seed 42.

su CIFAR10-DVS e 54,93 su THU-EACT-50-CHL, tutte con seed 42.[2] I ritardi del mixer temporale sono quelli riscalati della tabella 4.1, mentre τ_d resta pari a 2 passi.

Il cambio di clock non rende la rete più economica, ma nemmeno più costosa nel modo che ci si potrebbe aspettare. Poiché Δ è calibrata per mantenere in media 40 passi, il numero medio dei passi, e con esso quello delle MAC dense, resta praticamente invariato. Crescono invece le operazioni sinaptiche, del 30% su DVS128 Gesture e del 28% su THU-EACT-50-CHL, perché i passi che la finestra fissa dedicava alla coda sono ora occupati da eventi. Lo stesso budget di passi viene speso osservando anziché aspettando. Per una futura implementazione hardware compaiono però due costi nuovi. La lunghezza della sequenza non è più limitata a 40 passi e cresce con la durata della registrazione, fino ai 285 passi del campione più lungo di THU-EACT-50-CHL, e con essa crescono latenza ed energia del singolo campione. Inoltre il buffer del mixer temporale, che deve conservare le correnti fino al ritardo massimo, si allunga con i ritardi espressi in passi, da 8 a 15 su DVS-Lip e a 24 su DVS128 Gesture, mentre lo stato dei neuroni resta invariato.

Infine, la rappresentazione a lunghezza variabile non rende la rete invariante al ritmo. Ritardi e costanti di tempo restano espressi in passi, e un gesto eseguito al doppio della velocità occupa la metà dei passi. La robustezza a questo tipo di variazione viene esaminata nell'analisi trasversale dei risultati.

Il capitolo lascia così in eredità un clock in cui un passo ha lo stesso significato fisico dall'inizio alla fine della sequenza e da un campione all'altro. È la condizione che rende sensato chiedere alla memoria della rete di confrontare il presente con il passato a distanze fisiche prestabilite, ed è da qui che parte il capitolo successivo.

[2]Su ciascun dataset il riferimento condivide il frazionamento in microbatch con le varianti del capitolo successivo. Su DVS-Lip e CIFAR10-DVS si tratta di un unico microbatch da 32 campioni.

Capitolo 5

Memoria temporale relazionale

Con il clock fissato nel capitolo precedente, ogni passo ha una durata fisica nota, e i ritardi del mixer temporale corrispondono a distanze precise nel passato. Questo capitolo si chiede come usare meglio quella memoria. La domanda non riguarda quanto passato conservare, ma che cosa farne quando lo si mette accanto al presente.

Il punto di partenza è il modo in cui il modello di riferimento tratta il passato. Il mixer a ritardi lo richiama e lo somma al presente, ma non li confronta mai direttamente. Il capitolo propone un modulo che separa il problema in tre parti, che cosa del passato conservare, quanto di quel passato esista davvero e come confrontarlo con il presente, e le affronta nell'ordine. Per tutto il capitolo l'ingresso è la rappresentazione a lunghezza variabile, e il termine di confronto è il modello di riferimento addestrato con essa, con una Macro-F1 di 56,85 su DVS-Lip, 94,23 su DVS128 Gesture, 69,36 su CIFAR10-DVS e 54,93 su THU-EACT-50-CHL (sezione 4.3).

5.1 Dal richiamo del passato al confronto tra passato e presente

Nel modello di riferimento il passato entra nella corrente di uno strato attraverso il mixer causale a più ritardi dell'equazione (4.1). A ogni passo il mixer rilegge le correnti di alcuni passi precedenti, le trasforma con una matrice per ciascun ritardo e le somma alla corrente presente. Si tratta, a tutti gli effetti, di un'operazione di richiamo. Il passato viene riportato nel presente, e il neurone che segue integra il risultato insieme a tutto il resto.

La forma della somma ha una conseguenza precisa. Ogni termine dipende da un solo istante, z_t dal presente e $W_k z_{t-d_k}$ dal passato a distanza d_k, e il contributo di ciascuno resta lo stesso qualunque cosa contengano gli altri. Il mixer è additivamente separabile nel tempo. Una relazione tra passato e presente, come il fatto che una certa configurazione delle feature ne segua un'altra, non viene mai calcolata in modo esplicito. Può emergere soltanto più avanti, dalla soglia del neurone, che reagisce quando più contributi si sommano oltre il valore critico, oppure dagli strati successivi. Il mixer mette a disposizione il passato e lascia ad altri il compito di confrontarlo con il presente.

Non è detto che questo sia un limite. Combinazioni lineari seguite da una soglia possono rappresentare, in linea di principio, molte relazioni, e una rete abbastanza grande è in grado di costruirle. In un regime compatto, però, ogni relazione che la rete deve ricavare implicitamente consuma capacità che potrebbe essere spesa altrove. L'idea di questo capitolo è rendere esplicito il confronto, con un termine che dipenda congiuntamente dal passato e dal presente.

L'idea ha un precedente illustre nello studio della percezione del movimento. Il rilevatore di correlazione proposto da Hassenstein e Reichardt per spiegare la risposta al moto di un insetto confronta i segnali di due fotorecettori vicini, moltiplicando il segnale ritardato dell'uno per quello istantaneo dell'altro [45]. Il prodotto, da solo, risponde al moto in entrambe le direzioni. Sottraendo il prodotto speculare, in cui i ruoli dei due recettori sono scambiati, si ottiene una risposta il cui segno indica il verso del movimento,

$$R(t) = s_1(t-\delta)\, s_2(t) - s_2(t-\delta)\, s_1(t), \tag{5.1}$$

dove s_1 e s_2 sono i segnali dei due recettori e δ il ritardo. Nelle sue varianti elaborate lo schema è equivalente ai modelli di energia del movimento, ed è ancora oggi un riferimento per i circuiti biologici che codificano la direzione [46, 47]. Due ingredienti lo distinguono da un semplice richiamo. Il prodotto rende la risposta dipendente dalla coincidenza di passato e presente, la sottrazione del termine speculare la rende sensibile all'ordine. Il capitolo precedente ha mostrato quanto l'ordine possa contare. Sulle rotazioni di DVS128 Gesture, perdere il verso del movimento bastava a confondere classi che per il resto erano identiche (sezione 4.1.3).

Il modulo proposto in questo capitolo trasferisce lo stesso schema dai pixel alle feature latenti (figura 5.1). Il ruolo dei due recettori è affidato a due proiezioni apprese delle feature, quello del segnale ritardato a un riassunto del passato, e il confronto avviene in ogni posizione spaziale tra il presente e più riassunti a distanze diverse. L'analogia motiva il calcolo ma non ne garantisce l'interpretazione. Nulla obbliga i canali della rete a diventare rilevatori di movimento, e le proiezioni possono combinare feature dal significato molto diverso.

Negli Spiking Transformer l'interazione esplicita tra istanti diversi è già stata affrontata, con un'attenzione che correla feature appartenenti a blocchi di passi [19]



Figura 5.1: Dal richiamo al confronto. (a) Il mixer a ritardi del modello di riferimento proietta le correnti di alcuni passi precedenti e le somma alla corrente presente, sicché passato e presente contribuiscono in modo indipendente. (b) Il rilevatore di correlazione di Hassenstein e Reichardt moltiplica il segnale ritardato di un recettore per quello istantaneo dell'altro e sottrae il prodotto speculare, ottenendo una risposta il cui segno dipende dal verso del moto. (c) Il mixer proposto riassume il passato in bande, lo richiama linearmente e lo confronta con il presente attraverso due proiezioni apprese, con un ruolo analogo a quello dei due recettori.

o con la combinazione di informazioni provenienti da più ritardi [20], come discusso nella sezione 1.5. La strada seguita qui è più locale e più economica. Il confronto non passa per una matrice di attenzione, ma per un termine bilineare a basso rango calcolato in ogni posizione, e si colloca nello stesso punto della rete in cui oggi opera il mixer a ritardi.

Da queste osservazioni, e dai capitoli che precedono, discendono alcuni requisiti. Il modulo deve essere causale e leggere soltanto il passato già osservato. Deve conservare la scala fisica introdotta dalla rappresentazione a lunghezza variabile, e quindi riassumere il passato a distanze fisiche prestabilite. Deve sapere quanta storia esista davvero, perché all'inizio di ogni sequenza il passato è incompleto. Deve mantenere il percorso lineare del mixer a ritardi, in modo che il confronto si aggiunga al richiamo anziché sostituirlo. Deve infine costare poco e partire, all'inizializzazione, dalla stessa funzione del modello di riferimento, così che ogni differenza sia frutto dell'addestramento.

I requisiti si traducono nella forma generale

$$u_t = z_t + \sum_{k=1}^{K} b_{k,t}\Big[W_k\, m_{k,t} + U_k\, \rho\Big(m_{k,t}, z_t\Big)\Big], \tag{5.2}$$

dove $z_t \in \mathbb{R}^C$ è la corrente dello strato al passo t in una posizione spaziale, prima del neurone. Il vettore $m_{k,t}$ riassume il passato alla k-esima scala, il coefficiente $b_{k,t}$ tiene conto della storia disponibile, $W_k m_{k,t}$ è il richiamo lineare, erede diretto del mixer a ritardi, e $\rho(m_{k,t}, z_t)$ è una relazione tra passato e presente, riportata nello spazio delle feature dalla matrice U_k. Il modulo che ne risulta viene indicato nel seguito come *interval-relational mixer* (IRM). Il nome descrive una combinazione di ingredienti noti e non rivendica un nuovo tipo di operatore.

La forma separa tre domande che conviene tenere distinte. Che cosa del passato viene conservato, nel riassunto $m_{k,t}$. Quanto di quel passato esiste davvero, nel coefficiente $b_{k,t}$. Come il passato viene confrontato con il presente, nella relazione ρ. Ponendo $\rho = 0$ si ottiene una memoria puramente lineare, che è a sua volta un controllo informativo, mentre cambiando la relazione a parità di tutto il resto si possono confrontare forme di interazione con proprietà diverse. La sezione 5.2 affronta le prime due domande, le sezioni successive la terza.

Per attribuire l'effetto con chiarezza, l'IRM sostituisce il mixer a ritardi in un solo punto della rete, nel secondo patch embedding, sulla corrente della convoluzione con passo 2, dopo la batch normalization e prima del neurone. È una scelta di progetto. A quel livello la rappresentazione ha 128 canali a un sedicesimo della risoluzione d'ingresso, le feature sono più astratte di quelle del primo stadio e le posizioni sono poche, 64 per un ingresso di 128×128 pixel, sicché il costo del confronto resta contenuto. Il primo patch embedding conserva il mixer a ritardi, mentre SSA, MLP e readout restano invariati.

5.2 Memoria causale a intervalli e disponibilità della storia

5.2.1 Dai ritardi puntuali alle bande

Il mixer a ritardi legge il passato in pochi punti. Con la rappresentazione a lunghezza variabile, su DVS-Lip i ritardi (2,4,7,15) campionano 4 dei 15 passi che separano il presente dal ritardo massimo, e su DVS128 Gesture 4 su 24 (figura 5.2). Ciò che accade nei passi intermedi raggiunge il mixer soltanto attraverso le altre vie della rete, in particolare la memoria dei neuroni. Una lettura così rada ha un difetto evidente. Un movimento che avviene un passo prima o un passo dopo cade fuori dal ritardo e sparisce dalla lettura, anche se la sua distanza fisica dal presente è quasi la stessa.

La memoria a intervalli sostituisce i punti con bande contigue di ritardi. Conservando come bordi i ritardi del modello di riferimento, $d_0 = 0 < d_1 < \cdots < d_K$, la k-esima banda raccoglie i ritardi

$$I_k = \{\ell \in \mathbb{N} \mid d_{k-1} < \ell \leq d_k\}, \qquad k = 1, \ldots, K. \tag{5.3}$$

Le bande coprono senza sovrapposizioni tutti i ritardi da 1 a d_K, il presente ne resta escluso e continua ad arrivare dal percorso diretto, e l'orizzonte complessivo coincide con quello del mixer a ritardi. Su DVS-Lip le bande sono [1,2], [3,4], [5,7] e [8,15], da circa 27 a 411 ms nel passato. Su THU-EACT-50-CHL diventano [1,3], [4,6], [7,11] e [12,23], e arrivano a 1,6 s.

Le bande si allargano allontanandosi dal presente. Il passato recente è descritto con la risoluzione di uno o due passi, quello più remoto con medie su otto o dodici passi. È una forma rudimentale di compressione del tempo, in cui la precisione con cui un evento viene collocato diminuisce con la sua distanza, vicina nello spirito alle rappresentazioni a scala logaritmica proposte per la memoria temporale [48]. Un'alternativa sarebbe lasciare che la rete apprenda la posizione dei ritardi, come avviene in alcune SNN dotate di ritardi sinaptici apprendibili [49]. Qui si è preferito conservare i bordi del modello di riferimento, così che memoria a punti e memoria a bande coprano esattamente lo stesso orizzonte.

5.2.2 Medie sulla storia disponibile

All'interno di ciascuna banda il passato viene riassunto dalla media delle correnti. All'inizio della sequenza, però, una banda può essere vuota o soltanto in parte riempita, perché i passi corrispondenti precederebbero il primo evento. Indicando con $J_{k,t} = \{\ell \in I_k \mid \ell \leq t\}$ i ritardi della banda effettivamente osservati al passo t,

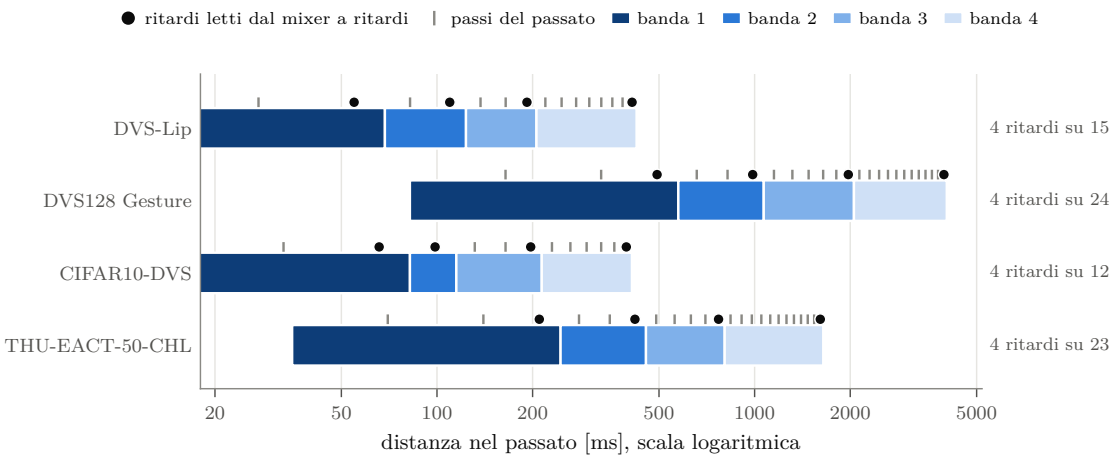


Figura 5.2: Ritardi del mixer a ritardi e bande della memoria a intervalli, in tempo fisico, con la rappresentazione a lunghezza variabile. I cerchi indicano i quattro ritardi letti dal mixer, i trattini grigi tutti i passi del passato fino al ritardo massimo. Le bande, delimitate dagli stessi ritardi, coprono senza lacune l'intero orizzonte e si allargano allontanandosi dal presente. A destra, quanti dei passi del passato vengono letti dal mixer a ritardi.

contato a partire da zero, e con $n_{k,t} = |J_{k,t}|$ il loro numero, il riassunto è

$$m_{k,t} = \begin{cases} \dfrac{1}{n_{k,t}} \displaystyle\sum_{\ell \in J_{k,t}} z_{t-\ell}, & n_{k,t} > 0, \\ 0, & n_{k,t} = 0. \end{cases} \tag{5.4}$$

La media divide per la storia realmente disponibile e non per la larghezza nominale della banda. Un riassunto parziale non viene così diluito da zeri che non corrispondono ad alcuna osservazione.

La media produce due effetti. Attenua il rumore e rende la memoria tollerante a piccoli spostamenti temporali, perché un movimento anticipato o ritardato di un passo resta quasi sempre nella stessa banda. Il prezzo è la perdita dell'ordine all'interno della banda, che la media rende invisibile. Le bande strette vicino al presente e il percorso diretto limitano il danno dove l'ordine fine conta di più, ma non lo annullano.

5.2.3 Quanta storia è davvero disponibile

Nel mixer a ritardi il passato mancante è rappresentato da zeri nel buffer. Poiché le proiezioni non hanno bias, uno zero produce una corrente nulla, e la scelta è innocua dal punto di vista numerico. Lascia però indistinguibili due situazioni molto diverse, una feature che vale davvero zero e un passato che non esiste ancora.

Il fenomeno non è marginale (figura 5.3). Con le lunghezze della rappresentazione a lunghezza variabile, la banda più lontana dispone di una storia completa nel 62% dei passi su DVS-Lip e nel 70% su CIFAR10-DVS, ma soltanto nel 43% su THU-EACT-50-CHL e nel 39% su DVS128 Gesture, dove in un terzo dei passi la trova del tutto vuota. Quando il ritardo massimo supera la metà della lunghezza tipica della sequenza, come accade su questi due dataset, la memoria più lontana lavora più spesso a regime ridotto che a regime pieno.

Per rendere esplicita questa informazione, ogni banda riceve la propria frazione di storia disponibile, $q_{k,t} = n_{k,t}/|I_k|$, che modula il contributo della banda attraverso il coefficiente

$$b_{k,t} = \mathbf{1}\big[n_{k,t} > 0\big]\; 2\,\sigma\big(\alpha_k\, q_{k,t} + \beta_k\big), \tag{5.5}$$

dove σ è la sigmoide e α_k, β_k sono due parametri apprendibili per banda. All'inizializzazione $\alpha_k = \beta_k = 0$, e il coefficiente vale 1 per ogni banda che contenga almeno un passo. Durante l'addestramento la rete può imparare ad attenuare o ad amplificare un riassunto in funzione di quanto è completo, al costo di $2K = 8$ parametri.

Il coefficiente dipende soltanto dal tempo trascorso dall'inizio della sequenza, non dalle feature, e non va confuso con il gate di contenuto della variante condizionale del mixer a ritardi discussa nel capitolo 3. Rende tuttavia accessibile alla rete l'età

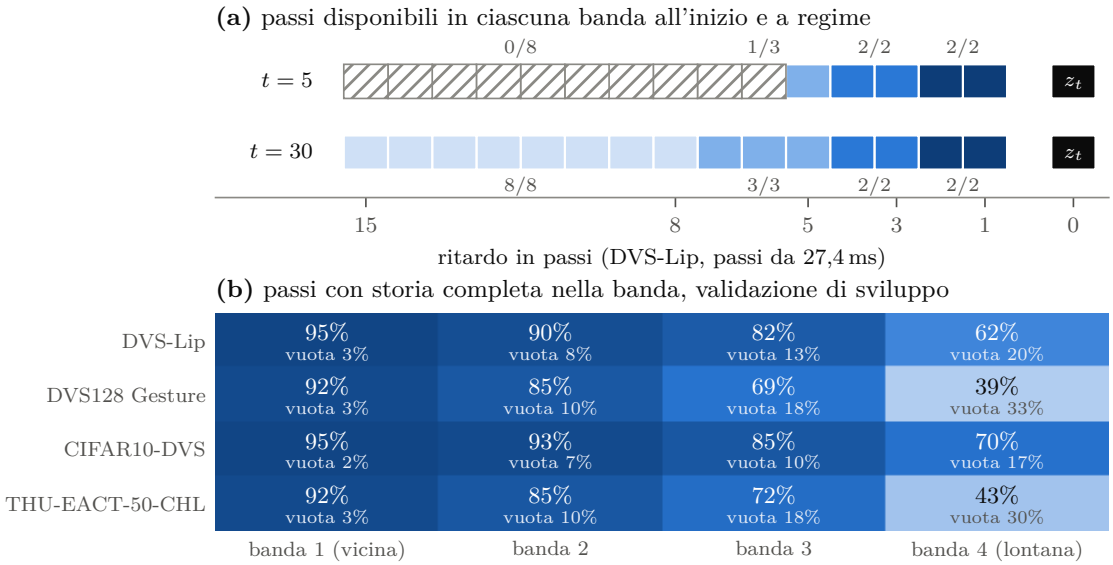


Figura 5.3: Storia disponibile nella memoria a intervalli. (a) Le bande di DVS-Lip al passo $t = 5$, vicino all'inizio della sequenza, e al passo $t = 30$, a regime. I passi tratteggiati precedono il primo evento e non esistono. Sopra e sotto ciascuna banda, il numero di passi osservati rispetto alla sua larghezza. (b) Frazione dei passi della validazione di sviluppo in cui ciascuna banda dispone di una storia completa e, in piccolo, frazione in cui è del tutto vuota.

della registrazione, cioè quanti passi siano trascorsi dal primo evento, ma non la sua durata complessiva, che resta ignota fino all'ultimo passo.

Con questo meccanismo la memoria tiene separate tre condizioni che sul piano numerico potrebbero confondersi, le stesse discusse a proposito dell'equazione (1.13). Il passato che precede il primo evento non esiste e non entra nel conteggio. Un bin valido privo di eventi è un'osservazione di silenzio, partecipa alla media e contribuisce a $n_{k,t}$. Il padding introdotto dal batching, infine, non entra in alcun calcolo e non fa avanzare il conteggio.

5.2.4 Implementazione e costo

Il calcolo delle medie non richiede di sommare ogni volta tutti i passi di una banda. La memoria conserva le correnti degli ultimi d_K passi, esattamente come il buffer del mixer a ritardi, e un contatore della storia disponibile per ciascun campione. Durante l'addestramento le somme di ogni banda si ottengono come differenze di somme cumulative lungo il tempo, sicché il costo per passo non dipende dalla larghezza delle bande. Rispetto al mixer a ritardi restano le stesse K proiezioni $C \times C$ per posizione e passo, applicate ai riassunti anziché ai singoli punti. A queste si aggiungono, per ogni canale, una manciata di operazioni elementari per banda, tra sottrazioni, divisioni e coefficienti, contro le KC moltiplicazioni e accumuli che ogni canale richiede alle proiezioni. Con $C = 128$ il sovraccarico aritmetico resta di oltre un ordine di grandezza inferiore al costo delle proiezioni.

Lo stato da conservare tra un passo e il successivo coincide con il buffer del mixer a ritardi nello stesso punto, $d_K \times C$ valori per posizione, più un contatore per campione. Su DVS-Lip, con $d_K = 15$ e 64 posizioni, sono 122 880 valori per campione. Le formule sono verificate nel codice rispetto a una loro definizione letterale, insieme alla causalità del modulo e all'equivalenza tra l'elaborazione dell'intera sequenza, quella a blocchi e quella passo per passo. Come per il resto della rete, la causalità riguarda il modulo, perché durante l'addestramento la batch normalization aggrega passi diversi (sezione 2.4.2).

All'inizializzazione le matrici W_k partono dagli stessi valori nulli del mixer a ritardi e i coefficienti di disponibilità valgono 1, sicché il modulo restituisce esattamente la corrente presente. Poiché anche il mixer a ritardi parte da matrici nulle, e i pesi condivisi vengono generati con lo stesso seed in un flusso pseudo-casuale separato da quello del nuovo modulo, la rete con la memoria a intervalli parte esattamente dalla stessa funzione del modello di riferimento.

La memoria a intervalli, presa da sola, resta un richiamo. Offre un riassunto del passato più completo e più robusto dei punti isolati, e sa quanto di quel passato esista davvero, ma lo restituisce al presente come una somma. Anticipando un risultato discusso più avanti, questa sola modifica non migliora il modello di

riferimento, né su DVS-Lip, dove la Macro-F1 passa da 56,85 a 54,67, né su THU-EACT-50-CHL, dove passa da 54,93 a 54,44, e la media delle ultime 16 epoche conferma lo stesso ordinamento. Riassumere meglio il passato non basta. Il passo successivo consiste nel confrontarlo con il presente.

Appendice A

```
    import os
    os.system("echo 1")
```

$\mathcal{O}(n \log n)$
`numpy`

Bibliografia

[1] Patrick Lichtsteiner, Christoph Posch e Tobi Delbruck. «A 128 × 128 120 dB 15 μs Latency Asynchronous Temporal Contrast Vision Sensor». In: *IEEE Journal of Solid-State Circuits* 43.2 (2008), pp. 566–576. DOI: 10.1109/JSSC.2007.914337 (cit. a p. 3).

[2] Guillermo Gallego et al. «Event-Based Vision: A Survey». In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.1 (2022), pp. 154–180. DOI: 10.1109/TPAMI.2020.3008413 (cit. alle pp. 3, 5, 58, 84).

[3] Alex Zihao Zhu, Liangzhe Yuan, Kenneth Chaney e Kostas Daniilidis. «Unsupervised Event-Based Learning of Optical Flow, Depth, and Egomotion». In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 989–997. URL: https://openaccess.thecvf.com/content_CVPR_2019/html/Zhu_Unsupervised_Event-Based_Learning_of_Optical_Flow_Depth_and_Egomotion_CVPR_2019_paper.html (cit. alle pp. 8, 9).

[4] Amos Sironi, Manuele Brambilla, Nicolas Bourdis, Xavier Lagorce e Ryad Benosman. «HATS: Histograms of Averaged Time Surfaces for Robust Event-Based Object Classification». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 1731–1740. URL: https://openaccess.thecvf.com/content_cvpr_2018/html/Sironi_HATS_Histograms_of_CVPR_2018_paper.html (cit. a p. 9).

[5] Emre O. Neftci, Hesham Mostafa e Friedemann Zenke. «Surrogate Gradient Learning in Spiking Neural Networks: Bringing the Power of Gradient-Based Optimization to Spiking Neural Networks». In: *IEEE Signal Processing Magazine* 36.6 (2019), pp. 51–63. DOI: 10.1109/MSP.2019.2931595 (cit. alle pp. 13, 17, 19).

[6] Bing Han, Gopalakrishnan Srinivasan e Kaushik Roy. «RMP-SNN: Residual Membrane Potential Neuron for Enabling Deeper High-Accuracy and Low-Latency Spiking Neural Network». In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 13558–13567 (cit. a p. 14).

[7] Wulfram Gerstner, Werner M. Kistler, Richard Naud e Liam Paninski. *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, 2014. DOI: 10.1017/CBO9781107447615. URL: https://neuronaldynamics.epfl.ch/online/Ch1.S3.html (cit. a p. 15).

[8] Wei Fang et al. «SpikingJelly: An Open-Source Machine Learning Infrastructure Platform for Spike-Based Intelligence». In: *Science Advances* 9.40 (2023), eadi1480 (cit. alle pp. 16, 84).

[9] Wei Fang, Zhaofei Yu, Yanqi Chen, Timothée Masquelier, Tiejun Huang e Yonghong Tian. «Incorporating Learnable Membrane Time Constant to Enhance Learning of Spiking Neural Networks». In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021, pp. 2661–2671 (cit. a p. 17).

[10] Yujie Wu, Lei Deng, Guoqi Li, Jun Zhu e Luping Shi. «Spatio-Temporal Backpropagation for Training High-Performance Spiking Neural Networks». In: *Frontiers in Neuroscience* 12 (2018), p. 331. DOI: 10.3389/fnins.2018.00331 (cit. a p. 19).

[11] Razvan Pascanu, Tomas Mikolov e Yoshua Bengio. «On the Difficulty of Training Recurrent Neural Networks». In: *Proceedings of the 30th International Conference on Machine Learning (ICML)*. 2013, pp. 1310–1318 (cit. alle pp. 19, 67).

[12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser e Illia Polosukhin. «Attention Is All You Need». In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017 (cit. a p. 21).

[13] Alexey Dosovitskiy et al. «An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale». In: *International Conference on Learning Representations (ICLR)*. 2021 (cit. a p. 21).

[14] Zhaokun Zhou, Yuesheng Zhu, Chenlin He, Yaowei Wang, Shuicheng Yan, Yonghong Tian e Li Yuan. «Spikformer: When Spiking Neural Network Meets Transformer». In: *International Conference on Learning Representations (ICLR)*. 2023 (cit. alle pp. 22, 31).

[15] Man Yao, Jiakui Hu, Zhaokun Zhou, Li Yuan, Yonghong Tian, Bo Xu e Guoqi Li. «Spike-driven Transformer». In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 36. 2023 (cit. alle pp. 23, 24, 26, 44).

[16] Chenlin Zhou, Liutao Yu, Zhaokun Zhou, Han Zhang, Jingyuan Wang, Huihui Zhou, Zhengyang Ma e Yonghong Tian. «Spikingformer: A Key Foundation Model for Spiking Neural Networks». In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 40. 3. 2026, pp. 2236–2244. DOI: 10.1609/aaai.v40i3.37207 (cit. a p. 26).

[17] Chenlin Zhou et al. «QKFormer: Hierarchical Spiking Transformer using Q-K Attention». In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 37. 2024. DOI: 10.52202/079017-0416 (cit. alle pp. 27, 29, 31, 32, 34, 35, 37–39, 45, 51).

[18] Chenlin Zhou et al. *QKFormer: repository ufficiale e nota sull'implementazione della SSA*. URL: https://github.com/zhouchenlin2096/QKFormer#notice-implementation-issue-in-ssa (visitato il giorno 02/11/2026) (cit. alle pp. 29, 66).

[19] Dongsung Lee, Yuhang Li, Youngeun Kim, Siyuan Xiao e Priyadarshini Panda. «Spiking Transformer with Spatial-Temporal Attention». In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025, pp. 13948–13958 (cit. alle pp. 39, 40, 42, 48, 93).

[20] K. Shi, H. Liu, Z. Song, Y. Liu, J. Zhang, S. Wang, J. Wu, M. Zhang e Y. Yang. «Temporal Interaction in Spiking Transformers with Multi-Delay Mixer». In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2026 (cit. alle pp. 39, 40, 42, 43, 95).

[21] Mohammad Karamimanesh, Ebrahim Abiri, MohammadReza Shahsavari, Kaveh Hassanli, André van Schaik e Jason Eshraghian. «Spiking Neural Networks on FPGA: A Survey of Methodologies and Recent Advancements». In: *Neural Networks* 186 (2025), p. 107256. DOI: 10.1016/j.neunet.2025.107256 (cit. alle pp. 46, 49).

[22] Z. Li, W. Mao, S. Zhang, Q. Dong e Z. Wang. «An Efficient Sparse Hardware Accelerator for Spike-Driven Transformer». In: *IEEE Asia-Pacific Conference on Applied Electromagnetics (APACE)*. 2024. DOI: 10.1109/APACE62360.2024.10877394 (cit. a p. 46).

[23] Chao Du, Qifan Wen, Zhiheng Wei e Huazhi Zhang. «Energy Efficient Spike Transformer Accelerator at the Edge». In: *Intelligent Marine Technology and Systems* 2 (2024), p. 24. DOI: 10.1007/s44295-024-00040-5 (cit. a p. 49).

[24] Xuerui Qiu et al. «Quantized Spike-Driven Transformer». In: *International Conference on Learning Representations (ICLR)*. 2025 (cit. a p. 50).

[25] Gengshen Tan, Yang Wang, Huiguang Han, Yiyong Cao, Feng Wu e Zheng-Jun Zha. «Multi-Grained Spatio-Temporal Features Perceived Network for Event-Based Lip-Reading». In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022, pp. 20094–20103 (cit. a p. 55).

[26] Arnon Amir, Brian Taba, David Berg et al. «A Low Power, Fully Event-Based Gesture Recognition System». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 7243–7252 (cit. a p. 55).

[27] Hongmin Li, Hanchao Liu, Xiangyang Ji, Guoqi Li e Luping Shi. «CIFAR10-DVS: An Event-Stream Dataset for Object Classification». In: *Frontiers in Neuroscience* 11 (2017), p. 309 (cit. a p. 56).

[28] Yue Gao, Jiaxuan Lu, Siqi Li, Nan Ma, Shaoyi Du, Yipeng Li e Qionghai Dai. «Action Recognition and Benchmark Using Event Cameras». In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45.12 (2023) (cit. a p. 56).

[29] Ana I. Maqueda, Antonio Loquercio, Guillermo Gallego, Narciso García e Davide Scaramuzza. «Event-Based Vision Meets Deep Learning on Steering Prediction for Self-Driving Cars». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 5419–5427 (cit. a p. 58).

[30] Kaiming He, Xiangyu Zhang, Shaoqing Ren e Jian Sun. «Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification». In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2015, pp. 1026–1034 (cit. a p. 62).

[31] Ilya Loshchilov e Frank Hutter. «Decoupled Weight Decay Regularization». In: *International Conference on Learning Representations (ICLR)*. 2019 (cit. a p. 67).

[32] Ilya Loshchilov e Frank Hutter. «SGDR: Stochastic Gradient Descent with Warm Restarts». In: *International Conference on Learning Representations (ICLR)*. 2017 (cit. a p. 67).

[33] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens e Zbigniew Wojna. «Rethinking the Inception Architecture for Computer Vision». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 2818–2826 (cit. a p. 67).

[34] Paulius Micikevicius et al. «Mixed Precision Training». In: *International Conference on Learning Representations (ICLR)*. 2018 (cit. a p. 67).

[35] Sergey Ioffe e Christian Szegedy. «Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift». In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. 2015, pp. 448–456 (cit. a p. 68).

[36] Hanle Zheng, Yujie Wu, Lei Deng, Yifan Hu e Guoqi Li. «Going Deeper With Directly-Trained Larger Spiking Neural Networks». In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35. 12. 2021, pp. 11062–11070 (cit. a p. 68).

[37] Marina Sokolova e Guy Lapalme. «A Systematic Analysis of Performance Measures for Classification Tasks». In: *Information Processing & Management* 45.4 (2009), pp. 427–437 (cit. a p. 69).

[38] Quinn McNemar. «Note on the Sampling Error of the Difference between Correlated Proportions or Percentages». In: *Psychometrika* 12.2 (1947), pp. 153–157 (cit. a p. 70).

[39] Thomas G. Dietterich. «Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms». In: *Neural Computation* 10.7 (1998), pp. 1895–1923 (cit. a p. 70).

[40] Bradley Efron e Robert J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993 (cit. a p. 70).

[41] Xavier Bouthillier et al. «Accounting for Variance in Machine Learning Benchmarks». In: *Proceedings of Machine Learning and Systems (MLSys)*. Vol. 3. 2021 (cit. a p. 70).

[42] Mark Horowitz. «1.1 Computing's Energy Problem (and What We Can Do about It)». In: *IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*. 2014, pp. 10–14. DOI: 10.1109/ISSCC.2014.6757323 (cit. a p. 73).

[43] Claude E. Shannon. «Communication in the Presence of Noise». In: *Proceedings of the IRE* 37.1 (1949), pp. 10–21 (cit. a p. 80).

[44] Min Liu e Tobi Delbruck. «Adaptive Time-Slice Block-Matching Optical Flow Algorithm for Dynamic Vision Sensors». In: *Proceedings of the British Machine Vision Conference (BMVC)*. 2018 (cit. a p. 84).

[45] Bernhard Hassenstein e Werner Reichardt. «Systemtheoretische Analyse der Zeit-, Reihenfolgen- und Vorzeichenauswertung bei der Bewegungsperzeption des Rüsselkäfers Chlorophanus». In: *Zeitschrift für Naturforschung B* 11.9–10 (1956), pp. 513–524 (cit. a p. 93).

[46] Jan P. H. van Santen e George Sperling. «Elaborated Reichardt Detectors». In: *Journal of the Optical Society of America A* 2.2 (1985), pp. 300–321 (cit. a p. 93).

[47] Alexander Borst e Thomas Euler. «Seeing Things in Motion: Models, Circuits, and Mechanisms». In: *Neuron* 71.6 (2011), pp. 974–994 (cit. a p. 93).

[48] Karthik H. Shankar e Marc W. Howard. «A Scale-Invariant Internal Representation of Time». In: *Neural Computation* 24.1 (2012), pp. 134–193 (cit. a p. 96).

[49] Ilyass Hammouamri, Ismail Khalfaoui-Hassani e Timothée Masquelier. «Learning Delays in Spiking Neural Networks using Dilated Convolutions with Learnable Spacings». In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. a p. 96).